Universidad de Sevilla

Escuela Técnica Superior de Ingeniería Informática

Departamento de Ciencias de la Computación e Inteligencia Artificial



Máster Universitario en Lógica, Computación e Inteligencia Artificial

Trabajo de Fin de Máster

Búsqueda en árbol de Monte Carlo descentralizada para la asignación de tareas multi-robot bajo incertidumbre

---

Dirigido por:
Ignacio Pérez
Hurtado de Mendoza

Realizado por:
Manuel
Olías López

Presentado el 21 de octubre de 2026

Resumen

Un equipo de robots que debe repartirse el trabajo se enfrenta a un problema engañosamente simple: decidir quién hace qué y en qué orden. Es la asignación de tareas multi-robot (*Multi-Robot Task Allocation*, MRTA) y la dificultad llega cuando el escenario se parece al mundo real: tareas que ningún robot puede completar en solitario, plazos que se cierran, energía finita que hay que ir a reponer y nada que dure exactamente lo previsto ni salga siempre bien. Esto último, la incertidumbre, da nombre a la variante del problema que aquí se aborda, **MRTAU** (*MRTA under Uncertainty*). Y si se prescinde del planificador central que lo ve todo, cada robot queda reducido a su información local y a lo que sus compañeros le comunican.

Este trabajo lleva ese problema a un esquema puramente distribuido. Formaliza primero su variante bajo incertidumbre como un DEC-POGSMDP, un modelo que dota a la decisión descentralizada de la lógica de eventos de los procesos semi-markovianos generalizados para tratar sin fisuras el tiempo continuo y la concurrencia. Sobre él construye un algoritmo de búsqueda en árbol de Monte Carlo descentralizado, Dec-MCTS, en el que cada robot planifica con su propio árbol y comunica a sus vecinos una distribución de probabilidad sobre sus planes, y lo enfrenta a cuatro algoritmos de referencia —dos sin comunicación y los métodos de pujas por consenso CBAA y CBBA—, leídos todos como políticas de comunicación con grados crecientes de coordinación.

La evaluación, sobre un catálogo de 360 escenarios diseñado para aislar un factor del problema cada vez y resuelto en $5\,400$ ejecuciones, confirma la hipótesis de partida: el algoritmo construido es el que mejor resuelve el problema, por delante de los cuatro algoritmos de referencia de forma consistente. El trabajo delimita además dónde esa ventaja se estrecha y qué precio paga por ella, y entrega el sistema completo con el que se ha obtenido.

Palabras Clave: asignación de tareas multi-robot, sistemas multi-robot distribuidos, planificación bajo incertidumbre, búsqueda en árbol de Monte Carlo, Dec-MCTS.

Abstract

A team of robots that has to share out the work faces a deceptively simple problem: deciding who does what and in what order. This is Multi-Robot Task Allocation (MRTA), and the difficulty arrives when the setting resembles the real world: tasks no robot can complete on its own, deadlines that expire, finite energy that has to be recharged and nothing that lasts exactly as long as expected or always succeeds. That last point, uncertainty, gives its name to the variant addressed here, MRTAU (MRTA under Uncertainty). And if the central planner that sees everything is dispensed with, each robot is left with its local information and whatever its teammates communicate to it.

This work takes that problem to a purely distributed scheme. It first formalizes its variant under uncertainty as a DEC-POGSMDP, a model that endows decentralized decision-making with the event logic of generalized semi-Markov processes in order to handle continuous time and concurrency seamlessly. On that model it builds a decentralized Monte Carlo tree search algorithm, Dec-MCTS, in which each robot plans with its own tree and communicates to its neighbors a probability distribution over its plans, and it pits that algorithm against four reference algorithms—two without communication and the consensus-based auction methods CBAA and CBBA—all of them read as communication policies with increasing degrees of coordination.

The evaluation, on a catalog of 360 scenarios designed to isolate one factor of the problem at a time and resolved in $5\,400$ runs, confirms the starting hypothesis: the algorithm built here is the one that best solves the problem, consistently ahead of the four reference algorithms. The work further delimits where that advantage narrows and what price it pays for it, and it delivers the complete system with which it was obtained.

Keywords: multi-robot task allocation, distributed multi-robot systems, planning under uncertainty, Monte Carlo tree search, Dec-MCTS.

Agradecimientos

Me gustaría expresar mi más sincero agradecimiento a mi tutor, Ignacio Pérez Hurtado de Mendoza, por guiarme en este proyecto. Fue suya la idea de este trabajo y sin él (y su repositorio de github) no hubiera podido llevarlo a cabo.

También me gustaría agradecer a mi familia, por acompañarme y apoyarme durante todo este tiempo.

Índice general

Resumen I

Abstract II

Agradecimientos III

1. Introducción **1**
1.1. Objetivos del trabajo 2
1.2. Aportaciones del trabajo 3
1.3. Estructura de la memoria 4

2. Marco Teórico **5**
2.1. Definición del problema 6
2.1.1. MRTA: el problema de asignación de tareas multi-robot 6
2.1.2. MRTAU: el problema bajo incertidumbre 9
2.1.3. Algoritmos *online* frente a *offline* 10
2.1.4. Algoritmos centralizados frente a descentralizados 11
2.2. Procesos de Decisión de Markov 12
2.2.1. Proceso de Decisión de Markov (MDP) 12
2.2.2. Proceso de Decisión de Markov Parcialmente Observable (POMDP) . 13
2.2.3. Proceso de Decisión Semi-Markoviano (SMDP) 15
2.2.4. Proceso de Decisión Semi-Markoviano Generalizado (GSMDP) . . . 16
2.2.5. Procesos de Decisión de Markov Descentralizados 19
2.3. El problema MRTAU distribuido como DEC-POGSMDP 21
2.3.1. MRTA como MDP 22
2.3.2. MRTAU como GSMDP 23
2.3.3. DEC-MRTAU como DEC-POGSMDP 25
2.4. Algoritmos propuestos 27
2.4.1. *Baselines*: Greedy y Random 28
2.4.2. Algoritmos basados en pujas: CBAA y CBBA 30
2.4.3. Monte Carlo Tree Search Descentralizado (Dec-MCTS) 35

3. Estado del arte **46**
3.1. El problema MRTA y su evolución 46
3.2. Formalización del problema: modelos de decisión 47
3.3. Métodos de resolución 48
3.3.1. Programación dinámica exacta 49
3.3.2. Enfoques basados en mercado 49
3.3.3. Planificación basada en muestreo y búsqueda en árbol 50
3.4. Asignación de tareas bajo incertidumbre 51
3.5. Posicionamiento y aportaciones 51

4. Materiales y métodos **53**
4.1. Materiales y entorno de ejecución 53
4.2. Organización del software 54
4.2.1. La simplificación de la comunicación 55
4.2.2. El registro de la ejecución 56
4.3. La función de recompensa 57
4.4. El simulador distribuido de eventos discretos 57
4.4.1. La cola de eventos 58
4.4.2. Hiperparámetros del motor de simulación 59
4.5. Dec-MCTS: decisiones e hiperparámetros de implementación 60
4.5.1. Refinamientos sobre el esquema base 60
4.5.2. La selección de hiperparámetros 61

5. Casos de estudio **63**
5.1. Configuración común de los escenarios 63
5.2. Los cinco bloques 67
5.2.1. Bloque A: El tamaño del equipo 69
5.2.2. Bloque B: La ventana de ejecución 69
5.2.3. Bloque C: La carga por robot 69
5.2.4. Bloque D: Las coaliciones 70
5.2.5. Bloque E: El nivel de incertidumbre 71

6. Experimentación y pruebas **72**
6.1. Protocolo experimental y criterios de análisis 72
6.2. Resultados sobre el conjunto del catálogo 73
6.2.1. Recompensa media 73
6.2.2. La diferencia entre CBBA y Dec-MCTS 74
6.2.3. Sensibilidad del agregado a la composición del catálogo 76
6.3. Resultados por bloque 77
6.3.1. Bloque A: El tamaño del equipo 77
6.3.2. Bloque B: La forma de la ventana de ejecución 78
6.3.3. Bloque C: La carga por robot 80
6.3.4. Bloque D: Las coaliciones 81
6.3.5. Bloque E: El nivel de incertidumbre 81
6.4. Análisis de magnitudes secundarias 82
6.4.1. Robots que agotan la batería 83
6.4.2. Descomposición de la recompensa 83
6.4.3. Distancia, uso del horizonte y coste de cómputo 85
6.5. Síntesis de los resultados 86

7. Conclusiones y trabajo futuro **89**
7.1. Logros y aportaciones principales 90
7.2. Limitaciones 91
7.3. Trabajo futuro 92

Capítulo 1

Introducción

Un almacén en el que una serie de vehículos autónomos traslada mercancía, una red de camiones que tienen que realizar un número de entregas o la intervención de un equipo de máquinas en una zona afectada por un desastre son situaciones distintas que comparten una misma estructura de decisión. En ninguna de ellas hay una única tarea que ejecutar, sino un conjunto de trabajos que repartir entre varios agentes: algunos exigen la presencia simultánea de varias máquinas, cada uno tiene su momento oportuno y ninguno dispone de energía ilimitada. Decidir quién hace qué, en qué orden y cuándo interrumpir la actividad para reponer energía es, en esencia, el problema del que se ocupa este trabajo.

Ese problema se conoce en la literatura como asignación de tareas multi-robot (*Multi-Robot Task Allocation*, MRTA) [12], y las particularidades anteriores lo sitúan en una de sus variantes más exigentes: las coaliciones introducen dependencias entre las planificaciones de robots distintos [19] y las ventanas temporales lo emparentan con el problema de rutas de vehículos con ventanas de tiempo [28]. Salvo en su forma más elemental el problema es NP-duro [12], y es la acumulación simultánea de requisitos la que hace inadecuados los enfoques diseñados para variantes más simples [17]. A ello se añade que la formulación habitual es determinista, hipótesis que un sistema robótico real no sostiene: las tareas no duran exactamente lo previsto, pueden fracasar y su consumo de energía no se conoce con exactitud. Bajo esa incertidumbre un plan fijo deja de ser una solución aceptable, pues un retraso puede agotar la ventana de una tarea y un consumo mayor del previsto inmovilizar a un robot, y la solución pasa a ser una política que decide la siguiente acción a la vista de lo que efectivamente ha ocurrido [8].

Queda una segunda decisión, sobre la arquitectura de control. El modelo del que parte este trabajo [25] resuelve el problema de forma *centralizada*: un planificador único con acceso al estado de todos los robots y todas las tareas calcula la asignación del equipo completo, con las limitaciones conocidas de esa opción, esto es, un punto único de fallo, una escalabilidad pobre y la necesidad de comunicación fiable con todos ellos. En un esquema *puramente distribuido* cada robot decide, en cambio, a partir de su información local y una comunicación establecida entre él y sus compañeros. Esta arquitectura gana en robustez y en escalabilidad, y paga la pérdida del estado global, pues la coordinación ya no se impone desde fuera, sino que debe

emerger de la comunicación. Ahí residen el interés y la dificultad de este trabajo: llevar a un esquema así un método de planificación que necesita razonar sobre lo que hará el equipo entero, cuando cada robot solo puede observarse a sí mismo.

1.1. Objetivos del trabajo

El objetivo principal de este trabajo es la **construcción de un algoritmo de búsqueda en árbol de Monte Carlo descentralizado** (Dec-MCTS) que resuelva el problema anterior en un esquema puramente distribuido, en el que cada robot mantiene su propio árbol de búsqueda y comunica a sus vecinos una distribución de probabilidad sobre sus planes más prometedores. El punto de partida es el algoritmo de Best et al. [5], formulado para un problema de percepción activa que comparte con el abordado aquí su estructura esencial (varios agentes que planifican secuencias de acciones para optimizar un objetivo común sin conocer los planes ajenos), pero cuyos rasgos concretos son completamente distintos, pues no contempla coaliciones, ni ventanas de ejecución, ni restricciones de energía, ni la posibilidad de que una tarea fracase. Trasladarlo a este problema obliga a redefinir el espacio de acciones, el modelo generativo con el que se simulan los episodios, el criterio con que se puntúa un plan y la información que se comunica. Este es el núcleo del trabajo y lo que más esfuerzo ha requerido.

Sobre ese algoritmo se formula la **hipótesis de partida**: que Dec-MCTS, el de mayor grado de coordinación de los considerados, debe rendir por encima de los algoritmos distribuidos de referencia. Cabe esperarlo porque la información que sus robots intercambian es más rica que la de cualquier otro de los métodos comparados, de modo que debería ser el que mejor aproveche una comunicación acotada. Contrastarla exige, además de construir el algoritmo, disponer de todo lo necesario para medirla, lo que se traduce en cuatro objetivos secundarios:

- **Formalizar** el problema distribuido con un modelo de decisión que trate a la vez el tiempo continuo, la duración estocástica de las acciones y la información parcial de la que dispone cada agente.
- **Desarrollar** un simulador distribuido de eventos discretos en el que cada robot ejecute su propio algoritmo sin acceso al estado global, e implementar sobre una interfaz común los cuatro algoritmos de referencia: dos *baselines* sin comunicación y los algoritmos de pujas por consenso CBAA y CBBA [7], métodos ya establecidos que aquí se implementan a partir de su formulación original.

- **Construir un conjunto de prueba** que no se limite a ordenar los algoritmos, sino que permita atribuir las diferencias observadas a factores concretos del problema.
- **Evaluar** los cinco algoritmos sobre ese conjunto y caracterizar las condiciones bajo las cuales su rendimiento cambia, delimitando dónde el método propuesto rinde y dónde deja de hacerlo.

1.2. Aportaciones del trabajo

La aportación principal es el propio algoritmo descentralizado descrito en la Sección 2.4.3, cuya construcción constituye un resultado en sí mismo con independencia de la comparación posterior, ya que supone rehacer un método concebido para otro problema hasta convertirlo en uno ejecutable para este. El resultado experimental, además, la respalda: sobre el catálogo completo, el algoritmo construido resulta ser el que mejor resuelve el problema, por delante de los cuatro algoritmos de referencia.

Junto a ella, el trabajo realiza dos aportaciones conceptuales. La primera es la formalización del problema distribuido como un **DEC-POGSMDP** (Sección 2.3.3), un modelo que dota a un proceso de decisión descentralizado y parcialmente observable del mecanismo de eventos y relojes ocultos de los procesos semi-markovianos generalizados, y que describe con ello a la vez el tiempo continuo, la duración estocástica de las acciones y la información local de cada agente, evitando que la política de un robot se condicione a desenlaces que aún no se han producido. La segunda es la lectura unificada de los cinco algoritmos como políticas de comunicación que resuelven ese mismo modelo con grados crecientes de coordinación (Sección 2.4), desde la ausencia total de comunicación hasta el intercambio de distribuciones sobre planes completos. Esta lectura atribuye las diferencias de rendimiento a la riqueza de la información intercambiada, y los resultados la respaldan, ya que la ordenación medida reproduce la prevista.

El trabajo entrega, por último, el sistema completo con el que se obtienen los resultados (el simulador distribuido, los cinco algoritmos y las herramientas de análisis), un catálogo de escenarios construido expresamente para aislar factores del problema y una caracterización de la ventaja obtenida: las condiciones en las que se reduce, el único grupo de escenarios en el que el algoritmo no lidera y el precio que paga en recorrido y en cómputo.

1.3. Estructura de la memoria

El resto de la memoria se organiza como sigue. El Capítulo 2 establece los fundamentos formales: define el problema, introduce de forma incremental la familia de los procesos de decisión de Markov hasta el modelo que captura todas sus características, formaliza el problema sobre ese modelo y presenta los cinco algoritmos comparados. El Capítulo 3 sitúa esa formalización y esos algoritmos frente a la literatura existente, distingue el plano de la formalización del problema del de su resolución y organiza las contribuciones previas en familias.

El Capítulo 4 describe cómo se ha materializado todo lo anterior en un sistema ejecutable: los materiales empleados, la organización del software, el simulador distribuido de eventos discretos, la función de recompensa y las decisiones e hiperparámetros de implementación de Dec-MCTS. El Capítulo 5 presenta el conjunto de prueba sobre el que se ejecuta ese sistema, con la configuración común a todos los escenarios y los cinco bloques en que se reparten, cada uno de los cuales varía un único aspecto del problema mientras el resto permanece fijado.

El Capítulo 6 recoge los resultados de la evaluación: fija primero los criterios de comparación y los aplica después al catálogo como un único conjunto, a su desglose por bloques y a magnitudes distintas de la métrica objetivo. El Capítulo 7 valora qué aportan esos resultados al objetivo planteado, delimita las limitaciones del trabajo y el alcance de lo que puede afirmarse, y expone las líneas de continuación que los resultados abren.

La memoria incluye también una bibliografía con las referencias que han sido fundamentales para el desarrollo del trabajo.

Capítulo 2

Marco Teórico

Este capítulo establece los fundamentos formales sobre los que se asienta el resto del trabajo. El objetivo es triple: definir con precisión el problema que se aborda, dotarse de un modelo matemático adecuado para razonar sobre él y, con ese modelo, presentar los algoritmos que se compararán en el resto de la memoria.

La Sección 2.1 define el problema "desde fuera", en términos de las *entradas* que recibe un algoritmo de asignación y de las *salidas* que se esperan de él, sin recurrir todavía a ningún formalismo. Se parte del problema clásico de asignación de tareas multi-robot (MRTA) y se enriquece con las particularidades del escenario abordado (coaliciones, ventanas de ejecución, heterogeneidad y restricciones de batería) hasta llegar a su versión bajo incertidumbre (MR-TAU). La sección concluye introduciendo las dos dimensiones transversales que clasifican los enfoques de resolución: el momento del cómputo (*online* frente a *offline*) y la arquitectura de control (centralizada frente a descentralizada).

La Sección 2.2 proporciona el modelado matemático. Se introduce, de forma incremental, la familia de los Procesos de Decisión de Markov: desde el modelo básico (MDP) hasta la variante semi-Markoviana generalizada, descentralizada y parcialmente observable que acabará capturando todas las características del problema. Cada modelo relaja o extiende una hipótesis del anterior, lo que motiva de forma natural la transición de uno al siguiente.

La Sección 2.3 enlaza ambas piezas, formalizando el problema definido al principio como una instancia de los modelos anteriores. La correspondencia se construye también de manera incremental, añadiendo una hipótesis cada vez: la versión determinista se modela como un MDP, la versión bajo incertidumbre como un GSMDP y, finalmente, la versión distribuida —núcleo de este trabajo— como un DEC-POGSMDP.

Por último, la Sección 2.4 presenta los algoritmos propuestos, que pueden leerse de forma unificada como distintas *políticas de comunicación* que resuelven el mismo DEC-POGSMDP con grados crecientes de coordinación: desde los *baselines* sin comunicación (Random y Greedy), pasando por los métodos de mercado basados en pujas (CBAA y CBBA), hasta la planificación distribuida que comunica distribuciones sobre los planes (Dec-MCTS).

2.1. Definición del problema

Antes de elegir un formalismo con el que razonar sobre el problema conviene describirlo de forma autocontenida, en términos de las *entradas* que recibe un algoritmo de asignación (la descripción de un escenario) y de las *salidas* que se esperan de él (una solución válida y su calidad). En esta sección se presenta primero el problema de asignación de tareas multi-robot en su forma clásica (Sección 2.1.1) y, sobre él, las particularidades que caracterizan el escenario abordado en este trabajo, incluyendo las fuentes de incertidumbre (Sección 2.1.2). Posteriormente se discuten dos dimensiones transversales que clasifican los enfoques de resolución: el momento del cómputo (Sección 2.1.3) y la arquitectura de control (Sección 2.1.4).

2.1.1. MRTA: el problema de asignación de tareas multi-robot

El problema de asignación de tareas multi-robot (*Multi-Robot Task Allocation*, MRTA) consiste, en su forma más elemental, en distribuir un conjunto de tareas $\mathcal{T} = \{T_1, \ldots, T_m\}$ entre un conjunto de robots $\mathcal{R} = \{R_1, \ldots, R_n\}$ de modo que se optimice una medida de rendimiento del equipo (ya sea el tiempo de ejecución, el número de tareas resueltas, la distancia recorrida o cualquier combinación de las mismas) [12].

La taxonomía de Gerkey y Matarić [12] organiza las variantes de MRTA según tres ejes: el número de tareas que un robot puede ejecutar simultáneamente (*single-task*, ST, frente a *multi-task*, MT), el número de robots que requiere cada tarea (*single-robot*, SR, frente a *multi-robot*, MR) y el horizonte de la asignación (*instantaneous assignment*, IA, frente a *time-extended assignment*, TA). Trabajos posteriores ampliaron esta clasificación para capturar las dependencias entre tareas y agentes [19] y las restricciones temporales y de orden [21].

El problema abordado en este trabajo se basa en el escenario propuesto por Pérez-Hurtado et al. [25]. Es una versión enriquecida de MRTA con las siguientes particularidades:

- **Robots *single-task* (ST).** Los robots pueden ejecutar una única tarea en un determinado instante de tiempo.
- **Tareas *multi-robot* (MR).** Cada tarea T_j cuenta con un número requerido de robots para su ejecución (*workers*) $q_j \in \{1, \ldots, n\}$. Para completarse, q_j robots compatibles deben coincidir en su ubicación durante un intervalo común. Esta característica introduce dependencias entre las planificaciones de distintos robots y sitúa el problema, en la

taxonomía de Korsah et al. [19], en la clase de dependencias entre-planificaciones (*cross-schedule dependencies*).

- **Asignación con horizonte temporal (TA).** Los robots no reciben una única tarea, sino que planifican *secuencias* de tareas a lo largo del tiempo. Los robots cuentan con acceso completo a la información del escenario incluyendo el conjunto completo de tareas a ejecutar y cómo estas tareas se irán "abriendo" y "cerrando" con el tiempo. De esta manera, cada robot puede planificar una ruta, aunque no se comprometa a ella, lo que convierte el problema en un problema de enrutamiento y planificación cercano al *Vehicle Routing Problem* [28].
- **Ventanas de ejecución.** Cada tarea T_j solo puede ejecutarse dentro de un intervalo temporal $[t_j^s, t_j^l]$, donde t_j^s representa el instante más temprano en que puede iniciarse una tarea y t_j^l es el instante límite para completar la tarea. La propiedad anterior junto a esta convierten el problema en uno aún más similar a *Vehicle Routing Problem with Time Windows* (VRPTW) [28].
- **Listas de compatibilidades (heterogeneidad).** Los robots son heterogéneos: cada robot r_i posee un conjunto de compatibilidades, es decir, tareas que puede ejecutar. Esta tabla de compatibilidades se puede modelar como una función binaria $\mathit{Cap} : \{(i,j) \in \mathbb{N} \times \mathbb{N} \mid 1 \leq i \leq n, 1 \leq j \leq m\} \to \{0,1\}$ donde $\mathit{Cap}(i,j) = 1$ si el robot R_i puede ejecutar la tarea T_j y $\mathit{Cap}(i,j) = 0$ en caso contrario.
- **Restricciones de batería y recarga.** Cada robot R_i dispone de un nivel de batería b_i (limitado por una capacidad máxima B_i) que se consume al desplazarse y al ejecutar tareas. Para evitar el agotamiento, los robots pueden acudir a un conjunto de estaciones de recarga $\mathcal{S}$, lo que añade decisiones logísticas adicionales al plan. El consumo de batería se puede modelar dentro de la función de utilidad y las estaciones de recarga se pueden considerar como tareas con dependencias especiales.

Estas características permiten especificar formalmente una instancia del problema y el concepto de solución.

Definición 2.1 (Instancia de MRTA)**.** Una instancia del problema MRTA propuesto viene dada por la tupla $\langle \mathcal{R}, \mathcal{T}, \mathcal{S}, \mathit{Cap}, d \rangle$, donde:

- $\mathcal{R}$ es el conjunto de robots, donde cada robot $R_i \in \mathcal{R}$ se describe por su ubicación inicial x_i^0, su capacidad de batería B_i, su velocidad de desplazamiento v_i y su tasa de consumo de batería durante el desplazamiento *rate*-R_i.

- $\mathcal{T}$ es el conjunto de tareas, donde cada tarea $t_j \in \mathcal{T}$ se describe por su ubicación x_j, su duración d_j, su ventana de ejecución $[t_j^s, t_j^l]$, su número de *workers* requerido q_j, su recompensa w_j y su tasa de consumo de batería durante la ejecución *rate*-T_j.
- $\mathcal{S}$ es el conjunto de estaciones de recarga, donde cada estación $S_k \in \mathcal{S}$ viene dada por su ubicación x_k y su tasa de recarga de batería *rate*-S_k.
- *Cap*$(\cdot, \cdot)$ es la función de compatibilidad.
- $d(\cdot, \cdot)$ es la función que asigna a cada par de ubicaciones la distancia entre ellas.

Definición 2.2 (Solución y factibilidad)**.** Una solución es un **plan** π que asigna a cada robot una secuencia ordenada de tareas y operaciones de recarga, junto con los instantes en que las emprende. La solución es **factible** si satisface simultáneamente:

1. *Compatibilidad*: todo robot asignado a una tarea es compatible con ella.
2. *Coalición*: cada tarea ejecutada cuenta con exactamente q_j robots compatibles presentes durante un intervalo de ejecución.
3. *Ventanas*: la ejecución de cada tarea respeta su intervalo $[t_j^s, t_j^l]$.
4. *Energía*: el nivel de batería de cada robot se mantiene no negativo a lo largo de toda su ruta, considerando consumos y recargas.
5. *Secuenciación*: cada robot ejecuta una sola tarea a la vez, de forma coherente con los tiempos de desplazamiento.

La calidad de una solución factible se evalúa mediante una **función de utilidad** de naturaleza cooperativa y multiobjetivo, que premia el número de tareas completadas y el número de robots que preservan su integridad (no agotan la batería), y penaliza el tiempo total de ejecución (*makespan*) y el número de tareas que quedan pendientes. Una formulación habitual es

$$U(\pi) \;=\; \lambda_c \sum_{T_j \in \mathcal{T}_{\text{comp}}} w_j \;-\; \lambda_p \,|\mathcal{T}_{\text{pend}}| \;+\; \lambda_s \, n_{\text{surv}} \;-\; \lambda_t \, C_{\text{máx}}, \tag{2.1}$$

donde $\mathcal{T}_{\text{comp}}$ y $\mathcal{T}_{\text{pend}}$ son los conjuntos de tareas completadas y pendientes, n_{surv} es el número de robots que finalizan sin agotar su batería, $C_{\text{máx}}$ es el *makespan* y $\lambda_c, \lambda_s, \lambda_t, \lambda_p \geq 0$ ponderan los distintos objetivos, con $\lambda_c + \lambda_s + \lambda_t + \lambda_p = 1$. El objetivo del algoritmo es, por tanto, hallar el plan π^* que maximice U.

2.1.2. MRTAU: el problema bajo incertidumbre

La formulación anterior es *determinista*: supone que los tiempos de desplazamiento, las duraciones de las tareas, el consumo de energía y el éxito de cada ejecución son conocidos de antemano y se cumplen con exactitud. En un sistema robótico real esta hipótesis no se sostiene. La variante *MRTAU* (MRTA *under Uncertainty*) añade a la Definición 2.1 las siguientes fuentes de incertidumbre:

1. **Resultado de tareas incierto.** La ejecución de una tarea puede fracasar: tras comenzar la ejecución de una tarea T_j, ésta se completa con éxito con cierta probabilidad ρ_j y fracasa con probabilidad $1 - \rho_j$.

2. **Duraciones de tarea estocásticas.** La duración de ejecutar una tarea es igualmente aleatoria. Las tareas cuentan con una distribución de probabilidad de tiempo de ejecución $\mathcal{N}(\mu_t^s, \sigma_t^s)$ en caso de éxito y una distinta $\mathcal{N}(\mu_t^f, \sigma_t^f)$ en caso de fracaso.

3. **Consumo de batería estocástico.** Debido a la estocasticidad de los tiempos de ejecución de las tareas, el consumo de batería durante la ejecución también está sujeto a variabilidad, lo que vuelve incierta la viabilidad energética de cualquier tarea.

La consecuencia de introducir estas fuentes de incertidumbre es profunda y condiciona todo el trabajo: una solución entendida como un plan fijo calculado de antemano deja de ser adecuada. Un plan que parezca factible respecto a los valores nominales puede violar una ventana de ejecución porque un desplazamiento tardó más de lo previsto, o dejar a un robot sin batería por un consumo mayor del esperado. En consecuencia, el concepto de solución se desplaza de un *plan estático* a una *política adaptativa*: una regla que decide la siguiente acción de cada robot en función de los resultados estocásticos que se van observando. Esta observación motiva tanto el empleo de algoritmos que computan durante la ejecución (Sección 2.1.3) como la formalización del problema mediante procesos de decisión secuencial bajo incertidumbre, que se aborda en las Secciones 2.2 y 2.3.

En este nuevo paradigma, es indispensable establecer una diferencia entre una política y un episodio, y consecuentemente, entre la función de utilidad y la función de recompensa:

- **Episodio (ξ):** Representa una realización concreta del sistema *multi-robot* desde el estado inicial hasta un posible estado final. Debido a la naturaleza estocástica del entorno,

la ejecución secuencial de una misma política dará lugar a múltiples episodios posibles, cada uno con diferentes duraciones de tareas o fallos imprevistos. Cada episodio individual se evalúa mediante una **función de recompensa** del episodio $R(\xi)$:

$$R(\xi) \;=\; \lambda_c \sum_{T_j \in \mathcal{T}_{\text{comp}}} w_j \;-\; \lambda_p \,|\mathcal{T}_{\text{pend}}| \;+\; \lambda_s \, n_{\text{surv}} \;-\; \lambda_t \, C_{\text{máx}}, \tag{2.2}$$

Al depender de la incertidumbre, $R(\xi)$ se comporta como una variable aleatoria.

- **Política (π):** Representa la estrategia o reglas de decisión que rigen las decisiones de los robots. Una política no puede evaluarse justamente a través de un único episodio aislado debido a la variabilidad del entorno. En su lugar, se evalúa mediante la **función de utilidad** $U(\pi)$, la cual se define como la esperanza matemática de las recompensas de todos los episodios que dicha política es capaz de generar:

$$U(\pi) \;=\; \mathbb{E}_{\xi \sim \pi}\left[R(\xi)\right] \tag{2.3}$$

donde la esperanza se calcula sobre la distribución de probabilidad de las trayectorias ξ inducidas por la política π. El objetivo en el problema MRTAU se convierte, por tanto, en hallar la política óptima π^* que maximice la utilidad esperada $U(\pi)$.

2.1.3. Algoritmos *online* frente a *offline*

Definido el problema, conviene introducir dos criterios que clasifican los enfoques de resolución. El primero concierne a *cuándo* se calcula la solución en relación con su ejecución. Un algoritmo ***offline*** (o de planificación *a priori*) construye la solución completa antes de que comience la ejecución, asumiendo un modelo completo y estático del problema. Una vez calculada, la solución se ejecuta sin modificaciones. Este paradigma es adecuado cuando el entorno es determinista y conocido, y cuando el coste de la planificación puede asumirse fuera del bucle de ejecución. Su limitación fundamental es la fragilidad ante la incertidumbre: cualquier desviación entre el modelo empleado para planificar y la realidad observada arruina la solución calculada a priori.

Un algoritmo ***online*** (o reactivo), por el contrario, intercala el cómputo con la ejecución: las decisiones se toman de manera adaptativa a medida que se dispone de nueva información. Esto permite reaccionar ante los eventos estocásticos descritos en la Sección 2.1.2 (el fallo de una tarea, un retraso o un consumo imprevisto) replanificando con el estado más reciente.

La contrapartida es que el tiempo de cómputo en cada decisión está acotado, por lo que estos métodos se apoyan en heurísticas o en algoritmos de tipo *anytime* debidamente optimizados, capaces de devolver una solución factible en cualquier instante y de mejorarla si se les concede más tiempo. El algoritmo de búsqueda *Monte Carlo Tree Search* (MCTS) es el ejemplo perfecto de método *online anytime* [6].

Dado que el problema MRTAU combina incertidumbre y tiempos continuos, un enfoque puramente *offline* resulta inviable: el espacio de contingencias posibles crece de forma combinatoria y no puede enumerarse por anticipado. Por ello, los algoritmos considerados en este trabajo son esencialmente *online*, en línea con el modelo base de Pérez-Hurtado et al. [25].

2.1.4. Algoritmos centralizados frente a descentralizados

El segundo criterio de clasificación concierne a *quién* toma las decisiones y de *qué* información dispone para hacerlo. En una arquitectura **centralizada** existe una entidad coordinadora (un planificador único) con acceso al estado de todos los robots y todas las tareas, que calcula la asignación para el conjunto de robots completo y comunica a cada robot las acciones que debe ejecutar. Su ventaja teórica es la posibilidad de alcanzar la solución óptima global, al razonar sobre información completa. Sus inconvenientes son conocidos: constituye un único punto de fallo, escala de forma deficiente con el número de agentes (el espacio de decisiones conjunto crece exponencialmente) y exige comunicación fiable y de baja latencia con todos los robots.

En una arquitectura **descentralizada** (o distribuida) no existe tal entidad: cada robot es un agente autónomo que decide sus acciones a partir de su información *local* (su propio estado y, eventualmente, los mensajes que recibe de sus vecinos), sin acceso directo al estado interno de los demás. La coordinación, cuando es necesaria, emerge del intercambio explícito de información. Esta arquitectura es intrínsecamente robusta, escala mejor y se adapta a escenarios con comunicación limitada o intermitente. Su precio es la pérdida de la garantía de optimalidad global: al razonar sobre información parcial, los agentes pueden tomar decisiones localmente racionales pero globalmente subóptimas, y pueden surgir conflictos (por ejemplo, dos robots que se dirigen a la misma tarea) que deben resolverse mediante mecanismos de consenso [9].

La Tabla 2.1 resume estos compromisos. El presente trabajo adopta deliberadamente el paradigma descentralizado: partiendo del modelo centralizado de Pérez-Hurtado et al. [25],

Cuadro 2.1: Compromisos entre arquitecturas de control para MRTA.

Criterio	**Centralizado**	**Descentralizado**
Información disponible	Estado global	Estado local + mensajes
Optimalidad	Óptimo global posible	Subóptimo (en general)
Robustez ante fallos	Punto único de fallo	Tolerante a fallos
Escalabilidad	Pobre	Buena
Coste de comunicación	Alto y centralizado	Distribuido y acotado

el objetivo es transitar hacia una resolución puramente distribuida. Como se justifica en la Sección 2.3, esta transición no es un mero detalle de implementación: la ausencia de observabilidad global cambia la naturaleza formal del problema y obliga a sustituir la información compartida por mecanismos de coordinación basados en comunicación.

2.2. Procesos de Decisión de Markov

La toma de decisiones secuencial bajo incertidumbre se modela habitualmente mediante la familia de los Procesos de Decisión de Markov (*Markov Decision Process*). En esta sección se introducen, de forma incremental, los cinco modelos relevantes para este trabajo. Cada uno relaja o extiende una hipótesis del anterior: partiendo de un modelo inicial MDP (Sección 2.2.1), el POMDP elimina la observabilidad total (Sección 2.2.2), el SMDP elimina el tiempo discreto (Sección 2.2.3). En este punto se planteará el problema fundamental que surge al aplicar el SMDP a un escenario con acciones concurrentes y continuas (el *leak temporal*), el GSMDP surge como una respuesta al problema (Sección 2.2.4). Por último, el DEC-POSMDP combina todas estas características dentro de un contexto multi-agente descentralizado (Sección 2.2.5). Esta progresión guiará la formalización de la Sección 2.3.

2.2.1. Proceso de Decisión de Markov (MDP)

El *Proceso de Decisión de Markov* es el modelo fundamental para la toma de decisiones secuencial en entornos estocásticos pero completamente observables [3, 26].

Definición 2.3 (MDP)**.** Un Proceso de Decisión de Markov es una tupla $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma \rangle$ donde:

- $\mathcal{S}$ es un conjunto finito de **estados**.
- $\mathcal{A}$ es un conjunto finito de **acciones**.

- $\mathcal{T} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ es la **función de transición**, donde $\mathcal{T}(s, a, s') = P(s_{k+1} = s' \mid s_k = s, a_k = a)$ define la probabilidad de alcanzar el estado s' al aplicar la acción a desde el estado s.
- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ es la **función de recompensa** que atribuye un valor real $\mathcal{R}(s, a)$ a cada acción a tomada en un determinado estado s, ayudando al agente a priorizar aquellas acciones que maximicen su beneficio a largo plazo.
- $\gamma \in [0, 1]$ es el **factor de descuento**, que pondera la importancia de las recompensas futuras frente a las inmediatas. Un valor $\gamma \to 0$ indica a los agentes que prioricen recompensas inmediatas mientras que un valor $\gamma \to 1$ favorece las recompensas futuras.

La hipótesis central que da nombre al modelo es la **propiedad de Markov**: la distribución del estado siguiente depende únicamente del estado y la acción actuales, y no de la historia previa,

$$P(s_{k+1} \mid s_k, a_k, \ldots, s_0, a_0) = P(s_{k+1} \mid s_k, a_k). \tag{2.4}$$

En este contexto, una solución a este problema es una **política**, esto es, una función $\pi : \mathcal{S} \to \mathcal{A}$ que, dado un estado, devuelva una acción a aplicar.

El objetivo es hallar una política que maximice la recompensa acumulada esperada. La calidad de una política se mide mediante la **función de utilidad** U_π, que satisface la ecuación de Bellman [3]:

$$U_\pi(s) = \mathcal{R}(s, \pi(s)) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{T}(s, \pi(s), s')\, U_\pi(s'). \tag{2.5}$$

La política óptima π^* es la que maximiza $U_\pi(s)$ en todo estado. Dos hipótesis implícitas del MDP serán relajadas más adelante: (i) el agente observa el estado completo en cada instante y (ii) las transiciones ocurren en pasos de tiempo discretos y unitarios. Por ello es necesario introducir los siguientes modelos.

2.2.2. Proceso de Decisión de Markov Parcialmente Observable (POMDP)

En numerosos problemas el agente no conoce directamente el estado del sistema, sino observaciones parciales y posiblemente ruidosas a partir de las cuales debe inferirlo. El **POMDP** (*Partially Observable Markov Decision Process*) extiende el MDP para capturar esta limitación [16].

Definición 2.4 (POMDP)**.** Un Proceso de Decisión de Markov Parcialmente Observable es una tupla $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \Omega, \mathcal{O}, \gamma \rangle$ donde:

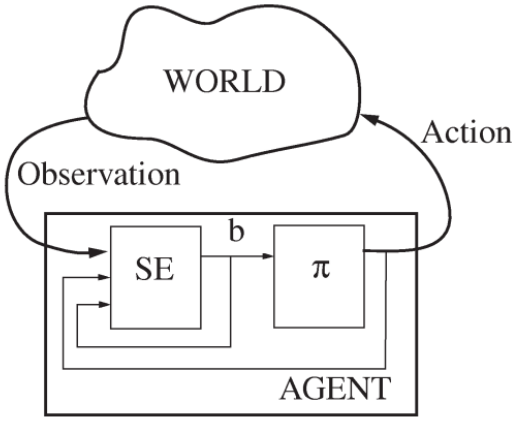


Figura 2.1: En un POMDP, un agente se puede descomponer en un *estimador de estado* (SE) y una *política* (π).

- $\mathcal{S}$, $\mathcal{A}$, $\mathcal{T}$, $\mathcal{R}$ y γ describen un MDP (Definición 2.3).
- Ω, un conjunto finito de todas las posibles **observaciones**.
- $\mathcal{O} : \mathcal{S} \times \mathcal{A} \times \Omega \to [0, 1]$, la **función de observación**, donde $\mathcal{O}(s', a, o) = P(o_{k+1} = o \mid s_{k+1} = s', a_k = a)$ es la probabilidad de percibir la observación o una vez se alcanza el estado s' tras aplicar la acción a.

El agente no conoce el estado actual del entorno, por lo que mantiene una **creencia** (*belief*) $b \in \Delta(\mathcal{S})$: una distribución de probabilidad sobre los estados posibles. La creencia es un estadístico suficiente de toda la historia de acciones y observaciones, y se actualiza de forma bayesiana tras cada par (a, o):

$$b'(s') = \eta\, \mathcal{O}(s', a, o) \sum_{s \in \mathcal{S}} \mathcal{T}(s, a, s')\, b(s), \tag{2.6}$$

donde η es un factor de normalización que garantiza que la suma de probabilidades sea 1. Esta actualización combina la creencia a priori de un agente sobre el entorno dinámico $\mathcal{T}(s, a, s')\, b(s)$ con la nueva observación obtenida $\mathcal{O}(s', a, o)$, permitiendo al agente refinar progresivamente su conocimiento sobre el estado real. Un agente se puede descomponer en dos partes, como muestra la Figura 2.1.

El objetivo de un agente en un POMDP, al igual que en un MDP, es encontrar una política óptima π^* que maximice la recompensa acumulada esperada. Sin embargo, en este caso, la política se define sobre los estados de creencia y no sobre los estados físicos del sistema, ya que el agente no puede observarlos directamente. Un POMDP puede reformularse como un

MDP sobre el espacio de creencias (*belief-MDP*); el inconveniente es que dicho espacio es continuo y de dimensión $|\mathcal{S}| - 1$, lo que hace que la resolución exacta sea intratable en el caso general [16]. De ahí que, para abordar esta dificultad, se han propuesto numerosas estrategias de aproximación.

2.2.3. Proceso de Decisión Semi-Markoviano (SMDP)

Los modelos anteriores asumen que el tiempo avanza en pasos discretos y que toda acción consume una unidad temporal. Esto es inadecuado cuando las acciones tienen duraciones variables y continuas, como desplazarse, ejecutar una tarea o recargar. El SMDP (*Semi-Markov Decision Process*) generaliza el MDP permitiendo que el tiempo de permanencia en cada estado (o el tiempo de transición entre estados; son dos maneras de interpretar un mismo formalismo) sea una variable aleatoria de valor real [14].

Definición 2.5 (SMDP)**.** Un Proceso de Decisión Semi-Markoviano es una tupla $\langle \mathcal{S}, \mathcal{A}, \mathcal{F}, \mathcal{R}, \gamma \rangle$ que generaliza la Definición 2.3 mediante una **distribución conjunta de transición y duración**

$$\mathcal{F}(s', \tau \mid s, a) = P(s_{k+1} = s',\, \tau_k \leq \tau \mid s_k = s, a_k = a), \tag{2.7}$$

que describe simultáneamente la probabilidad de transitar a s' y la distribución del tiempo continuo $\tau \in [0, +\infty)$ que tarda en producirse la transición. La función de transición discreta del MDP se recupera como la marginal $\mathcal{T}(s, a, s') = \lim_{\tau \to \infty} \mathcal{F}(s', \tau \mid s, a)$. Adicionalmente, se sustituye el factor de descuento discreto $\gamma \in (0, 1)$ por una **tasa de descuento continua** $\beta > 0$, que permite aplicar un descuento en función del tiempo de transición.

La denominación *Semi-Markoviano* surge de una relajación de la propiedad de Markov. En un MDP clásico, el futuro del sistema depende exclusivamente del estado actual. En un SMDP, dado que las acciones tienen una duración continua τ, el tiempo que el agente lleva ejecutando una acción puede afectar al estado resultante de la misma.

Por ejemplo, si un robot inicia la acción de trasladarse a un pasillo lejano (cuya duración típica es de 10 segundos) y se analiza el sistema transcurridos 9 segundos, la probabilidad de alcanzar el estado destino de forma inminente es mayor que si acabara de comenzar la acción. En consecuencia, la propiedad de Markov se viola durante el transcurso de la acción. Sin embargo, la propiedad de Markov se recupera por completo en los instantes de decisión

(es decir, en los momentos clave en los que una acción concluye, se alcanza un nuevo estado s' y el agente debe elegir la siguiente acción a).

Esta naturaleza continua altera la formulación de la **función de utilidad**. En el caso discreto, la ecuación de Bellman representa conceptualmente una esperanza matemática ($\mathbb{E}_{s'}$) sobre el conjunto discreto de estados futuros del sistema. La Ecuación (2.5) es equivalente a:

$$U_\pi(s) = \mathbb{E}_{s'}[\,\mathcal{R}(s,\pi(s)) + \gamma\, U_\pi(s') \mid s, \pi(s)\,] \tag{2.8}$$

Al transicionar al paradigma SMDP, el estado futuro ya no depende de un único paso temporal, sino de dos variables aleatorias conjuntas: el estado de destino s' (discreto) y el tiempo de transición τ (continuo). De este modo, la ecuación de Bellman para un SMDP bajo un horizonte descontado se establece como:

$$U_\pi(s) = \mathbb{E}_{s',\tau}\big[\,\mathcal{R}(s,\pi(s)) + e^{-\beta\tau}\, U_\pi(s') \,\big|\, s, \pi(s)\big]\,. \tag{2.9}$$

En esta fórmula, el factor de descuento se modela mediante una función exponencial negativa $e^{-\beta\tau}$.

No obstante, el SMDP asume implícitamente que en cada instante hay una única acción activa, cuya finalización constituye el siguiente instante de decisión. En un sistema *multi-robot* esta hipótesis se rompe (las acciones de distintos robots se solapan en el tiempo), y al intentar aplicar el SMDP en esas condiciones surge un problema que conviene plantear con precisión, pues condiciona todos los modelos posteriores.

2.2.4. Proceso de Decisión Semi-Markoviano Generalizado (GSMDP)

La fuga de información temporal (*leak temporal*). Considérese la situación de la Figura 2.2: el robot R_1 está ejecutando la tarea T_1, iniciada en un instante anterior, y en el instante de decisión t_k el robot R_2 queda libre y debe elegir su siguiente acción. La maquinaria del SMDP asocia a cada acción el núcleo $\mathcal{F}(s', \tau \mid s, a)$, que en el momento en que la acción comienza determina conjuntamente el estado de destino s' y la duración τ. Si esa duración (o, de forma equivalente, el instante de finalización ya muestreado) se almacena en el estado, entonces en t_k el estado contiene el instante exacto en que T_1 concluirá y R_1 se liberará. La política de R_2, que decide en función del estado global, accedería así a información sobre un evento futuro que en una ejecución real todavía no se ha producido. Esta fuga de información,

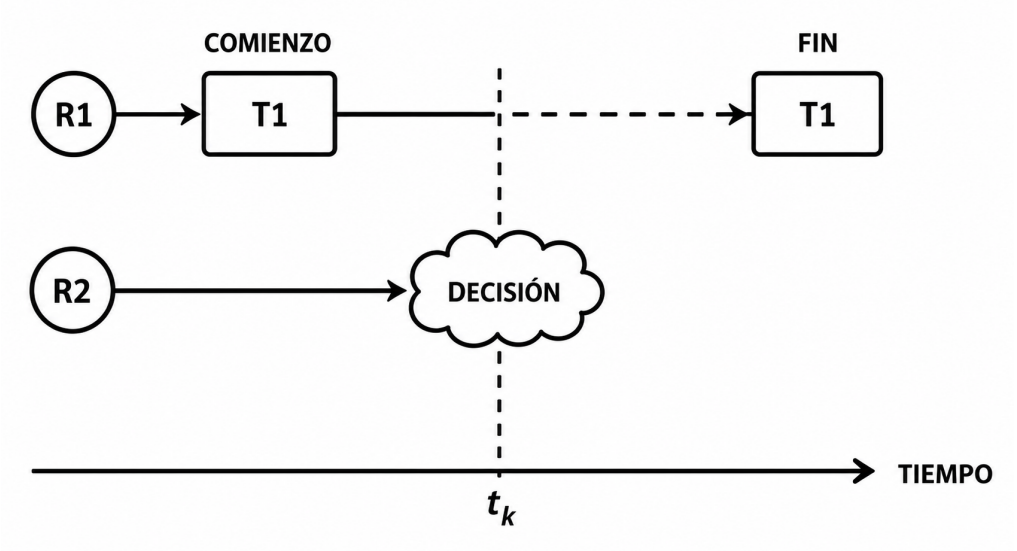


Figura 2.2: Ilustración del *leak temporal*. En el instante t_k, el robot R_2 debe decidir su siguiente acción mientras R_1 ejecuta la tarea T_1, comenzada con anterioridad. Sin embargo, el instante de finalización de T_1 (el evento FIN, aún no producido) ya se ha muestreado y guardado en el estado al COMIENZO de la tarea, la decisión de R_2 podría depender de información futura que no estaría disponible en una ejecución real.

que denominaremos *leak temporal*, es una limitación intrínseca del SMDP en presencia de concurrencia y observabilidad total. La única salida es desacoplar la información que el modelo necesita internamente de la que la política puede observar: un mecanismo de eventos con relojes ocultos. Ese mecanismo es el del GSMP.

El *Proceso de Decisión Semi-Markoviano Generalizado* (GSMDP) [29], construido sobre el formalismo del *Proceso Semi-Markoviano Generalizado* (GSMP) [13], resuelve el problema sobre ambos al modelar el solapamiento de acciones mediante eventos con relojes, ocultos para el agente.

Definición 2.6 (GSMP y GSMDP)**.** Un *Proceso Semi-Markoviano Generalizado* (GSMP) modela el sistema mediante un conjunto de **eventos**. En cada estado discreto s hay un subconjunto $\mathcal{E}(s)$ de eventos *activos*. A cada evento activo $e \in \mathcal{E}(s)$ se le asocia un **reloj** cuyo tiempo de disparo τ_e se muestrea de su distribución F_e en el momento en que el evento se activa. Los relojes avanzan en paralelo y el evento cuyo reloj expira primero,

$$e^* = \arg \min_{e \in \mathcal{E}(s)} \tau_e, \tag{2.10}$$

desencadena la transición al siguiente estado s'. Los relojes de los eventos que permanecen

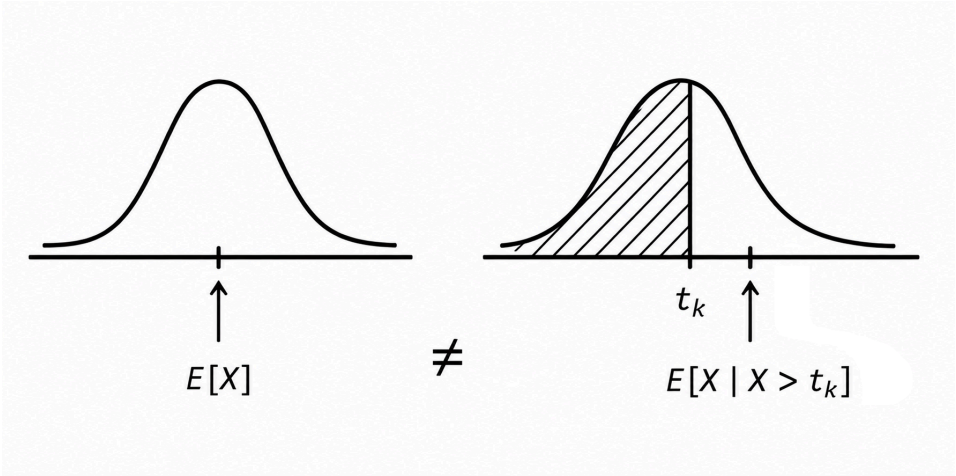


Figura 2.3: La duración de una tarea no es, en general, sin memoria. La esperanza incondicional $\mathbb{E}[X]$ (izquierda) difiere de la esperanza condicionada a que la tarea siga en ejecución transcurrido un tiempo t_k, $\mathbb{E}[X \mid X > t_k]$ (derecha): el tiempo ya transcurrido desplaza la distribución del tiempo restante.

activos en s' conservan su tiempo transcurrido y los de los eventos recién activados en s' se muestrean de nuevo. El *Proceso de Decisión Semi-Markoviano Generalizado* (GSMDP) extiende el GSMP añadiendo que algunos eventos son **controlables** por el decisor (las acciones $a \in \mathcal{A}$).

La representación por relojes resuelve el dilema anterior y ofrece, por tanto, una doble ventaja. En primer lugar, el valor muestreado de cada reloj τ_e es una variable *latente* del modelo: la política condiciona únicamente sobre el estado discreto y sobre los tiempos transcurridos δ_e, nunca sobre los instantes de disparo todavía no realizados. Esto elimina el *leak temporal*. En segundo lugar, al modelar el tiempo de finalización de las acciones en el conjunto de eventos, ahora el estado global puede almacenar los tiempos transcurridos de las acciones δ_e. Las duraciones de las acciones no son, en general, sin memoria (en este trabajo se modelan como gaussianas). En consecuencia, el tiempo restante de una acción en curso depende del tiempo que ya lleva ejecutándose: como ilustra la Figura 2.3, la esperanza de la duración total $\mathbb{E}[X]$ difiere de la esperanza condicionada a que la tarea siga activa transcurrido un tiempo δ, $\mathbb{E}[X \mid X > \delta]$ (en la figura, con $\delta = t_k$). Un modelo que no considera esta información sería incapaz de estimar correctamente su finalización y, además, perdería la propiedad de Markov.

El tiempo transcurrido δ_e es el estadístico suficiente que permite condicionar correctamente la distribución del tiempo restante de cada acción en curso, F_e condicionada al suceso $\{\tau_e > \delta_e\}$, lo que además restaura la propiedad de Markov sobre el estado global.

El SMDP es el caso particular del GSMDP en que a lo sumo un evento está activo en cada instante de decisión. El CTMDP (*Continuous-Time MDP*) es el caso especial en que todos los relojes siguen distribuciones exponenciales: la propiedad de ausencia de memoria de la exponencial hace que el tiempo transcurrido de un reloj activo no aporte información adicional sobre su tiempo restante.

$$X \sim \mathit{Exp}(\lambda) \implies P(X > t + s \mid X > s) = P(X > t) \tag{2.11}$$

En la práctica, la resolución exacta de un GSMDP con distribuciones generales es computacionalmente intratable; los métodos aproximados recurren a representaciones por fases exponenciales [29] o a planificadores basados en simulación, como MCTS, que muestrean el tiempo de disparo de cada reloj en el evento de finalización (*sample-at-fire*) y no en su inicio, materializando así de forma natural la ausencia de *leak*.

2.2.5. Procesos de Decisión de Markov Descentralizados

Cuando varios agentes deben cooperar para maximizar una recompensa común, pero cada uno solo dispone de observaciones locales y no accede ni al estado global ni a las observaciones de los demás, el modelo apropiado es el *Proceso de Decisión de Markov Descentralizado y Parcialmente Observable* (DEC-POMDP) [4, 23].

Definición 2.7 (DEC-POMDP)**.** Un DEC-POMDP para n agentes es una tupla $\langle \mathcal{I}, \mathcal{S}, \{\mathcal{A}_i\}, \mathcal{T}, \mathcal{R}, \{\Omega_i\}, \mathcal{O}, \gamma \rangle$ donde:

- $\mathcal{S}$ es el conjunto de **estados**.
- $\mathcal{I} = \{1, \ldots, n\}$ es el conjunto de **agentes** cooperativos.
- $\{\mathcal{A}_i\}_{1 \le i \le n}$ son los conjuntos de acciones locales; la **acción conjunta** es $\mathbf{a} = (a_1, \ldots, a_n)$.
- $\mathcal{T}(s, \mathbf{a}, s')$ es la transición sobre el estado global, condicionada a la acción conjunta.
- $\mathcal{R}(s, \mathbf{a})$ es una **recompensa común** al equipo (agentes plenamente cooperativos).
- $\{\Omega_i\}$ son los conjuntos de observaciones locales. La **observación conjunta** es $\mathbf{o} = (o_1, \ldots, o_n)$.
- $\mathcal{O}(\mathbf{o} \mid s', \mathbf{a})$ es la probabilidad de la observación conjunta.
- γ es el factor de descuento.

La dificultad esencial es que cada agente debe actuar basándose únicamente en su propia historia de observaciones locales, sin poder construir de forma independiente una creencia

sobre el estado global, pues esta depende también de lo que los demás hayan observado y hecho. Esta interdependencia hace que resolver óptimamente un DEC-POMDP de horizonte finito sea estrictamente más difícil que un POMDP de un solo agente [4]. En la práctica, ello obliga a recurrir a métodos aproximados y a estructuras de comunicación explícita que reduzcan la incertidumbre sobre las intenciones ajenas. Esta comunicación se modela dentro del marco formal del DEC-POMDP como acciones locales (agente que envía un mensaje) y como observaciones locales (agente que recibe el mensaje).

El *Proceso de Decisión Semi-Markov Descentralizado y Parcialmente Observable* (DEC-POSMDP) integra en un único modelo la descentralización y la observabilidad parcial del DEC-POMDP con las duraciones continuas del SMDP [24]. Se obtiene intercambiando en la Definición 2.7 la función de transición $\mathcal{T}(s, \mathbf{a}, s')$ por la **distribución conjunta de transición y duración** $\mathcal{F}(s', \tau \mid s, \mathbf{a})$ y el factor de descuento discreto γ por la **tasa de descuento continua** $\beta > 0$.

Ahora bien, el DEC-POSMDP hereda del SMDP la misma limitación discutida en la Sección 2.2.4: postula directamente la distribución $\mathcal{F}$ y no modela de forma explícita el solapamiento de acciones de duración continua mediante un mecanismo de eventos. Del mismo modo que el GSMDP extiende al SMDP, podemos extender el DEC-POSMDP incorporando la lógica de eventos del GSMP, dando lugar a lo que en este trabajo denominamos *Proceso de Decisión Semi-Markov Generalizado Descentralizado y Parcialmente Observable* (DEC-POGSMDP).

Definición 2.8 (DEC-POGSMDP)**.** Un DEC-POGSMDP para n agentes es la tupla $\langle \mathcal{I}, \mathcal{S}, \{\mathcal{A}_i\}, \mathcal{F}, \mathcal{E}, \mathcal{R}, \{\Omega_i\}, \mathcal{O}, \beta \rangle$, que incorpora a un DEC-POSMDP el mecanismo de eventos del GSMP (Definición 2.6):

- $\mathcal{E}$ es el conjunto de **eventos**. En cada estado s, un subconjunto $\mathcal{E}(s) \subseteq \mathcal{E}$ está activo y contiene las finalizaciones de las acciones de duración continua que los agentes tienen en curso. A cada evento activo e se le asocia un **reloj oculto** $\tau_e \sim F_e$, muestreado en su activación; el evento cuyo reloj expira primero desencadena la transición y el avance del reloj global.
- Los relojes son **latentes**: ninguna observación o_i contiene el valor de un reloj. Cada agente observa únicamente las componentes discretas de su estado y el tiempo transcurrido δ_e de sus propias acciones en curso.

Cuadro 2.2: Progresión de modelos según las hipótesis del problema.

Variante	**Tiempo**	**Incertidumbre**	**Observabilidad**
MRTA (MDP)	discreto	no	global
MRTAU (GSMDP)	continuo	sí	global
DEC-MRTAU (DEC-POGSMDP)	continuo	sí	local + comunicación

La función de utilidad resultante es:

$$U_\pi(s) = \mathbb{E}_{s',\tau}\left[\, \mathcal{R}(s,\pi(s)) + e^{-\beta\tau}\, U_\pi(s') \,\middle|\, s,\pi(s)\right]. \tag{2.12}$$

No hemos encontrado en la literatura una definición formal de este modelo bajo este nombre; lo introducimos como la progresión natural que resulta de añadir la lógica de eventos del GSMP (Sección 2.2.4) a un DEC-POSMDP. El cambio es conceptualmente menor (reutiliza íntegramente el mecanismo de eventos ya definido para el GSMP) pero metodológicamente importante, pues es lo que garantiza que el modelo trate correctamente las duraciones continuas concurrentes.

El DEC-POGSMDP forma el marco teórico que captura todas las características del problema MRTAU distribuido: cooperación de múltiples robots (descentralización), desconocimiento del estado de los compañeros (observabilidad parcial), duraciones continuas y estocásticas modeladas mediante eventos con relojes ocultos (semi-Markov generalizado) e incertidumbre en el éxito de las tareas. La Sección 2.3 desarrolla esta correspondencia.

2.3. El problema MRTAU distribuido como DEC-POGSMDP

Con el problema definido en la Sección 2.1 y los modelos presentados en la Sección 2.2, falta enlazar ambas piezas: formalizar el problema como una instancia de la familia de procesos de Markov. Se procede de forma incremental, añadiendo una hipótesis cada vez, hasta llegar al modelo definitivo. La Tabla 2.2 resume esta progresión.

Las características estáticas de la instancia (Definición 2.1) (como las ubicaciones, ventanas de ejecución, tabla de capacidades, las recompensas) se consideran parámetros conocidos del modelo. Lo que los siguientes apartados formalizan es la parte dinámica: los estados que evolucionan, las acciones que los modifican y la información disponible en cada decisión.

2.3.1. MRTA como MDP

Considérese primero la versión determinista y con observabilidad total: un único planificador centralizado, conocimiento perfecto del estado de robots y tareas, transiciones deterministas y tiempo discretizado. Bajo estas hipótesis, el problema se modela como un MDP (Definición 2.3).

- **Estado ($\mathcal{S}$).** El estado global agrega el estado dinámico de robots y tareas junto con el instante de tiempo actual:

$$s = \left(t,\ s^{R_1}, \ldots, s^{R_n},\ s^{T_1}, \ldots, s^{T_m}\right) \tag{2.13}$$

  - t representa el reloj global, es necesario para comprobar las ventanas de ejecución.
  - Las variables $s^{R_i},\ 1 \leq i \leq n$ codifican los estados internos de cada robot. Cada estado contiene la información de:

$$s^{R_i} = (x_i,\ b_i,\ j,\ t_i^{\text{ini}},\ op_i) \tag{2.14}$$

    - x_i contiene la posición actual del robot.
    - $b_i \in \mathbb{R}$ contiene el nivel de batería actual del robot.
    - $j \in \{1 \ldots m\} \cup \{0\}$ es un identificador de la tarea a la que el robot está actualmente asignado, $j = 0$ en caso de que no esté asignado a ninguna.
    - $t_i^{\text{ini}} \in \mathbb{R}$ es el instante de tiempo en que el robot inició su acción actual. Junto con el reloj global t, permite obtener el tiempo transcurrido $\delta_i = t - t_i^{\text{ini}}$ de la acción en curso. En la versión determinista esta distinción es meramente formal; su relevancia completa se hace explícita al introducir la incertidumbre en la Sección 2.3.2.
    - $op_i \in \{\textsc{Available}, \textsc{Waiting}, \textsc{Executing}, \textsc{Finished}\}$ incluye el estado operativo actual del robot.
  - Las variables $s^{T_j},\ 1 \leq j \leq m$ codifican los estados internos actuales de cada tarea. $s^{T_j} \in \{\textsc{Pending}, \textsc{Assigned}, \textsc{Executing}, \textsc{Completed}\}$.

- **Acciones ($\mathcal{A}$).** Cuando un robot queda libre, es necesario que tome una nueva decisión: ejecutar una nueva tarea, recargar en la estación de recarga más cercana o terminar (desplazarse a la estación de recarga más cercana y no ejecutar más acciones). Esto

define el espacio de acciones:

$$\mathcal{A} = \{\textsc{ExecuteTask}(R_i, T_j), \textsc{Recharge}(R_i), \textsc{Finish}(R_i)\} \tag{2.15}$$

- **Transición ($\mathcal{T}$).** En la versión determinista, $\mathcal{T}(s, a, s') \in \{0, 1\}$: ejecutar una tarea (con su coalición completa, dentro de su ventana) la marca como Completed con certeza, desplaza a los robots y consume una cantidad de batería conocida. Esta función de transición se encarga de modelar solamente las transiciones entre los estados de una manera coherente con el problema, respetando las propiedades de una solución (Definición 2.2).
- **Recompensa ($\mathcal{R}$).** Refleja la función objetivo de la Ecuación (2.1): premia tareas completadas y robots que sobreviven, y penaliza tiempo transcurrido y tareas pendientes. La penalización de tareas pendientes no se aplica hasta alcanzar un estado final. Consideramos una misma recompensa para todas las tareas resueltas, es por eso que la puntuación otorgada a un episodio completo ξ debe ser:

$$\mathcal{R}(\xi) \;=\; \lambda_c\,|\mathcal{T}_{\text{comp}}| \;-\; \lambda_p\,|\mathcal{T}_{\text{pend}}| \;+\; \lambda_s\,n_{\text{surv}} \;-\; \lambda_t\,C_{\text{máx}}, \tag{2.16}$$

  donde $\mathcal{T}_{\text{comp}}$ y $\mathcal{T}_{\text{pend}}$ son los conjuntos de tareas completadas y pendientes, n_{surv} es el número de robots que finalizan sin agotar su batería, $C_{\text{máx}}$ es el *makespan* y $\lambda_c, \lambda_s, \lambda_t, \lambda_p \geq 0$ ponderan los distintos objetivos, con $\lambda_c + \lambda_s + \lambda_t + \lambda_p = 1$.
- **Factor de descuento ($\gamma = 1$).** Cualquier $\gamma < 1$ descontaría la recompensa terminal en función de la longitud del episodio, lo que distorsionaría artificialmente el objetivo: dos políticas que completen las mismas tareas obtendrían valores distintos según hayan tardado más o menos pasos, y esa penalización por tiempo ya está explícita en la función de recompensa.

Este modelo es un punto de partida conceptual y la base de los *baselines* deterministas, pero incurre en dos simplificaciones que lo alejan del problema real (Sección 2.1.2): ignora que las duraciones son continuas y estocásticas, y que el éxito de una tarea no está garantizado.

2.3.2. MRTAU como GSMDP

Al reintroducir las fuentes de incertidumbre de la Sección 2.1.2, es necesario aplicar un cambio de modelo. El MDP puede absorber la incertidumbre en el resultado de las tareas,

pero no las duraciones continuas y estocásticas. Las tareas pueden ahora fracasar, por lo que se añade el estado FAILED al conjunto de estados de tarea, y los robots pueden agotar su batería debido a la variabilidad de los tiempos, por lo que se añade igualmente el estado operativo FAILED.

Manteniendo todavía la hipótesis de observabilidad global, podría pensarse en modelar esta variante como un SMDP, sustituyendo la transición discreta por la distribución conjunta de transición y duración $\mathcal{F}(s', \tau \mid s, a)$. Sin embargo, como se estableció en la Sección 2.2.4, hacerlo provocaría inevitablemente el *leak temporal* (Figura 2.2): bajo observabilidad total, fijar la duración de una tarea en el instante en que comienza expondría su instante de finalización a cualquier robot que decida antes de que termine. La variante MRTAU debe modelarse, por tanto, como un **GSMDP** (Definición 2.6), tratando cada ejecución en curso como un evento con un reloj oculto.

La cola de eventos. La dinámica se articula mediante una cola de eventos. Cada acción no determinista de duración continua (ejecución de tareas) programa en la cola un evento de finalización con su instante de disparo (el valor de su reloj). El sistema avanza saltando de evento en evento: en cada paso se extrae de la cola el evento con menor instante de disparo, se avanza el reloj global t hasta él, se actualiza el estado correspondiente (la tarea pasa a COMPLETED o FAILED, el robot a AVAILABLE, etc.) y, si la transición deja a algún robot libre, se le solicita una nueva decisión. Los instantes de disparo permanecen en la cola, internos al modelo, pero nunca forman parte del estado que la política observa: es esta ocultación la que elimina el *leak*.

Resultado y duración desconocidos hasta la finalización. La consecuencia central de este mecanismo es que, al iniciar una acción EXECUTETASK(T_j), *no se conoce ni cuánto durará la ejecución ni si tendrá éxito*. El reloj del evento de finalización codifica ambos resultados, que permanecen ocultos hasta que el evento se dispara. Solo en el instante de finalización se revela el desenlace: éxito con probabilidad ρ_j y duración muestreada de $\mathcal{N}(\mu_t^s, \sigma_t^s)$, o fracaso con probabilidad $1 - \rho_j$ y duración de $\mathcal{N}(\mu_t^f, \sigma_t^f)$. Hasta ese momento, el estado solo refleja que la tarea está EXECUTING y, a través del instante de inicio t_i^{ini} almacenado en el estado del robot (Sección 2.3.1), cuánto tiempo lleva en ejecución. Este tiempo transcurrido $\delta_i = t - t_i^{\text{ini}}$ es información del pasado, legítimamente observable, y es además el estadístico suficiente que permite condicionar correctamente la distribución del tiempo restante mediante $F \mid \{\tau > \delta_i\}$

(Figura 2.3).

Así modelada, la variante MRTAU es un GSMDP completamente observable. La principal consecuencia es que la solución óptima ya no es una asignación fija calculada *a priori*, sino una política adaptativa que responde a los resultados observados a medida que los eventos se disparan; esto justifica el empleo de algoritmos *online*. No obstante, este modelo aún conserva una hipótesis fuerte: la observabilidad global, coherente solo con una arquitectura centralizada.

2.3.3. DEC-MRTAU como DEC-POGSMDP

El núcleo de este trabajo consiste en eliminar la última hipótesis: la observabilidad global. En un sistema verdaderamente distribuido no existe coordinador ni estado compartido; cada robot decide de forma autónoma a partir de su información local. Esta relajación transforma el GSMDP en un DEC-POGSMDP (Sección 2.2.5), dando lugar a la variante *DEC-MRTAU*, modelo formal definitivo del problema, definido por la tupla

$$\left\langle \mathcal{I}, \mathcal{S}, \{\mathcal{A}_i\}, \mathcal{F}, \mathcal{E}, \mathcal{R}, \{\Omega_i\}, \mathcal{O} \right\rangle, \tag{2.17}$$

cuyos elementos se concretan a continuación.

Agentes ($\mathcal{I}$). El conjunto de n robots cooperativos. Comparten un único objetivo de equipo, por lo que el problema es plenamente cooperativo.

Estado global ($\mathcal{S}$). Idéntico al de las variantes anteriores, $s = (t, s^{R_1}, \ldots, s^{R_n}, s^{T_1}, \ldots, s^{T_m})$. La diferencia esencial es que este estado global ningún agente lo observa por completo: existe en el modelo, pero no en la percepción de los robots.

Acciones ($\{\mathcal{A}_i\}_{1 \leq i \leq n}$). Cada robot dispone de sus acciones locales $\{$ExecuteTask(T_j), Recharge, Finish$\}$. Adicionalmente, según el algoritmo de coordinación, los robots realizan acciones de comunicación (difundir una puja, una distribución sobre sus planes, etc.) que no modifican el entorno físico pero condicionan las decisiones de los demás. Las tareas *multi-robot* ($q_j > 1$) son aquí especialmente delicadas, pues exigen acordar coaliciones sin observabilidad mutua, lo que solo puede lograrse mediante la comunicación.

Transiciones y Duraciones ($\mathcal{F}$). Dinámica semi-Markoviana generalizada con duraciones continuas estocásticas e incertidumbre en el éxito de las tareas (el componente "$+U$"), inducida por el mecanismo de eventos y condicionada a la acción conjunta: $\mathcal{F}(s', \tau \mid s, \mathbf{a})$.

Eventos ($\mathcal{E}$). Como en la variante centralizada (Sección 2.3.2), las acciones de duración continua generan eventos de finalización con relojes ocultos, gestionados por una cola de eventos; el resultado y la duración de cada tarea solo se revelan al dispararse el evento correspondiente. Es este componente la que distingue el DEC-POGSMDP de un mero DEC-POSMDP y la que garantiza, también en el caso distribuido, que ningún agente anticipe el desenlace de una ejecución en curso.

Recompensa ($\mathcal{R}$). Función global cooperativa (Ecuación (2.1)), común a todos los agentes: maximiza tareas completadas y robots que preservan su integridad, y minimiza tiempo de ejecución y tareas pendientes.

Observabilidad ($\{\Omega_i\}_{1\le i\le n}$, $\mathcal{O}$). Es el elemento que distingue conceptualmente al DEC-MRTAU. El espacio de observaciones de cada robot es la tupla

$$o_i = \left(s^{R_i},\, \mathbf{s}^T,\, \mathcal{M}_{\text{in}}\right), \tag{2.18}$$

formada por su propio estado, el estado de las tareas y los mensajes recibidos. La función de observación $\mathcal{O}$ actúa como un *filtro de la realidad* con una estructura híbrida característica de este problema:

- **Parte totalmente observable.** El robot conoce con exactitud su propio estado s^{R_i} — batería, ubicación, disponibilidad y, en particular, el tiempo transcurrido δ_i de su propia acción en curso— y el estado de todas las tareas $\mathbf{s}^T$. Sobre esta parte el modelo se comporta de forma observable, y sobre ella cada robot condiciona correctamente la finalización de su propia ejecución.
- **Parte oculta.** El robot desconoce el estado interno y la ubicación de los demás robots, $s^{R_k}, k \neq i$: no sabe dónde están sus compañeros, cuánta batería les queda ni qué tarea pretenden ejecutar; tampoco, por tanto, los relojes de sus acciones. Es esta componente la que introduce la observabilidad parcial y sitúa el problema en el marco DEC-POGSMDP en lugar del GSMDP.

Obsérvese que esta estructura refuerza la ausencia de *leak temporal*: si en el modelo centralizado había que ocultar deliberadamente los relojes para evitar la fuga, aquí la imposibilidad de observar a los compañeros la convierte en estructural (un robot no puede "ver el futuro" de otro porque no puede observar a ese otro en absoluto). La lógica de eventos del DEC-POGSMDP sigue siendo necesaria, no obstante, para tratar correctamente la propia ejecución de cada robot, cuyo desenlace tampoco debe conocerse por anticipado.

El papel de la comunicación. La incertidumbre sobre las intenciones ajenas (en particular, el riesgo de que dos robots seleccionen la misma tarea o de que no se forme la coalición requerida) no puede eliminarse mediante observación directa, pero sí mitigarse mediante el intercambio explícito de mensajes $\mathcal{M}_{\text{in}}$. La comunicación es, por tanto, el mecanismo que sustituye a la observabilidad global ausente: en lugar de ver el estado de los compañeros, cada robot infiere parcialmente sus intenciones a partir de lo que estos comunican. Bajo este prisma, los distintos algoritmos analizados en este trabajo (desde los *baselines* sin comunicación hasta los métodos de pujas CBAA/CBBA y la planificación distribuida Dec-MCTS) pueden entenderse como diferentes políticas de comunicación que resuelven el mismo DEC-POGSMDP con grados crecientes de coordinación. Esta lectura unificada permite comparar de forma directa la planificación distribuida con el MCTS centralizado del modelo base [25].

En síntesis, la transición desde un planteamiento centralizado (MDP/GSMDP con observabilidad global) a uno puramente distribuido (DEC-POGSMDP) no es una mera elección de implementación, sino una reformulación que cambia la clase de complejidad del problema y obliga a sustituir la información global por mecanismos de coordinación basados en comunicación. Es esta reformulación la que define la contribución principal del trabajo.

2.4. Algoritmos propuestos

Con la única excepción de la variante más elemental de MRTA (ST-SR-IA), el resto de variantes de MRTA, y en particular el problema MRTAU abordado en este trabajo, son NP-duras [12]. En consecuencia, salvo en instancias triviales, no es realista aspirar a hallar la solución óptima. Esta dificultad se agrava al introducir la incertidumbre (Sección 2.1.2): cuando las ejecuciones son estocásticas, la solución deja de ser una asignación fija y pasa a ser una política que debe adaptarse al estado observado en cada instante (Sección 2.3.3). Los algoritmos no persiguen, por tanto, la optimalidad, sino soluciones suficientemente buenas

obtenidas con un coste de cómputo y de comunicación asumible.

En el problema distribuido, la comunicación desempeña un papel central en esa búsqueda: es el mecanismo que sustituye la observabilidad global ausente y permite a los robots coordinar sus decisiones (Sección 2.3.3). Ahora bien, la comunicación no puede suponerse ilimitada. Definir una comunicación perfecta y sin coste, en la que cada robot difundiera de forma continua su estado completo al resto, cancelaría por completo el sentido del planteamiento descentralizado: cada agente podría reconstruir el estado global y el problema colapsaría de nuevo en uno centralizado, contradiciendo las propias motivaciones (robustez, escalabilidad y tolerancia a comunicación limitada o intermitente) que justifican el enfoque distribuido (Sección 2.1.4). El interés reside, precisamente, en alcanzar buenas soluciones intercambiando una cantidad de información acotada.

Bajo este prisma, los algoritmos considerados pueden leerse de forma unificada como distintas **políticas de comunicación** que resuelven el mismo DEC-POGSMDP con grados crecientes de coordinación: desde la ausencia total de comunicación (*baselines*, Sección 2.4.1), pasando por la negociación de coaliciones mediante pujas (CBAA y CBBA, Sección 2.4.2), hasta la planificación distribuida que comunica distribuciones sobre los planes (Dec-MCTS, Sección 2.4.3). Todos comparten la naturaleza *online* y reactiva justificada en la Sección 2.1.3: se invocan en cada instante de decisión y devuelven una única acción. La política global emerge de la invocación repetida del procedimiento conforme avanza la cola de eventos.

2.4.1. *Baselines*: Greedy y Random

Los dos *baselines* constituyen las referencias frente a las que se medirá la aportación del resto de algoritmos. Su rasgo definitorio es la **ausencia de comunicación**: cada robot decide exclusivamente a partir de su propio estado s^{R_i} y del estado de las tareas s^T, ignorando por completo a los demás robots (el conjunto de mensajes $\mathcal{M}_{\text{in}}$ de la observación (2.18) permanece vacío). En términos del DEC-POGSMDP representan la política de comunicación degenerada y se comportan como si cada robot resolviera un MDP puramente local.

Como consecuencia de la falta de comunicación, los *baselines* carecen de coordinación (no razonan sobre las coaliciones q_j ni sobre los conflictos con otros robots) y no trabajan con la incertidumbre (no emplean la probabilidad de éxito ρ_j ni las distribuciones de duración). La única condición de factibilidad que comprueban es la energética: una tarea T_j se considera

alcanzable si el robot dispone de batería suficiente para desplazarse hasta ella:

$$\text{coste}_{ij} \;=\; \frac{d(x_i, x_j)}{v_i} \;\cdot\; \textit{rate-}R_i \;\leq\; b_i. \tag{2.19}$$

Ambos algoritmos comparten el mismo esqueleto de decisión y se diferencian únicamente en el criterio de selección de tarea. El esqueleto distingue tres situaciones: si no quedan tareas pendientes en el escenario, el robot ha terminado su cometido y emite FINISH; si quedan tareas pendientes pero ninguna es alcanzable con la batería actual, la mejor opción es RECHARGE y reintentar más tarde; en otro caso, el robot selecciona una de las tareas alcanzables y la ejecuta. El **Random** (Algoritmo 1) elige una tarea alcanzable de manera aleatoria, lo que conforma el peor escenario de coordinación posible. El **Greedy** (Algoritmo 2) elige la tarea alcanzable más cercana según la función de distancia $d(\cdot,\cdot)$, introduciendo una heurística mínima de eficiencia, pero sin ninguna anticipación de conflictos.

Algoritmo 1 RANDOM — decisión de R_i en un instante de decisión

Entrada: observación $o_i = (s^{R_i}, \mathbf{s}^T, \mathcal{M}_{\text{in}})$
Salida: acción local

$\mathcal{P} \leftarrow \{\, T_j \in \mathcal{T} : s^{T_j} = \text{PENDING} \,\}$ ▷ tareas pendientes
$\mathcal{A}_{\text{alc}} \leftarrow \{\, T_j \in \mathcal{P} : \text{coste}_{ij} \leq b_i \,\}$ ▷ alcanzables, Ec. (2.19)
if $\mathcal{P} = \varnothing$ **then return** FINISH
else if $\mathcal{A}_{\text{alc}} = \varnothing$ **then return** RECHARGE
else
    $T_j \leftarrow$ tarea elegida uniformemente al azar de $\mathcal{A}_{\text{alc}}$
    return EXECUTETASK(T_j)
end if

Algoritmo 2 GREEDY — decisión de R_i en un instante de decisión

Entrada: observación $o_i = (s^{R_i}, \mathbf{s}^T, \mathcal{M}_{\text{in}})$
Salida: acción local

$\mathcal{P} \leftarrow \{\, T_j \in \mathcal{T} : s^{T_j} = \text{PENDING} \,\}$
$\mathcal{A}_{\text{alc}} \leftarrow \{\, T_j \in \mathcal{P} : \text{coste}_{ij} \leq b_i \,\}$ ▷ Ec. (2.19)
if $\mathcal{P} = \varnothing$ **then return** FINISH
else if $\mathcal{A}_{\text{alc}} = \varnothing$ **then return** RECHARGE
else
    $T_j \leftarrow \arg\min_{T_k \in \mathcal{A}_{\text{alc}}} d(x_i, x_k)$ ▷ tarea alcanzable más cercana
    return EXECUTETASK(T_j)
end if

2.4.2. Algoritmos basados en pujas: CBAA y CBBA

El siguiente paso incorpora la coordinación mediante mecanismos de mercado. El *Consensus-Based Auction Algorithm* (CBAA) y su extensión, el *Consensus-Based Bundle Algorithm* (CBBA), fueron propuestos por Choi, Brunet y How [7] como métodos descentralizados en los que cada agente puja por las tareas según una función de *score* y los conflictos se resuelven mediante una fase de consenso basada en comunicación local. A diferencia de los *baselines*, aquí los robots intercambian información: difunden sus pujas y reciben las de sus vecinos. Este intercambio se modela mediante las acciones de comunicación del DEC-POGSMDP (Sección 2.3.3) y se realiza mediante las funciones CommunicateBid y ReceiveBid, que encapsulan la difusión de pujas y la recepción de las pujas ajenas.

Aproximación determinista del *score*. Ambos algoritmos adoptan un tratamiento determinista de la incertidumbre que caracteriza a MRTAU. Por un lado, la probabilidad de éxito ρ_j de una tarea se asume conocida y se emplea directamente como peso de su puntuación: las tareas con mayor probabilidad de completarse con éxito reciben un *score* más alto. Por otro, la duración esperada de la ejecución de una tarea se puede calcular combinando las medias de las distribuciones de éxito $\mathcal{N}(\mu_j^s, \sigma_j^s)$ y de fracaso $\mathcal{N}(\mu_j^f, \sigma_j^f)$ como:

$$\bar{d}_j \;=\; \rho_j\, \mu_j^s \;+\; (1-\rho_j)\, \mu_j^f, \tag{2.20}$$

En la práctica, esto significa que las pujas se resuelven en cada instante de decisión de manera determinista, haciendo uso de las esperanzas matemáticas de las variables aleatorias probabilidad de éxito y tiempo de ejecución.

Concretamente, para un robot R_i que evalúa la tarea T_j desde una posición de partida en un instante de disponibilidad t_i^{libre}, se estima el tiempo de viaje $\Delta_{ij}^{\text{viaje}} = d(x_i, x_j)/v_i$ y el instante de llegada $t_{ij}^{\text{llegada}} = t_i^{\text{libre}} + \Delta_{ij}^{\text{viaje}}$. Si la llegada excede el límite de la ventana ($t_{ij}^{\text{llegada}} > t_j^l$), la puja es nula. En otro caso, considerando el tiempo de espera $\Delta_{ij}^{\text{espera}} = \max(0,\, t_j^s - t_{ij}^{\text{llegada}})$ y la duración esperada (2.20), se define la inversión temporal total $\tau_{ij} = \Delta_{ij}^{\text{viaje}} + \Delta_{ij}^{\text{espera}} + \bar{d}_j$ y la puja (*score*) es finalmente:

$$\nu_{ij} \;=\; \underbrace{\frac{\rho_j}{1+\tau_{ij}}}_{\text{valor base}} \cdot \underbrace{\frac{a_j+1}{q_j}}_{\text{factor de cooperación}}, \tag{2.21}$$

donde a_j es el número de robots ya asignados a T_j y q_j el número de robots requeridos para la ejecución. El valor base es creciente en la probabilidad de éxito ρ_j y decreciente en el tiempo invertido τ_{ij}, mientras que el factor de cooperación fomenta la formación de coaliciones.

Estimación del instante de disponibilidad. El cálculo de la puja (2.21) requiere conocer el instante t_i^{libre} en que el robot evaluado queda libre y puede emprender el desplazamiento hacia T_j. Para el robot que toma la decisión esta información es inmediata: se encuentra en estado AVAILABLE y su instante de disponibilidad es el reloj global, $t_i^{\text{libre}} = t$. La dificultad surge al evaluar las pujas de los demás robots, pues un robot puede hallarse a mitad de una acción de duración estocástica y su instante de finalización no es conocido. Para resolverlo se emplea una función auxiliar, ESTIMATENEXTAVAILABILITY, que infiere t_k^{libre} a partir de la situación en que se encuentra el robot R_k. Caben cuatro casos:

- **Acción de duración determinista.** Si el robot está disponible, desplazándose o recargando, la duración de su acción en curso es conocida (los desplazamientos y las recargas son deterministas en el modelo, al depender solo de distancias, velocidades y tasas de recarga). Su instante de disponibilidad es, por tanto, el instante de finalización ya programado, sin necesidad de estimación.
- **En espera de coalición (tarea PENDING).** El robot ha llegado a una tarea, pero la coalición de q_j trabajadores aún no se ha completado, de modo que permanece a la espera de sus compañeros. Como dicha coalición podría no llegar a formarse, se adopta una estimación conservadora: el robot quedará libre, en el peor caso, cuando se agote la ventana de ejecución, esto es, en el instante límite t_j^l.
- **Coalición completa sin iniciar (tarea ASSIGNED).** La coalición está formada y la ejecución comenzará en un instante conocido, pero todavía no ha empezado. Al no disponer de información sobre el transcurso de la ejecución, la disponibilidad se estima como ese instante de inicio más la duración esperada $\bar{d}_j$ de la Ecuación (2.20).
- **En ejecución (tarea EXECUTING).** El robot ejecuta la tarea desde un instante de inicio t^{inicio} y ha transcurrido un tiempo $\delta = t - t^{\text{inicio}}$. Este es el caso no trivial y se desarrolla a continuación.

El caso de una tarea en ejecución conecta directamente con la discusión del GSMDP (Sección 2.2.4): el tiempo transcurrido δ es información del pasado, legítimamente observable, y constituye el estadístico suficiente que permite condicionar la distribución del tiempo restante

(Figura 2.3). No puede emplearse la duración esperada incondicional, ya que el hecho de que la tarea siga en curso transcurrido δ desplaza esa distribución; se busca, en su lugar, la esperanza de la duración total X condicionada al suceso $\{X > \delta\}$, esto es, $\mathbb{E}[X \mid X > \delta]$.

A esta dependencia del tiempo transcurrido se añade que la duración no obedece a una única distribución, sino a una combinación de dos componentes, ya que el desenlace de la tarea permanece desconocido hasta su finalización (Sección 2.3.2): con probabilidad *a priori* ρ_j la tarea concluirá con éxito y su duración se distribuye según $\mathcal{N}(\mu_j^s, \sigma_j^s)$, mientras que con probabilidad $1 - \rho_j$ fracasará con duración $\mathcal{N}(\mu_j^f, \sigma_j^f)$ de modo que

$$X \sim \rho_j \cdot \mathcal{N}(\mu_j^s, \sigma_j^s) \;\; + \;\; (1 - \rho_j) \cdot \mathcal{N}(\mu_j^f, \sigma_j^f) \tag{2.22}$$

Observar que la tarea no ha terminado ($X > \delta$) aporta información no solo sobre el tiempo restante, sino también sobre cuál de los dos desenlaces es más probable. El cálculo procede, en consecuencia, en dos etapas.

Primera etapa: actualización bayesiana del desenlace. Sea ϕ la función de densidad de la distribución normal y sea Φ su función de distribución acumulada ($\Phi(z) = P(Z \leq z)$), la probabilidad de que cada componente supere el tiempo transcurrido es respectivamente $(1 - \Phi(z_s))$ y $(1 - \Phi(z_f))$, con $z_s = (\delta - \mu_j^s)/\sigma_j^s$ y $z_f = (\delta - \mu_j^f)/\sigma_j^f$. La probabilidad marginal de que una tarea supere el tiempo transcurrido es

$$P(X > \delta) \;=\; \rho_j \,(1 - \Phi(z_s)) \;+\; (1 - \rho_j)\,(1 - \Phi(z_f)). \tag{2.23}$$

El teorema de Bayes proporciona entonces la probabilidad *a posteriori* de que la ejecución acabe en éxito, dado que sigue en curso,

$$P(S \mid X > \delta) \;=\; \frac{P(S)\,P(X > \delta \mid S)}{P(X > \delta)} \;=\; \frac{\rho_j \left(1 - \Phi(z_s)\right)}{\rho_j \left(1 - \Phi(z_s)\right) + (1 - \rho_j) \left(1 - \Phi(z_f)\right)}, \tag{2.24}$$

y, de forma análoga, la probabilidad a posteriori de fracaso,

$$P(F \mid X > \delta) \;=\; \frac{(1 - \rho_j) \left(1 - \Phi(z_f)\right)}{\rho_j \left(1 - \Phi(z_s)\right) + (1 - \rho_j) \left(1 - \Phi(z_f)\right)}. \tag{2.25}$$

Intuitivamente, si las ejecuciones fallidas tienden a prolongarse, una tarea que se demora ve incrementada su probabilidad a posteriori de fracaso.

Segunda etapa: esperanza condicionada por componente. Condicionada a cada desenlace,

la duración sigue una normal truncada por la izquierda en δ. Para el desenlace de éxito, su función de densidad es

$$f_{X_s|X_s>\delta}(x) \;=\; \frac{f_{X_s}(x)}{P(X_s>\delta)}\,\mathbf{1}_{(\delta,\,+\infty)}(x), \tag{2.26}$$

donde $\mathbf{1}_{(\delta,+\infty)}$ es la función indicatriz. Sustituyendo la densidad normal $f_{X_s}(x) = \frac{1}{\sigma_j^s}\,\phi\big((x-\mu_j^s)/\sigma_j^s\big)$ y $P(X_s>\delta) = 1-\Phi(z_s)$, la esperanza condicionada se expresa como

$$\mathbb{E}[X \mid X>\delta, S] \;=\; \int_\delta^{+\infty} x\,\frac{f_{X_s}(x)}{P(X_s>\delta)}\,dx \;=\; \frac{1}{1-\Phi(z_s)}\int_\delta^{+\infty}\frac{x}{\sigma_j^s}\,\phi\!\left(\frac{x-\mu_j^s}{\sigma_j^s}\right)dx. \tag{2.27}$$

El cambio de variable $z = (x-\mu_j^s)/\sigma_j^s$, que implica $x = \mu_j^s + \sigma_j^s\,z$ y $dx = \sigma_j^s\,dz$, transforma esta integral en

$$\begin{aligned}
\mathbb{E}[X \mid X>\delta, S] &= \frac{1}{1-\Phi(z_s)}\int_{z_s}^{+\infty}\big(\mu_j^s + \sigma_j^s\,z\big)\,\phi(z)\,dz \\
&= \frac{\mu_j^s}{1-\Phi(z_s)}\int_{z_s}^{+\infty}\phi(z)\,dz \;+\; \frac{\sigma_j^s}{1-\Phi(z_s)}\int_{z_s}^{+\infty} z\,\phi(z)\,dz \\
&= \frac{1-\Phi(z_s)}{1-\Phi(z_s)}\,\mu_j^s \;+\; \frac{\sigma_j^s}{1-\Phi(z_s)}\,\big[-\phi(z)\big]_{z_s}^{+\infty} \\
&= \mu_j^s \;+\; \sigma_j^s\,\frac{\phi(z_s)}{1-\Phi(z_s)},
\end{aligned} \tag{2.28}$$

donde se han empleado las identidades $\Phi'(z) = \phi(z)$ y $\phi'(z) = -z\,\phi(z)$ para resolver la primera y la segunda integral, respectivamente. Procediendo de manera idéntica para el desenlace de fracaso se llega a

$$\mathbb{E}[X \mid X>\delta, F] \;=\; \mu_j^f \;+\; \sigma_j^f\,\frac{\phi(z_f)}{1-\Phi(z_f)}. \tag{2.29}$$

Combinando ambas etapas, se tiene entonces la esperanza condicionada de la duración total,

$$\mathbb{E}[X \mid X>\delta] \;=\; \mathbb{E}[X \mid X>\delta, S]\,P(S \mid X>\delta) \;+\; \mathbb{E}[X \mid X>\delta, F]\,P(F \mid X>\delta). \tag{2.30}$$

El instante de disponibilidad estimado del robot se obtiene finalmente sumando esta duración

al instante de inicio de la tarea,

$$t_k^{\text{libre}} \;=\; t^{\text{inicio}} + \mathbb{E}[X \mid X > \delta], \tag{2.31}$$

valor que luego se emplea en el cálculo de los tiempos de viaje, espera y, en última instancia, la puja ν_{ij} de la Ecuación (2.21). Conviene subrayar que esta estimación es una aproximación determinista coherente con el resto del *score* (se resume la distribución del tiempo restante en su esperanza), y que reproduce a nivel de planificación el mismo condicionamiento por tiempo transcurrido que el modelo de eventos del GSMDP realiza de forma exacta.

CBAA: subasta de tarea individual. En el CBAA (Algoritmo 3), cada robot evalúa todas las tareas alcanzables, las ordena por puja decreciente y recorre esa lista de preferencias. Para cada tarea candidata T_j, recaba mediante CommunicateBid las pujas de los vecinos que compiten por ella y cuenta cuántos la superan. La tarea dispone de $q_j - a_j$ plazas libres; el robot se adjudica una plaza (y emite ExecuteTask(T_j)) si y solo si figura entre los mejores postores que caben en esas plazas, es decir, si el número de pujas superiores es estrictamente menor que el de plazas disponibles. Si pierde el consenso en su tarea preferida, pasa a la siguiente; si agota la lista sin éxito, recarga o termina según queden o no tareas inalcanzables por batería.

Algoritmo 3 CBAA — decisión de R_i en un instante de decisión

Entrada: observación $o_i = (s^{R_i}, \mathbf{s}^T, \mathcal{M}_{\text{in}})$
Salida: acción local

$\mathcal{P} \leftarrow \{\, T_j : s^{T_j} = \text{Pending} \,\}$
if $\mathcal{P} = \varnothing$ **then return** Finish
end if
para cada $T_j \in \mathcal{P}$ alcanzable: calcular ν_{ij} según (2.21)
ordenar las tareas con $\nu_{ij} > 0$ por puja decreciente
for T_j en orden de preferencia **do**
    CommunicateBid(ν_{ij})
    $\{\nu_{kj}\}_{k \neq i} \leftarrow$ ReceiveBid() ▷ pujas de los vecinos por T_j
    $n_{\text{sup}} \leftarrow \big|\{\, k \neq i : \nu_{kj} > \nu_{ij} \,\}\big|$ ▷ vecinos que nos superan
    plazas $\leftarrow q_j - a_j$
    if *plazas* > 0 **y** $n_{\text{sup}} <$ *plazas* **then**
        return ExecuteTask(T_j) ▷ entramos entre los mejores postores
    end if
end for
if queda alguna tarea pendiente inalcanzable por batería **then return** Recharge
else return Finish
end if

CBBA: construcción de paquetes. El CBBA (Algoritmo 4) generaliza el anterior planificando secuencias de tareas. Cada robot construye de forma voraz un paquete (*bundle*): partiendo de su estado, añade iterativamente la tarea de mayor puja marginal (2.21), recalculada desde un estado virtual que actualiza la posición, el reloj y la batería de manera determinista como si las tareas ya incorporadas se hubieran ejecutado (procedimiento BUILDBESTBUNDLE). El proceso se detiene al alcanzar un tamaño máximo de paquete, al agotar la batería virtual o al no quedar tareas alcanzables. Pese a planificar secuencias a largo plazo, el robot solo compromete la primera tarea del paquete; el resto actúa como anticipación (*lookahead*) que informa la puja. La resolución de conflictos compara, sobre esa primera tarea, las pujas marginales de los vecinos cuyos paquetes la contienen: si el robot no es superado en más plazas de las disponibles, la ejecuta; en otro caso la elimina de la lista de tareas disponibles y reconstruye el paquete, repitiendo la negociación.

Los nombres CBAA y CBBA se conservan por ser las bases conceptuales de los algoritmos implementados y por su amplia difusión. No obstante, conviene matizar la correspondencia. En su formulación original [7], CBAA y CBBA (i) asumen tareas *single-robot*, (ii) computan una asignación estática y libre de conflictos *a priori*, iterando la fase de consenso hasta converger, y (iii) operan sobre un escenario determinista. Las implementaciones de este trabajo se apartan de esos tres supuestos: admiten tareas *multi-robot* permitiendo que los q_j mejores postores ganen plaza en una misma tarea (formación de coaliciones); se ejecutan de manera *online* reconstruyendo la subasta en cada instante de decisión y comprometiendo solo la acción inmediata; y operan sobre un escenario estocástico, apoyándose en el *score* que resuelve la variante determinista del problema. La formulación que captura con mayor fidelidad el primer punto es el *Consensus-Based Grouping Algorithm* (CBGA) [15], que generaliza las listas de pujas y de ganadores de CBBA para admitir varios agentes por tarea. Sin embargo, se ha conservado la nomenclatura CBAA/CBBA por ser estos dos algoritmos los que han servido como base para esta implementación.

2.4.3. Monte Carlo Tree Search Descentralizado (Dec-MCTS)

El último de los algoritmos considerados, y el de mayor grado de coordinación, es una variante descentralizada de la búsqueda en árbol de Monte Carlo (*Decentralized Monte Carlo Tree Search*, Dec-MCTS), inspirada en el algoritmo propuesto por Best et al. [5]. Frente a los métodos de pujas (Sección 2.4.2), que resumen su coordinación en una heurística determinista

Algoritmo 4 CBBA — decisión de R_i en un instante de decisión

Entrada: observación $o_i = (s^{R_i}, \mathbf{s}^T, \mathcal{M}_{\text{in}})$
Salida: acción local

$\mathcal{P} \leftarrow \{\, T_j : s^{T_j} = \textsc{Pending} \,\}$
if $\mathcal{P} = \varnothing$ **then return** FINISH
end if
$\mathcal{N} \leftarrow \varnothing$ ▷ lista negra de tareas perdidas
while True do
  $\mathcal{B} \leftarrow \textsc{BuildBestBundle}(R_i, \mathcal{N})$
  if $\mathcal{B} = \varnothing$ **then**
    return RECHARGE si hay tareas inalcanzables, en otro caso FINISH
  end if
  $T^* \leftarrow$ primera tarea de $\mathcal{B}$; $\nu^* \leftarrow$ puja marginal de T^*
  COMMUNICATEBUNDLEBID(paquete $\mathcal{B}$ y sus pujas)
  $\{\mathcal{B}_k\}_{k \neq i} \leftarrow$ RECEIVEBUNDLEBID() ▷ paquetes de los vecinos
  $n_{\text{sup}} \leftarrow \left|\{\, k \neq i : T^* \in \mathcal{B}_k \text{ con puja marginal} > \nu^* \,\}\right|$
  plazas $\leftarrow q_{T^*} - a_{T^*}$
  if *plazas* > 0 **y** $n_{\text{sup}} <$ *plazas* **then**
    return EXECUTETASK(T^*)
  else
    $\mathcal{N} \leftarrow \mathcal{N} \cup \{T^*\}$ ▷ perdemos la plaza; reintentar
  end if
end while

function BUILDBESTBUNDLE$(R_i, \mathcal{N})$
  inicializar estado virtual (posición, reloj y batería) en el estado actual de R_i
  $\mathcal{B} \leftarrow [\,]$
  while $|\mathcal{B}| < L_{\text{máx}}$ **do**
    $T^* \leftarrow$ tarea alcanzable de mayor puja marginal (2.21) desde el estado virtual, excluyendo las ya incluidas en $\mathcal{B}$ (y las de $\mathcal{N}$ si $\mathcal{B}$ está vacío)
    if no existe tal tarea **then break**
    end if
    añadir T^* a $\mathcal{B}$; avanzar el estado virtual asumiendo su ejecución
  end while
  return $\mathcal{B}$
end function

evaluada en cada instante, Dec-MCTS razona explícitamente sobre el horizonte largo mediante simulación estocástica de **rollouts** (episodios ficticios simulados) y comunica a sus vecinos no una única decisión, sino una distribución de probabilidad sobre planes completos. Es, por tanto, la política de comunicación más rica de las analizadas y la que permite la comparación más directa con el MCTS centralizado del modelo base [25].

Planificación *anytime* frente a planificación *on-request*. Existe una diferencia operativa fundamental entre Dec-MCTS y el resto de algoritmos de este capítulo que conviene fijar antes de entrar en detalles. Los baselines y los algoritmos de pujas son métodos *on-request* (bajo demanda): no mantienen estado entre decisiones y se invocan únicamente cuando un robot queda libre y la cola de eventos le solicita una acción; en ese momento ejecutan su lógica de principio a fin y devuelven una única acción. Dec-MCTS, en cambio, es un algoritmo *anytime* en el sentido pleno: cada robot lo ejecuta de forma continua en segundo plano, ampliando incrementalmente su árbol de búsqueda y refinando su distribución sobre planes, independientemente de si requiere o no una decisión. La calidad de su solución crece con el tiempo de cómputo disponible, y la búsqueda nunca se detiene salvo cuando el episodio termina.

En consecuencia, el grueso del cómputo de Dec-MCTS sucede entre instantes de decisión, no en ellos. Cuando un robot debe finalmente actuar, su árbol ya contiene un conocimiento acumulado del valor de cada plan, y la decisión es prácticamente inmediata.

Búsqueda en árbol de Monte Carlo: el algoritmo clásico. La búsqueda en árbol de Monte Carlo [6] es un método de planificación que construye incrementalmente un árbol de búsqueda asimétrico mediante simulaciones aleatorias, sesgándolo progresivamente hacia las regiones más prometedoras del espacio de decisiones. Cada **nodo** del árbol representa un estado (o, de forma equivalente en este trabajo, una secuencia de acciones desde la raíz) y cada **arista** representa una acción. La raíz corresponde al estado actual del robot. A diferencia de los métodos de programación dinámica, MCTS no requiere enumerar el espacio de estados ni disponer de la función de transición en forma cerrada: le basta con un *simulador generativo* del problema, lo que lo hace idóneo para el GSMDP de la Sección 2.3.2, cuya dinámica se define precisamente mediante una cola de eventos simulables.

Cada **iteración** del algoritmo clásico encadena cuatro fases [6]:

1. **Selección.** Partiendo de la raíz, se desciende por el árbol eligiendo en cada nodo un hijo

según una *política de selección* (descrita más abajo) hasta alcanzar un nodo expandible, esto es, un nodo con al menos una acción aún no explorada.

2. **Expansión.** Se añade al nodo expandible un nuevo hijo correspondiente a una de sus acciones no exploradas, extendiendo el árbol en un nivel.

3. **Simulación (*rollout*).** Desde el nodo recién creado se ejecuta una *política de simulación* (*rollout policy*), típicamente aleatoria o heurística, que completa una trayectoria hasta un estado terminal. Se evalúa la función objetivo sobre esa trayectoria, obteniendo una recompensa que se convierte en la puntuación de la hoja.

4. **Retropropagación.** La recompensa obtenida se propaga hacia arriba, actualizando las puntuaciones de todos los nodos del camino recorrido desde la hoja hasta la raíz.

Cada nodo j mantiene dos estadísticos: el número de visitas n_j y la suma (o media) de recompensas recibidas, a partir de los cuales se estima su valor empírico $\bar{X}_j$. Tras agotar el presupuesto de cómputo, el algoritmo devuelve la acción correspondiente al hijo de la raíz considerado mejor (que es habitualmente el más visitado gracias a la política de selección).

El dilema exploración–explotación y la cota de confianza superior (UCB). El núcleo de MCTS reside en la política de selección, que debe resolver en cada nodo el clásico dilema entre **explotación** (descender por las ramas que hasta ahora han dado mejores recompensas) y **exploración** (visitar ramas poco muestreadas cuyo valor es todavía incierto). Este es exactamente el dilema del problema de la tragaperras (*multi-armed bandit problem*) [18], en el que un jugador debe repartir sus tiradas entre varias máquinas tragaperras de recompensa desconocida.

La solución a este problema es el método de la **cota de confianza superior** (*Upper Confidence Bound*, UCB). En su formulación UCB1 [2], a cada brazo j se le asocia una cota optimista de su valor, suma de su media empírica y de un término de incertidumbre decreciente con el número de visitas, y se elige siempre el brazo de mayor cota:

$$\mathrm{UCB}_j \;=\; \underbrace{\bar{X}_j}_{\text{explotación}} \;+\; \underbrace{C\sqrt{\frac{\ln N}{n_j}}}_{\text{exploración}}, \tag{2.32}$$

donde $\bar{X}_j$ es la recompensa media del brazo j, n_j su número de visitas, $N = \sum_k n_k$ el total de tiradas y $C > 0$ una constante que regula el peso de la exploración. El término de exploración

crece logarítmicamente con N y decrece con n_j: un brazo poco visitado recibe un *bonus* alto que fuerza su exploración, mientras que un brazo muy visitado se juzga casi exclusivamente por su media empírica. La métrica utilizada para evaluar una política de tiradas se denomina arrepentimiento (*regret*), es la diferencia entre la puntuación máxima obtenible y la puntuación esperada de la política. UCB1 garantiza un arrepentimiento acotado polinómicamente [2].

Al trasladar UCB1 a la fase de selección de MCTS (tratando cada nodo como una tragaperras cuyos brazos son sus hijos) se obtiene el algoritmo **UCT** (*UCB applied to Trees*) [18], la variante de MCTS más difundida. En UCT, la cota de la Ecuación (2.32) se aplica recursivamente en cada nivel del árbol, balanceando exploración y explotación de forma local en cada nodo.

De MCTS a Dec-MCTS: aportaciones de Best et al. El algoritmo de Best et al. [5] adapta MCTS al contexto multi-robot descentralizado. Su problema de partida (planificación de secuencias de acciones para percepción activa con múltiples robots) difiere del MRTAU abordado en este trabajo, pero comparte con él la estructura esencial: cada robot debe planificar una secuencia de acciones para optimizar una función objetivo global $g(\mathcal{B})$ que depende de los planes de todos los robots, $\mathcal{B} = \{\mathcal{B}^1, \ldots, \mathcal{B}^n\}$, sin conocer directamente los planes ajenos $\mathcal{B}^{(-i)}$. Esta es precisamente la situación impuesta por la observabilidad parcial del DEC-POGSMDP (Sección 2.3.3). Las aportaciones de Best et al. relevantes para este trabajo son cuatro:

1. **Un árbol local por robot.** En lugar de un único árbol conjunto sobre el espacio de acciones de todos los robots (que crecería exponencialmente con el número de agentes), cada robot R_i mantiene su propio árbol $\mathcal{T}^i$, que contiene únicamente sus acciones. Esto reduce drásticamente el factor de ramificación y habilita una ejecución del algoritmo en paralelo

2. **Distribución de probabilidad sobre planes y comunicación.** Cada robot resume su árbol en una distribución de probabilidad ψ^i sobre un subconjunto reducido de sus secuencias de acciones más prometedoras, y la comunica periódicamente a sus vecinos. La coordinación ocurre en la fase de simulación: durante los rollouts, el robot R_i muestrea los planes de los demás $\mathcal{B}^k \sim \psi^k$ a partir de las distribuciones recibidas, de modo que la simulación avanza utilizando la información muestreada para determinar las preferencias de los demás robots. Esta representación probabilística de los planes ajenos contrasta con la representación determinista (una sola tarea o paquete) de los algoritmos de pujas

y constituye la principal aportación conceptual del método.

3. **Selección por UCT descontado (D-UCT).** Como las distribuciones de los vecinos cambian entre rondas de comunicación, la recompensa observada en cada nodo del árbol puede variar: una misma secuencia de acciones puede valer mucho o poco según lo que se asuma que harán los demás. Best et al. denominan *breakpoints* a estos cambios abruptos en la distribución de recompensa. Para que la búsqueda se adapte a ellos, sustituyen UCT por una variante descontada, **D-UCT**, que pondera cada simulación pasada por un factor de descuento γ_D que decae con su antigüedad, otorgando así más peso a las simulaciones recientes (obtenidas con las distribuciones de vecinos más actualizadas). D-UCT generaliza al D-UCB, aplicado al problema de la tragaperras no estacionario de Garivier y Moulines [10].

4. **Optimización de la distribución y utilidad local.** El algoritmo original incorpora otras dos modificaciones que en este trabajo se han adoptado solo parcialmente. Por un lado, una fase explícita de optimización de la distribución mediante descenso de gradiente, inspirada en métodos variacionales y en la teoría de *probability collectives*, que minimiza la divergencia de Kullback–Leibler entre la distribución producto de los robots y la distribución conjunta óptima. Por otro, el empleo de una *utilidad local* f^i que mide la contribución *marginal* del robot R_i al objetivo global (la diferencia entre la utilidad del equipo con y sin la participación de R_i), más sensible al plan propio que el objetivo global g. Como se detalla a continuación, la implementación de este trabajo prescinde de ambas en favor de alternativas más simples que, en las pruebas realizadas, resultaron más eficaces para el problema MRTAU.

El algoritmo implementado. La implementación desarrollada en este trabajo es una combinación pragmática del MCTS clásico con aquellas modificaciones de Best et al. [5] que aportan valor al problema MRTAU. En concreto, se conservan el árbol local por robot, la comunicación de distribuciones sobre planes y la selección D-UCT, y se sustituye la optimización variacional de la distribución por una construcción directa a partir de las visitas del árbol. A continuación se describen sus componentes.

(i) Estructura del árbol y espacio de acciones. El árbol $\mathcal{T}^i$ del robot R_i se inicializa en su estado observado. Las aristas son las acciones locales del DEC-POGSMDP (Sección 2.3.3): ExecuteTask(T_j) para cada tarea pendiente alcanzable por batería, Recharge (salvo que

la batería esté ya prácticamente llena, para evitar expandir acciones innecesarias) y FINISH cuando no quedan tareas pendientes. Un camino de la raíz a un nodo representa, por tanto, una secuencia de tareas $\mathcal{B}^i = (T_{j_1}, T_{j_2}, \dots)$, es decir, un plan análogo al paquete (*bundle*) de CBBA (Sección 2.4.2), con la excepción de incluir también las acciones RECHARGE y FINISH.

(ii) La política de selección D-UCT. Cada nodo j almacena, en lugar del par clásico $(n_j, \bar{X}_j)$, dos puntuaciones descontadas: las visitas descontadas $N_j^{\gamma_D}$ y la suma de recompensas descontada $Q_j^{\gamma_D}$, definidas sobre el índice de iteración global t como

$$N_j^{\gamma_D} \;=\; \sum_{u=1}^{t} \gamma_D^{\,t-u}\, \mathbf{1}_{\{j \text{ visitado en } u\}}, \qquad Q_j^{\gamma_D} \;=\; \sum_{u=1}^{t} \gamma_D^{\,t-u}\, F_u\, \mathbf{1}_{\{j \text{ visitado en } u\}}, \tag{2.33}$$

donde F_u es la recompensa de la simulación u y $\gamma_D \in (0,1)$ es el factor de descuento. La media empírica descontada del nodo es $\bar{X}_j^{\gamma_D} = Q_j^{\gamma_D}/N_j^{\gamma_D}$. La política de selección elige el hijo que maximiza la cota D-UCT:

$$\text{D-UCT}_j \;=\; \bar{X}_j^{\gamma_D} \;+\; 2\,C_p \sqrt{\frac{\ln N_{\text{pad}}^{\gamma_D}}{N_j^{\gamma_D}}}, \qquad N_{\text{pad}}^{\gamma_D} = \sum_k N_k^{\gamma_D}, \tag{2.34}$$

con la convención de que todo hijo aún no visitado ($N_j^{\gamma_D} \approx 0$) tiene prioridad absoluta (cota $+\infty$), lo que garantiza que cada acción se explore al menos una vez antes de reincidir.

(iii) Simulación con cola de eventos y muestreo de los planes ajenos. Tanto la expansión de nodos como la simulación de *rollouts* dependen fundamentalmente de un procedimiento de simulación virtual partiendo de un estado real (el correspondiente al nodo raíz). Este simulador materializa la dinámica del GSMDP descrita en la Sección 2.3.2 mediante una cola de eventos idéntica a la del simulador real. Cada *rollout* parte de una réplica del estado observado y de una cola inicializada con los eventos pendientes (decisiones de robots disponibles, finalizaciones de tareas en curso, expiraciones de ventanas). El sistema avanza extrayendo siempre el evento de menor instante de disparo. La novedad respecto a un MCTS de un solo agente es el tratamiento de los robots:

- El **robot planificador** R_i se rige, mientras su trayectoria permanece dentro del árbol, por la política D-UCT (selección y expansión); una vez sale del árbol, por la *política de rollout* descrita en el punto (iv).
- Cada **robot vecino** R_k ($k \neq i$) sigue el plan muestreado $\mathcal{B}^k \sim \psi^k$ de su distribución comunicada, ejecutando en orden las tareas de su paquete; si lo agota o alguna deja de

ser válida, recurre a la misma política de rollout.

De este modo, una sola simulación evalúa la función objetivo sobre un escenario conjunto $\mathcal{B} = \mathcal{B}^i \cup \mathcal{B}^{(-i)}$ coherente con las intenciones muestreadas de los compañeros, que es la forma en que Dec-MCTS aproxima $\mathbb{E}_{\psi^{(-i)}}[\, g \mid \mathcal{B}^i \,]$.

(iv) Política de rollout marginal-aware. La política por defecto de los *rollouts* no es aleatoria (como muchas otras implementaciones definen), sino que sigue una heurística que tiene en cuenta de manera explícita los planes muestreados de los demás robots. Para ello se precomputa, a partir de los paquetes muestreados $\mathcal{B}^{(-i)}$, una **probabilidad de bloqueo** p_j^{blq} por cada tarea T_j, que estima la probabilidad de que la tarea sea completada por los demás robots: si el número de vecinos que incluyen T_j en su paquete alcanza la coalición requerida q_j, se toma $p_j^{\mathrm{blq}} = p_{\mathrm{MAX}}^{\mathrm{blq}}$; en otro caso, $p_j^{\mathrm{blq}} = 0$ (la tarea no podrá completarse sin la participación de R_i, por lo que conviene que este la incluya en su plan). La constante $p_{\mathrm{MAX}}^{\mathrm{blq}} \in (0, 1]$ es un hiperparámetro del algoritmo que conviene optimizar (intuitivamente, modela la confianza en que los demás robots realmente van a ejecutar la tarea). Esta cantidad codifica de alguna manera el razonamiento sobre coaliciones que, en los algoritmos de pujas, realizaba el factor de cooperación de la Ecuación (2.21).

Con ella, la política de *rollout* asigna a cada tarea pendiente y alcanzable un valor marginal y un coste, y selecciona la de mayor cociente valor/coste:

$$\mathrm{score}(T_j) \;=\; \frac{\rho_j \left(1 - p_j^{\mathrm{blq}}\right)}{\mathrm{máx}\big(\varepsilon,\; \Delta_{ij}^{\mathrm{viaje}} + \bar{d}_j\big)}, \tag{2.35}$$

donde el numerador es el valor marginal esperado (la recompensa de la tarea, ponderada por su probabilidad de éxito y descontada la probabilidad de que ya la completen otros), $\Delta_{ij}^{\mathrm{viaje}} = d(x_i, x_j)/v_i$ es el tiempo de viaje, $\bar{d}_j$ la duración esperada de la Ecuación (2.20) y ε evita la división por cero. Si ninguna tarea es alcanzable por batería se opta por RECHARGE, y si no quedan tareas pendientes, por FINISH. Esta política *marginal-aware* guía los *rollouts* hacia trayectorias verosímiles y coordinadas, lo que reduce la varianza de las estimaciones frente a una política puramente aleatoria y permite obtener estimaciones de puntuación más precisas en un menor número de iteraciones.

(v) Construcción y comunicación de la distribución. Tras cada bloque de iteraciones, el robot resume su árbol en una distribución ψ^i de probabilidades sobre sus planes y la comparte con sus vecinos. A diferencia del algoritmo original, que optimiza esta distribución

Cuadro 2.3: Decisiones de diseño de la implementación frente a la propuesta original de Best et al. [5].

Componente	**Best et al. [5]**	**Implementación propia**
Árbol por robot	sí	sí
Selección	D-UCT	D-UCT
Comunicación	distribución sobre planes	distribución sobre planes
Distribución ψ^i	descenso de gradiente (var.)	visitas descontadas norm. (2.36)
Objetivo retropropagado	utilidad local f^i (marginal)	objetivo global g
Política de rollout	heurística por problema	*marginal-aware* (2.35)
Coordinación adicional	construcción de ψ^i	prob. de bloqueo (coaliciones q_j)

por descenso de gradiente, aquí se construye directamente a partir de la evidencia acumulada en el árbol: para cada hijo de la raíz se forma el plan que resulta de seguir, a partir de él, la cadena de descendientes más visitados (en visitas descontadas), y se le asigna una probabilidad proporcional a las visitas descontadas de dicho hijo,

$$\psi^i(\mathcal{B}^i) \;=\; \frac{N_c^{\gamma_D}}{\sum_{c'} N_{c'}^{\gamma_D}}, \tag{2.36}$$

donde c es el hijo de la raíz que encabeza el plan $\mathcal{B}^i$. La distribución resultante es la que reciben los vecinos para muestrear $\mathcal{B}^i$ en sus propias simulaciones, cerrando así el ciclo de coordinación.

(vi) Decisión y reaprovechamiento del árbol (replanning). Cuando la cola de eventos solicita finalmente una acción a R_i, este devuelve la acción del hijo de la raíz con mayor número de visitas descontadas (política de explotación robusta, preferida a la de mayor media por su menor sensibilidad al ruido). Acto seguido *re-enraíza* el árbol en ese hijo, descartando el resto de ramas y conservando el subárbol correspondiente a la acción elegida, de modo que el conocimiento acumulado sobre el futuro se reutiliza en la siguiente ronda de planificación en lugar de recomputarse desde cero. Al re-enraizar se consolidan las marcas temporales del subárbol resultante (se aplica el descuento pendiente y se actualiza el instante de referencia), evitando que la información heredada contenga errores acumulados de estimaciones previas.

La estructura global del algoritmo se resume en el Algoritmo 5 (bucle *anytime* y decisión) y en el Algoritmo 6 (una iteración de *rollout*). La Tabla 2.3 contrasta, finalmente, las decisiones de diseño de esta implementación con la propuesta original de Best et al.

Dec-MCTS es, de los algoritmos analizados, el que materializa con mayor fidelidad el espíritu del DEC-POGSMDP (Sección 2.3.3): cada robot resuelve un subproblema local de un

Algoritmo 5 DEC-MCTS — bucle *anytime* y decisión para R_i

Planificación de fondo (se invoca de forma continua en segundo plano)

function PERFORMBACKGROUNDPLANNING(observación o_i)
  $\psi^{(-i)} \leftarrow$ RECEIVEDISTRIBUTIONS() ▷ actualiza distribuciones conocidas
  if el árbol $\mathcal{T}^i$ no existe **then**
    inicializar $\mathcal{T}^i$ con raíz en el estado actual
  end if
  for $ITER_{\text{back}}$ iteraciones **do**
    ITERATION($o_i, \mathcal{T}^i, \psi^{(-i)}$) ▷ Algoritmo 6
  end for
  $\psi^i \leftarrow$ EXTRACTDISTRIBUTION($\mathcal{T}^i$) ▷ Ec. (2.36)
  COMMUNICATEDISTRIBUTION(ψ^i) ▷ intercambio asíncrono con vecinos
end function

Decisión (se invoca solo en los instantes de decisión)

function DECIDENEXTACTION(observación o_i)
  $\psi^{(-i)} \leftarrow$ RECEIVEDISTRIBUTIONS()
  if $\mathcal{T}^i$ vacío **then**
    for $ITER_{\text{emerg}}$ iteraciones **do** ▷ iteraciones de emergencia
      ITERATION($o_i, \mathcal{T}^i, \psi^{(-i)}$)
    end for
  end if
  $c^* \leftarrow$ hijo de la raíz con mayor $N_c^{\gamma_D}$ ▷ explotación robusta
  re-enraizar $\mathcal{T}^i$ en c^*
  $\psi^i \leftarrow$ EXTRACTDISTRIBUTION($\mathcal{T}^i$)
  COMMUNICATEDISTRIBUTION(ψ^i)
  return c^*
end function

solo agente (su propio árbol) sobre el que razona a largo plazo, y la única información que cruza la frontera de la observabilidad parcial son las distribuciones ψ^i comunicadas, que sustituyen a la observabilidad global ausente por una estimación probabilística de las intenciones ajenas. Su naturaleza *anytime* (la búsqueda corre de fondo y mejora con el tiempo) y su tolerancia a la comunicación intermitente (un robot que no recibe mensajes sigue planificando con la última distribución conocida, o con una política por defecto si nunca la recibió) lo hacen especialmente adecuado para el escenario distribuido. A cambio, es el algoritmo más costoso en cómputo y el de implementación más delicada, lo que motiva la comparación empírica frente a los *baselines* y los métodos de pujas.

Algoritmo 6 ITERACIÓN de Dec-MCTS — una simulación con cola de eventos

Entrada: observación o_i, distribuciones de vecinos $\psi^{(-i)}$, árbol $\mathcal{T}^i$

$\mathcal{B}^{(-i)} \leftarrow$ SAMPLEBUNDLES($\psi^{(-i)}$) ▷ muestrea un plan por vecino
$p^{\mathrm{blq}} \leftarrow$ BLOCKPROBABILITY($\mathcal{B}^{(-i)}$) ▷ coaliciones q_j de los vecinos
$s \leftarrow$ estado inicial a partir de o_i
$Q \leftarrow$ cola de eventos inicial; *camino* $\leftarrow$ [raíz]
enArbol $\leftarrow$ **true**; *nodo* $\leftarrow$ raíz
while $Q \neq \varnothing$ **y** s no es final **do**
    $e \leftarrow$ extraer de Q el evento de menor instante; avanzar el reloj de s
    if e es ROBOTDECISION de R_i **then**
        if *enArbol* **y** hay acción no expandida **then**
            expandir *nodo* con esa acción
            camino $\leftarrow$ *camino* + [*nodo*]; *enArbol* $\leftarrow$ **false**
        else if *enArbol* **then**
            nodo $\leftarrow$ SELECTD-UCT(*nodo*) ▷ Ec. (2.34)
            camino $\leftarrow$ *camino* + [*nodo*]
        else
            acción $\leftarrow$ ROLLOUTPOLICY(R_i, s, p^{blq}) ▷ Ec. (2.35)
        end if
    else if e es ROBOTDECISION de un vecino R_k **then**
        acción $\leftarrow$ siguiente tarea de $\mathcal{B}^k$ (o ROLLOUTPOLICY() si $\mathcal{B}^k$ no se puede seguir)
    end if
    aplicar la *acción* y procesar otros eventos de tarea
end while
$g \leftarrow$ puntuación del estado final s ▷ recompensa del rollout
retropropagar g por *camino*: $N_j^{\gamma_D} \mathrel{+}= 1$, $Q_j^{\gamma_D} \mathrel{+}= g$ (con descuento)

Capítulo 3

Estado del arte

El capítulo anterior (Capítulo 2) ha establecido el marco formal del trabajo: la definición del problema, la familia de modelos de decisión y la formalización del problema MRTAU distribuido como un DEC-POGSMDP, junto con los algoritmos que se compararán. Durante esta exposición ya se introdujo, sobre la marcha, buena parte de la literatura relevante, este capítulo no repite las definiciones ni las descripciones algorítmicas ya descritas. Su objetivo es complementario: situar el trabajo en el estado actual de la investigación, organizar las contribuciones existentes en familias y compararlas de forma crítica frente a las particularidades del problema abordado.

Conviene además distinguir dos planos que con frecuencia se entremezclan. Uno es el de la *formalización*: cómo plantear matemáticamente el problema, cuestión de la que se ocupan los modelos de decisión secuencial (Sección 3.2). El otro es el de la *resolución*: qué algoritmo computa una política sobre ese modelo (Sección 3.3). Ambos planos están relacionados, pero no deben confundirse: un mismo modelo admite múltiples métodos de resolución.

Como criterios de clasificación usaremos los ya introducidos en la Sección 2.1: el momento del cómputo (*online* frente a *offline*) y la arquitectura de control (centralizada frente a descentralizada). Añadiremos además como criterio adicional el tratamiento de la incertidumbre, componente esencial de todo el trabajo. Conviene acotar también el alcance: el foco se mantiene en los enfoques de planificación y coordinación para MRTA cooperativo. Quedan deliberadamente fuera las aproximaciones de aprendizaje por refuerzo multiagente, que aprenden políticas a partir de la experiencia en lugar de planificar sobre un modelo, y los enfoques de inteligencia de enjambre, por alejarse del planteamiento aplicado en este trabajo.

3.1. El problema MRTA y su evolución

El problema de asignación de tareas multi-robot tiene sus raíces en la optimización combinatoria clásica. Su forma más elemental (asignar tareas a agentes minimizando un coste) es el problema de asignación resoluble en tiempo polinómico mediante el método húngaro [20]. La incorporación de secuencias de tareas, rutas y restricciones temporales lo emparenta con

el *Vehicle Routing Problem* y, en presencia de ventanas de ejecución, con el VRPTW [28], conexión ya señalada al describir las particularidades del escenario (Sección 2.1.1).

El estudio de MRTA como disciplina propia se consolida con la taxonomía de Gerkey y Matarić [12], que clasifica las variantes según los ejes ST/MT, SR/MR e IA/TA, y establece además la dureza computacional de la mayoría de ellas. Trabajos posteriores ampliaron este marco para capturar las dependencias entre tareas y agentes, en particular las *cross-schedule dependencies* en las que se sitúa este problema [19] y las restricciones temporales y de orden [21]. Revisiones más recientes sintetizan la evolución del campo y sus principales familias de solución [17]. Frente a estas formulaciones, el problema abordado en este trabajo (Sección 2.1.1) combina simultáneamente un conjunto de características que rara vez aparecen juntas en la literatura: tareas *multi-robot* con coaliciones de tamaño q_j, ventanas de ejecución $[t_j^s, t_j^l]$, heterogeneidad mediante la función de compatibilidad $Cap(\cdot,\cdot)$ y restricciones de batería con recarga. Es esta acumulación de requisitos, más que cualquiera de ellos por separado, la que hace inadecuados los enfoques diseñados para variantes más simples.

3.2. Formalización del problema: modelos de decisión

El primer plano es el de la formalización: una vez delimitado el problema, ¿con qué lenguaje matemático se plantea? La respuesta predominante para la toma de decisiones secuencial bajo incertidumbre es la familia de los Procesos de Decisión de Markov presentada en la Sección 2.2. Su extensión natural al caso cooperativo con observabilidad parcial es el DEC-POMDP [4, 23], marco que captura con precisión la coordinación descentralizada con información local. La virtud de esta familia es su tratamiento de la incertidumbre y de la observabilidad parcial. Su contrapartida es la complejidad: resolver óptimamente un DEC-POMDP de horizonte finito es NEXP-completo [4], lo que restringe la resolución exacta a instancias diminutas y obliga en la práctica a recurrir a métodos aproximados.

Para acercar estos modelos a problemas robóticos reales, una decisión fundamental para este trabajo es el uso de *macro-acciones*. Una macro-acción es una acción de alto nivel y **duración variable** que encapsula internamente toda una secuencia de acciones primitivas de bajo nivel: en lugar de decidir paso a paso (avanzar, girar, etc.), el agente elige entre acciones del tipo "desplazarse a la tarea T_j y ejecutarla". Esto tiene dos consecuencias importantes. En primer lugar, **reduce el horizonte efectivo de decisión**: el agente solo decide en los instantes en que una macro-acción termina y debe elegir la siguiente, en lugar de en cada paso elemental,

lo que recorta drásticamente el número de puntos de decisión. En segundo lugar, el uso de macro-acciones obliga a introducir un modelo que sea capaz de **razonar sobre duraciones continuas**. Para ello usamos la lógica de los SMDP, que modelan los tiempos de ejecución como variables aleatorias.

El DEC-POSMDP [24] y los marcos asociados de planificación con macro-acciones [1] explotan esta idea para integrar la descentralización y la observabilidad parcial del DEC-POMDP con las duraciones continuas del SMDP, salvando así la principal barrera que aleja a estos modelos de los problemas robóticos reales. La conexión con este trabajo es directa: las acciones locales de cada robot (ExecuteTask(T_j), Recharge y Finish) son precisamente macro-acciones, ya que cada una abarca desplazamiento, espera, ejecución de duración continua y estocástica en lugar de un solo movimiento atómico.

Este es, por tanto, el punto de partida del modelo definitivo propuesto en la Sección 2.3.3. Ahora bien, las macro-acciones resuelven el problema del horizonte y de las duraciones, pero no el del **solapamiento de acciones concurrentes** entre robots, el DEC-POSMDP hereda del SMDP la incapacidad de tratar correctamente dicha concurrencia (el *leak temporal* de la Sección 2.2.4). Es esta carencia la que motiva su extensión con la lógica de eventos del GSMP, dando lugar al DEC-POGSMDP.

Disponer de un modelo no equivale, sin embargo, a saber resolverlo. La cuestión de qué algoritmo computa una política sobre estos modelos y, en particular, de cómo lidiar con la intratabilidad de su resolución exacta pertenece al segundo plano y es el objeto de la siguiente sección.

3.3. Métodos de resolución

Fijada una formalización, queda el problema de obtener la política óptima. Esta sección recorre las tres grandes familias de métodos de resolución. La primera, la programación dinámica, resuelve el modelo de forma exacta pero solo es viable en instancias diminutas; las otras dos, los métodos basados en mercado y la planificación basada en muestreo, no garantizan la optimalidad pero ofrecen soluciones en tiempos razonables de computación.

3.3.1. Programación dinámica exacta

El método de resolución nativo de los modelos de la sección anterior es la programación dinámica: algoritmos como la iteración de valores o la iteración de políticas calculan la política óptima resolviendo la ecuación de Bellman (Ecuación (2.5)). Para un MDP completamente observable con un espacio de estados reducido, estos métodos son exactos y eficientes. Su coste, sin embargo, crece muy rápidamente con la complejidad del modelo: la resolución exacta de un POMDP es intratable en el caso general por la dimensionalidad del espacio de creencias (Sección 2.2.2), y la de un DEC-POMDP es directamente NEXP-completa [4]. En consecuencia, la programación dinámica exacta solo resulta aplicable a instancias diminutas y funciona, en la práctica, más como referencia conceptual de optimalidad que como método utilizable a la escala del problema MRTAU. Las dos familias siguientes sacrifican esa optimalidad a cambio de escalabilidad.

3.3.2. Enfoques basados en mercado

La coordinación basada en mercado traslada a los sistemas multi-robot la lógica de las subastas: las tareas se asignan a los agentes que emiten la mejor puja según una función de *score*, y los conflictos se resuelven mediante negociación. Esta línea, heredera del *Contract Net Protocol*, fue popularizada en robótica por sistemas como MURDOCH [11] y sistematizada en la revisión de Dias et al. [9]. Su exponente descentralizado más influyente es el *Consensus-Based Auction Algorithm* (CBAA) y su extensión por paquetes, el *Consensus-Based Bundle Algorithm* (CBBA) [7], en los que cada agente puja localmente y los conflictos se resuelven mediante una fase de consenso sobre comunicación entre vecinos. La generalización a tareas que requieren varios agentes se aborda en el *Consensus-Based Grouping Algorithm* (CBGA) [15]. A diferencia de la programación dinámica, estos métodos no resuelven el modelo decisión-teórico, sino que sustituyen el cálculo de la política óptima por una heurística de asignación.

La gran ventaja de estos métodos es que encajan a la perfección con el enfoque descentralizado: son escalables, robustos ante fallos y operan con una comunicación acotada. Su principal limitación, sin embargo, reside en que su coordinación se concentra exclusivamente en una heurística de *score* de naturaleza determinista, que ni razona sobre el horizonte largo (más allá del *lookahead* del paquete de CBBA) ni modela explícitamente la incertidumbre. Como consecuencia directa de ese corto horizonte de visión, el rendimiento del algoritmo

depende fuertemente del diseño de dicha heurística: para garantizar un buen comportamiento a largo plazo, el *score* no puede limitarse a recompensar la tarea inmediata, sino que debe incorporar componentes adicionales como fomentar la formación de coaliciones para no dejar robots esperando indefinidamente o penalizar los niveles de batería muy bajos. Esto impide adoptar directamente la propia función de recompensa del problema (Ecuación (2.2)) como heurística y obliga a establecer prioridades e introducir constantes de ponderación, lo que convierte el ajuste de la heurística en una tarea delicada y específica del escenario. En su formulación original, además, computan una asignación estática *a priori* sobre un escenario determinista. Como se detalla en la Sección 2.4.2, las implementaciones de este trabajo se alejan de esos supuestos (admiten coaliciones, se ejecutan de forma *online* y operan sobre un escenario estocástico mediante una aproximación determinista del *score*), lo que ilustra tanto el valor como los límites de esta familia frente a un problema bajo incertidumbre.

3.3.3. Planificación basada en muestreo y búsqueda en árbol

Una tercera familia renuncia a resolver el modelo de forma cerrada y planifica mediante simulación. La búsqueda en árbol de Monte Carlo (MCTS) [6], y en particular su variante UCT [18] basada en el problema de la tragaperras [2], construye incrementalmente un árbol de búsqueda a partir de simulaciones aleatorias. Su rasgo distintivo es que solo requiere un simulador generativo del problema, sin necesidad de la función de transición en forma cerrada, lo que la hace idónea para una dinámica definida por una cola de eventos como la del GSMDP (Sección 2.3.2). Su carácter *anytime* la alinea, además, con la necesidad de cómputo acotado de los métodos *online* (Sección 2.1.3).

La adaptación de MCTS al contexto multi-robot descentralizado la proporciona Dec-MCTS [5], en el que cada robot mantiene su propio árbol y comunica a sus vecinos una distribución de probabilidad sobre sus planes, empleando una variante descontada de UCT (D-UCT) para adaptarse a los planes no estacionarios ajenos [10]. Esta representación probabilística de las intenciones de los compañeros constituye la forma de coordinación más rica de las analizadas y es la base de la implementación descrita en la Sección 2.4.3. Su precio es un coste de cómputo elevado y una implementación delicada, lo que justifica su comparación empírica frente a las familias anteriores. El coste de cómputo introducido queda plenamente justificado por la naturaleza del problema. Debido a su carácter *online*, el tiempo requerido para desplazarse, recargar o ejecutar una tarea puede utilizarse como tiempo de cómputo para

enriquecer una próxima decisión en lugar de ser tiempo "perdido" como en las aproximaciones *on-request*.

3.4. Asignación de tareas bajo incertidumbre

La inmensa mayoría de la literatura clásica de MRTA, incluidas las taxonomías [12, 19], asume un escenario *determinista*. El tratamiento integrado de la incertidumbre está comparativamente menos explorado y constituye un área activa. El trabajo de Choudhury et al. [8] sobre asignación dinámica de tareas bajo incertidumbre y restricciones temporales es, por su planteamiento, uno de los más cercanos en espíritu al abordado aquí.

Las herramientas formales para tratar esta incertidumbre se han presentado en el Capítulo 2: los modelos de decisión bajo observabilidad parcial y, de forma central, los procesos semi-Markovianos generalizados [13, 29], que modelan el solapamiento de acciones de duración continua mediante eventos con relojes ocultos. Lo que la literatura rara vez ofrece es la *combinación* de estas herramientas con el resto de exigencias del problema: descentralización plena, formación de coaliciones, ventanas de ejecución y restricciones de batería, todo ello de manera simultánea. En particular, el problema de la fuga de información temporal (Sección 2.2.4), que surge al modelar duraciones continuas concurrentes bajo observabilidad total, no suele plantearse de forma explícita en el contexto de MRTA, pese a ser esencial para la corrección del modelo.

3.5. Posicionamiento y aportaciones

En el plano de la formalización, los modelos de decisión (Sección 3.2) describen el problema pero su resolución exacta mediante programación dinámica (Sección 3.3.1) es generalmente intratable. En el plano de la resolución aproximada, los métodos basados en mercado (Sección 3.3.2) son escalables y descentralizados, pero condensan su coordinación en heurísticas deterministas con escasa anticipación, mientras que la planificación basada en muestreo (Sección 3.3.3) ofrece un punto intermedio *anytime* y descentralizable, a costa de un cómputo elevado.

Sobre este análisis se pueden definir las principales aportaciones de este trabajo. En primer lugar, la **formalización del problema MRTAU distribuido como un DEC-POGSMDP** (Sección 2.3.3): un modelo que, hasta donde se ha podido constatar, no aparece en la literatura

bajo este nombre y que se introduce como la progresión natural que resulta de dotar a un DEC-POSMDP de la lógica de eventos del GSMP, garantizando así el tratamiento correcto de las duraciones continuas concurrentes y la ausencia de *leak temporal*. En segundo lugar, la **definición unificada de los algoritmos como políticas de comunicación** que resuelven ese mismo DEC-POGSMDP con grados crecientes de coordinación (Sección 2.4), lo que permite comparar en igualdad de condiciones desde los *baselines* sin comunicación hasta la planificación distribuida de Dec-MCTS. En tercer lugar, y como hilo conductor de toda la memoria, la **transición desde el modelo centralizado de partida** (el MCTS centralizado de Pérez-Hurtado et al. [25]) **hacia una resolución puramente distribuida**: una reformulación que, como se argumentó en la Sección 2.3, no es un mero detalle de implementación, sino un cambio en la clase de complejidad del problema que obliga a sustituir la información global por mecanismos de coordinación basados en comunicación.

Capítulo 4

Materiales y métodos

Los dos capítulos anteriores han establecido la definición del problema DEC-MRTAU formalizado como un DEC-POGSMDP (Sección 2.3.3) y los cinco algoritmos que lo abordan, entendidos como políticas de comunicación con grados crecientes de coordinación (Sección 2.4). Este capítulo describe *cómo* se ha materializado todo ello en un sistema ejecutable: los materiales empleados, la arquitectura del software desarrollado y, sobre todo, las decisiones de implementación que no se deducen del modelo formal pero condicionan los resultados obtenidos.

La totalidad del código desarrollado, junto con los escenarios empleados y las herramientas de análisis, está disponible públicamente en `https://github.com/manuolias/DEC-MRTAU`, bajo licencia MIT.

4.1. Materiales y entorno de ejecución

La Tabla 4.1 resume las herramientas y el equipo empleados. El simulador y los cinco algoritmos se han desarrollado íntegramente en **C++17**, sin reutilizar código de terceros más allá de la biblioteca estándar y de un analizador de ficheros YAML.

La elección de C++ se debe a un requisito de rendimiento. El coste computacional del trabajo no está en el simulador, sino en Dec-MCTS: cada ronda de planificación de fondo ejecuta decenas de iteraciones y cada iteración simula un episodio completo del escenario hasta un estado terminal, de modo que una única ejecución puede llegar a simular millones de eventos. C++ permite además un control explícito de la memoria (los nodos del árbol de búsqueda se gestionan con punteros únicos, y el subárbol conservado al re-enraizar se transfiere sin copias), lo que evita el coste de recolección de basura que penalizaría a un lenguaje gestionado en un bucle tan intensivo.

Merece la pena destacar la modestia del equipo empleado. Todo el sistema se ejecuta en un ordenador portátil de gama media, sin acelerador gráfico ni requisitos de memoria apreciables: una ejecución individual ocupa unos pocos megabytes y un solo hilo, y la evaluación completa del catálogo definitivo se resuelve en unas pocas horas repartiendo el trabajo entre cuatro

Cuadro 4.1: Materiales y entorno de desarrollo y ejecución.

Componente	**Elección**
Lenguaje	C++17
Compilador	GNU g++ 13.3.0, optimización `-O3`
Sistema de construcción	CMake 3.28 + Make
Dependencia externa	yaml-cpp (carga de escenarios y configuración)
Escenarios y análisis	Python 3.12 (generación, extracción de métricas, cuaderno)
Control de versiones	Git
Sistema operativo	Ubuntu 24.04 LTS sobre WSL2 (núcleo 6.6)
Procesador	Intel Core i5-1135G7, 4 núcleos / 8 hilos a 2,4 GHz
Memoria	3,7 GiB disponibles para el entorno de ejecución

Cuadro 4.2: Estructura del repositorio.

Directorio	**Contenido**
`src/`	Motor: ejecutor de experimentos, simulador, escenario, estado y registro
`include/tau/`	Cabeceras del motor y los cinco algoritmos
`scenarios/`	Catálogos de escenarios en formato YAML (Capítulo 5)
`scripts/`	Generación de escenarios, lanzamiento de tandas y extracción de métricas
`logs/`	Registros de las ejecuciones
`analisis/`	Cuaderno de evaluación, tablas de resultados y figuras

procesos independientes. Esto no es un simple detalle, sino una propiedad deseable en el contexto del problema: los algoritmos evaluados están pensados para ejecutarse en tiempo real a bordo de robots con recursos limitados. Por ello, un método cuyo coste solo fuese asumible en una estación de trabajo dedicada sería difícilmente trasladable a un entorno real.

4.2. Organización del software

El código se organiza en torno a un único espacio de nombres (`tau`) y separa el motor (compilado a partir de ficheros de implementación) de los algoritmos (definidos íntegramente en cabeceras, lo que simplifica añadir o variar *solvers* sin tocar el resto del sistema). La Tabla 4.2 resume la estructura del repositorio.

Las clases principales se corresponden una a una con las piezas del modelo formal, lo que permite leer el código como una traducción directa del Capítulo 2. La clase `Scenario` representa la parte estática de una instancia (Definición 2.1): el grafo de nodos con sus distancias, las tareas con sus ventanas, probabilidades de éxito, distribuciones de duración y coaliciones requeridas, los robots con su batería, velocidad y lista de compatibilidades,

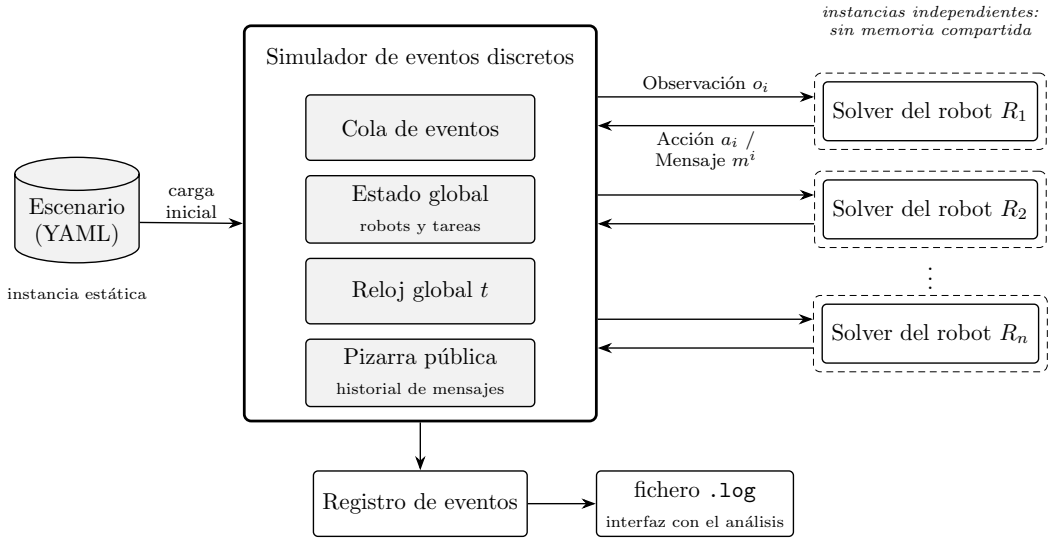


Figura 4.1: Arquitectura del simulador distribuido y flujo de datos entre sus componentes. Cada robot dispone de una instancia independiente de solver; la única información que cruza de un robot a otro es la comunicación de mensajes.

y las estaciones de recarga. Se carga desde un fichero YAML y no se modifica durante la ejecución. La clase `State` representa la parte dinámica, esto es, el estado global $s = (t, s^{R_1}, \ldots, s^{R_n}, s^{T_1}, \ldots, s^{T_m})$ de la Sección 2.3.1, con los estados operativos de robots y tareas definidos allí. La clase `Observation` materializa la observación local o_i de la Ecuación (2.18) y la clase `Action`, el espacio de acciones locales $\mathcal{A}_i$.

Una decisión fundamental es que el simulador no alberga un planificador único, sino un diccionario de *solvers* indexado por robot. Cada robot recibe su propia instancia del algoritmo, con su propio estado interno y el simulador nunca invoca un solver pasándole información de otro robot: la única entrada de la que dispone es la observación local que se le construye. Esta arquitectura reproduce fielmente la organización de un despliegue real, en el que cada robot ejecutaría su propia copia del algoritmo a bordo.

4.2.1. La simplificación de la comunicación

La comunicación entre robots aparece en el modelo formal como el conjunto de mensajes $\mathcal{M}_{\text{in}}$ de la observación (2.18), y se ha materializado de dos maneras distintas según la naturaleza de la información intercambiada.

En **CBAA y CBBA**, el intercambio de pujas que los Algoritmos 3 y 4 expresan mediante las funciones CommunicateBid y ReceiveBid se resuelve con llamadas directas: el robot

que está decidiendo evalúa a su vez las pujas de sus compañeros en lugar de recibirlas por un canal. Esta simplificación no altera el comportamiento del algoritmo, ya que la puja de la Ecuación (2.21) es una función determinista del estado del robot que la emite y todos los robots comparten la misma función de *score*: el valor calculado localmente coincide con el que el vecino habría difundido. En **Dec-MCTS**, en cambio, la información comunicada es una distribución sobre planes ψ^k que no puede reconstruirse desde fuera, pues depende del árbol de búsqueda privado de cada robot. Se emplea para ello una **pizarra pública**: una estructura mantenida por el simulador junto al estado, que asocia a cada robot la última distribución que ha publicado. Cada ronda de planificación de fondo sobrescribe la entrada correspondiente, y cada observación que se construye lleva la pizarra completa.

En ambos casos la comunicación se considera perfecta: inmediata, sin pérdidas, sin límite de ancho de banda y sobre un grafo de vecindad completo. Es una idealización deliberada y razonable para el objetivo perseguido, que es comparar estrategias de coordinación en igualdad de condiciones: introducir latencias o pérdidas añadiría un factor que afecta de manera distinta a cada algoritmo y dificultaría precisamente la comparación que se pretende establecer.

4.2.2. El registro de la ejecución

Cada ejecución produce un fichero de registro o *log*, que constituye la única salida del simulador. El fichero se organiza en tres bloques: una cabecera con la identificación de la ejecución (escenario, algoritmo, función de recompensa y número de réplica) y las dimensiones del problema; una traza cronológica de eventos, en la que cada línea registra con su marca temporal la aparición de tareas, robots y estaciones, los desplazamientos, el consumo de batería, el inicio y la resolución de cada tarea y los robots que terminan, recargan o agotan su batería; y un bloque final de métricas agregadas, entre las que se encuentran la recompensa obtenida y el tiempo real de cómputo consumido.

Registrar la traza completa, y no solo las métricas finales, tiene una clara ventaja: buena parte del análisis del Capítulo 6 no se apoya en el valor de la recompensa, sino en magnitudes derivadas de la traza que no estaban previstas al lanzar los experimentos, como el número de tareas que cada algoritmo llega siquiera a intentar. Poder reconstruirlas releyendo los registros, sin volver a ejecutar nada, es lo que permitió evaluar y diagnosticar problemas en el comportamiento de los algoritmos en lugar de limitarse a comparar sus rendimientos.

4.3. La función de recompensa

La función de utilidad de la Ecuación (2.1) pondera los diferentes objetivos mediante los coeficientes λ. Su implementación en código generaliza ligeramente esa expresión: la recompensa de un episodio se define como una combinación lineal de ocho componentes normalizados en $[0, 1]$, con pesos $k_1, \ldots, k_8$ cuya suma debe ser la unidad (la implementación lo verifica y rechaza cualquier configuración que no lo cumpla). Los tres primeros componentes recogen la proporción de tareas completadas, no pendientes y no fallidas; los tres siguientes, la proporción de robots disponibles, finalizados y no fallidos al finalizar el episodio; los dos últimos, el *makespan* máximo y la distancia total recorrida, ambos normalizados. La recompensa resultante está acotada en $[0, 1]$ por construcción.

Toda la experimentación emplea la configuración $k_1 = 1$ y $k_2, \ldots, k_8 = 0$. Es decir, la función de recompensa es exclusivamente la **proporción de tareas completadas**, que se corresponde con la Ecuación (2.2), tomando $\lambda_c = 1/m$ con recompensas unitarias $w_j = 1$ y anulando los demás términos. Esto se debe a que este es el objetivo fundamental a optimizar y la introducción de otros componentes podría desplazar el foco de la comparación. Se trata de evitar de esta manera que cualquier algoritmo priorice aumentar cualquier otra métrica sobre la proporción de tareas completadas.

4.4. El simulador distribuido de eventos discretos

El simulador es la pieza que sustituye al mundo físico: genera un entorno para los cinco algoritmos, ejecuta la dinámica del problema y determina las interacciones entre robots, tareas y estaciones. Su funcionamiento responde al paradigma de la **simulación de eventos discretos**. El tiempo no avanza en incrementos fijos, sino a saltos. Esto se realiza mediante un **reloj virtual** que se sitúa siempre en el instante del próximo evento pendiente (la llegada de un robot, el fin de una ejecución, el cierre de una ventana) y desde ahí salta al siguiente, sin la necesidad de recorrer el intervalo intermedio. Este reloj virtual es lo que permite la ejecución de miles de escenarios en un tiempo razonable.

El simulador es la realización en código de la dinámica del DEC-POGSMDP y, en particular, de la cola de eventos que la Sección 2.3.2 describe de forma abstracta. Dos aspectos de su diseño merecen detallarse: la gestión de la cola de eventos, de la que depende la corrección del modelo, y los dos hiperparámetros que determinan el ritmo de cómputo y comunicación

Cuadro 4.3: Tipos de evento del simulador, en orden de prioridad decreciente. A igualdad de instante de disparo se procesa primero el de prioridad más alta.

Evento	Efecto
Fin de tarea	Revela el desenlace, descuenta la batería y libera a la coalición
Inicio de tarea	Muestrea desenlace y duración de una tarea con coalición completa
Caducidad de tarea	Invalida una tarea cuya ventana se agota y libera a los robots en espera
Decisión de robot	Solicita una acción al solver del robot y la aplica
Planificación de fondo	Ejecuta una ronda de cómputo *anytime* y publica su distribución

de los algoritmos *anytime*.

4.4.1. La cola de eventos

El motor mantiene una cola de prioridad de eventos ordenada por instante de disparo. Cada evento registra su instante, su tipo, el robot y la tarea implicados y un valor entero de desempate generado aleatoriamente. La Tabla 4.3 recoge los cinco tipos existentes y su prioridad relativa. El bucle principal extrae repetidamente el evento más próximo, avanza el reloj global hasta su instante (nunca hacia atrás) y lo gestiona según su tipo, hasta que la cola se vacía o todos los robots han terminado su misión o agotado su batería. En este momento se da por finalizada la simulación.

La extracción sigue un orden lexicográfico sobre tres claves: primero el instante de disparo, después la prioridad del tipo y, por último, el valor aleatorio de desempate. Las dos últimas claves merecen justificación. La **prioridad por tipo** garantiza la coherencia del estado cuando varios eventos coinciden en el mismo instante: una finalización debe procesarse antes que cualquier decisión simultánea, para que el robot que queda libre decida sobre un estado ya actualizado, y la planificación de fondo se sitúa deliberadamente en último lugar, pues no altera el entorno físico y debe observar el estado ya consolidado. El **desempate aleatorio** evita establecer un orden de prioridad arbitrario entre robots o tareas: cuando varios robots deben decidir en el mismo instante, su orden se sortea en lugar de seguir el de sus identificadores. Sin este mecanismo, el robot R_1 elegiría siempre antes que R_2 ante un empate y tendría una ventaja sistemática al reservarse las tareas más apetecibles.

La decisión de diseño más relevante de la cola es la **separación entre el inicio y el fin de una tarea en dos eventos distintos**. Cuando la coalición de una tarea se completa, se programa un evento de inicio en el instante en que la ejecución puede comenzar. Al dispararse ese evento se muestrea de una sola vez todo lo que la ejecución tiene de aleatorio (el desenlace, según la

probabilidad de éxito ρ_j, y la duración, de la distribución normal correspondiente al desenlace obtenido) y con esos valores se programa el evento de finalización. Esa es la realización en código del **reloj oculto** del GSMP (Definición 2.6): el valor muestreado reside en la cola de eventos, que es una estructura interna del motor, y no en el estado. Como las observaciones que reciben los *solvers* se construyen exclusivamente a partir del estado, ningún robot puede conocer el instante de finalización ni el desenlace de una ejecución en curso; únicamente puede saber que la tarea está en ejecución y desde cuándo, que es información legítimamente observable. Es este mecanismo el que garantiza la ausencia de la fuga de información temporal discutida en la Sección 2.2.4 e ilustrada en la Figura 2.2. La alternativa natural, almacenar el instante de finalización en el estado de la tarea al comenzar la ejecución, habría reintroducido el *leak* temporal que se pretende evitar.

4.4.2. Hiperparámetros del motor de simulación

El motor introduce dos parámetros propios, que solo afectan a los algoritmos *anytime*. Conviene precisar que ambos se expresan en unidades de tiempo virtual, el del escenario simulado, y que cada ronda de planificación ejecuta un número fijo de iteraciones con independencia del tiempo de reloj real que consuma.

El primer parámetro es el **intervalo de planificación**, fijado en $0{,}1$ unidades de tiempo virtual. Cada robot con planificación *anytime* ejecuta una ronda de cómputo de fondo y publica su distribución con esa cadencia a lo largo de todo el episodio, de manera que un intervalo menor se traduce en más rondas de pensamiento y más intercambios de información por unidad de tiempo simulado.

El segundo es el **tiempo virtual de calentamiento**, fijado en 30 segundos virtuales, del que se deriva el número de rondas de planificación que se ejecutan antes de la primera decisión (con el intervalo anterior, 300 rondas). La presencia de un tiempo de calentamiento es fundamental para el correcto funcionamiento de los algoritmos *anytime*: solicitar a un robot su primera acción sin que se haya ejecutado previamente ninguna ronda de cómputo y comunicación implicaría decidir con un árbol de búsqueda vacío y sin ninguna información sobre las intenciones de sus compañeros. Esa primera decisión sería necesariamente desinformada y, dado que compromete al robot con una primera acción no necesariamente buena, este error podría arrastrarse durante el episodio completo. El calentamiento evita ese arranque en frío y garantiza que la primera decisión se tome en las mismas condiciones que todas las demás.

4.5. Dec-MCTS: decisiones e hiperparámetros de implementación

La Sección 2.4.3 describe el algoritmo Dec-MCTS implementado: el árbol local por robot, la selección D-UCT, el muestreo de los planes ajenos o la política de *rollout*. Esta sección no repite nada de ello; se ocupa de lo que aquella dejó abierto, esto es, de los refinamientos que separan la descripción del algoritmo de la implementación final y de los valores concretos de sus hiperparámetros.

4.5.1. Refinamientos sobre el esquema base

Una implementación literal del esquema MCTS canónico rinde por debajo de sus posibilidades en este problema por factores estructurales propios. Es por eso que, durante el proceso de construcción y refinamiento iterativo del algoritmo, se han establecido una serie de variaciones para el correcto funcionamiento del mismo. Estos cambios constituyen detalles de esta implementación específica; no son cambios fundamentales del algoritmo y por eso se han incluido en este apartado.

La recompensa retropropagada. Las cotas de tipo UCB exigen que las recompensas estén acotadas en $[0, 1]$, por lo que la puntuación de cada simulación se define como la fracción de tareas completadas en la finalización del *rollout*. Esta puntuación presenta, no obstante, un problema práctico: es discreta, de modo que numerosos planes distintos completan el mismo número de tareas y reciben una puntuación idéntica, lo que produce mesetas en las que la búsqueda carece de criterio para discriminar. Para evitar estas mesetas, se añade un término de desempate que premia arrancar las tareas pronto dentro de su ventana.

$$g \;=\; \frac{|\mathcal{T}_{\text{comp}}|}{m} \;+\; \frac{W_e}{m}\,\overline{\text{ant}}, \qquad \text{ant}_j \;=\; 1 - \frac{t_j^{\text{inicio}} - t_j^{s}}{t_j^{l} - t_j^{s}} \in [0,1], \tag{4.1}$$

donde $\overline{\text{ant}}$ es la anticipación media de las tareas completadas y $m = |\mathcal{T}|$. La elección $W_e < 1$ es esencial: garantiza que este bonus nunca pese más que una tarea completada, de modo que el orden inducido por el objetivo primario se conserva y el término solo actúa dentro de cada meseta. Este factor premia arrancar antes una tarea, que liberará antes a sus trabajadores y dejará más margen para las siguientes.

Urgencia temporal en la política de *rollout*. La Ecuación (2.35) puntúa cada tarea por el cociente entre su valor marginal y el tiempo invertido. La implementación refina esa expresión mediante un factor de urgencia $1 + W_u\,\tau/(\tau + \mathit{slack})$, donde τ es la suma del desplazamiento y la ejecución esperada y $\mathit{slack} = t_j^l - t^{\text{llegada}}$ es el margen restante hasta el cierre de la ventana. El factor tiende a 1 cuando el margen es amplio y crece hasta $1 + W_u$ cuando la tarea está a punto de volverse inalcanzable, priorizando así la ejecución de tareas que se cierran pronto.

Ensanchamiento progresivo con sesgo heurístico. Exigir que todos los hijos de un nodo se expandan antes de comenzar a discriminar entre ellos produce, con más de una docena de acciones y unas pocas decenas de iteraciones por ronda, árboles anchos y con apenas profundidad. En su lugar, el número máximo de hijos se limita a $1 + C_{\text{pw}}\,(N^{\gamma_D})^{\alpha_{\text{pw}}}$, creciendo con las visitas del nodo, y la acción que se expande a continuación es siempre la de mayor puntuación heurística entre las no exploradas, de modo que el presupuesto se concentra en las ramas a priori prometedoras. Sin este sesgo, el orden de expansión venía dado por el identificador de la tarea, un criterio arbitrario que sesgaba sistemáticamente los árboles poco desarrollados.

Eficiencia y restricciones de factibilidad. Un último grupo de decisiones menos relevantes se resume en la búsqueda de simplificaciones de cómputo y de comprobaciones de factibilidad de acciones que permiten simplificar considerablemente los árboles de búsqueda y los tiempos de ejecución. Para la comprobación de la factibilidad de acciones se utiliza un simulador de eventos idéntico al descrito en la Sección 4.4, con la misma cola de eventos y las mismas reglas. Antes de cada iteración, el robot reconstruye a partir de su observación un estado de simulación y una cola inicializada de forma coherente con la situación observada. Sobre esa cola se desarrollan tanto los *rollouts* como la evaluación de qué acciones son factibles en cada nodo del árbol.

4.5.2. La selección de hiperparámetros

La Tabla 4.4 recoge los valores empleados en toda la evaluación, fijados de forma empírica a lo largo del desarrollo comparando configuraciones alternativas sobre diferentes escenarios. El factor de descuento $\gamma_D = 0{,}9999$ se sitúa muy próximo a la unidad: conserva prácticamente toda la información acumulada, con un comportamiento cercano a UCT, pero se mantiene estrictamente menor que 1 para atenuar de forma gradual las simulaciones obsoletas tras un

Cuadro 4.4: Hiperparámetros de la implementación de Dec-MCTS empleada en la evaluación.

Parámetro	**Valor**	**Función**
γ_D	$0{,}9999$	Descuento de las estadísticas D-UCT (2.33)
C_p	$0{,}7$	Constante de exploración D-UCT (2.34)
Iteraciones por ronda	30	Cómputo por cada ronda de planificación de fondo
Iteraciones de emergencia	300	Cómputo inmediato si se pide una decisión con el árbol vacío
W_u	$1{,}0$	Peso de la urgencia temporal en la política de *rollout*
W_e	$0{,}5$	Peso del bono de anticipación en la recompensa (4.1)
C_{pw}	$1{,}5$	Coeficiente del ensanchamiento progresivo
α_{pw}	$0{,}6$	Exponente del ensanchamiento progresivo
$p^{\mathrm{blq}}_{\mathrm{MAX}}$	ρ_j	Descuento por tarea cubierta por la coalición vecina

cambio en las distribuciones de los vecinos. La constante de exploración $C_p = 0{,}7 \approx 1/\sqrt{2}$ satisface la condición teórica $C_p > 1/\sqrt{8}$ exigida por el análisis de convergencia de D-UCT [5, 10]; nótese que, con la formulación de la Ecuación (2.34), el bono efectivo de exploración es $2C_p = 1{,}4$. Por último, las 30 iteraciones por ronda pueden parecer escasas, pero deben leerse junto al intervalo de planificación de la Sección 4.4.2: con una ronda cada $0{,}1$ unidades de tiempo virtual, un robot acumula del orden de centenares de iteraciones entre dos decisiones consecutivas, más las 300 rondas del calentamiento inicial.

Con el sistema descrito, el capítulo siguiente presenta el catálogo de escenarios sobre el que se ejecuta y los criterios con los que fue diseñado.

Casos de estudio

El capítulo anterior ha descrito el sistema con el que se ejecutan los cinco algoritmos: el simulador de eventos discretos, la función de recompensa y las decisiones de implementación que los materializan. Falta describir *sobre qué* se ejecutan. Este capítulo presenta el catálogo de escenarios empleado en la evaluación y justifica los criterios con los que fue construido.

El conjunto de prueba no se ha diseñado con el único propósito de determinar qué algoritmo obtiene la mayor recompensa media, sino con el de poder atribuir las diferencias observadas a propiedades concretas del problema. Un catálogo de escenarios heterogéneos permitiría constatar que un método rinde más que otro, pero no explicar por qué; y en un problema que combina ventanas de ejecución, coaliciones, restricciones de batería e incertidumbre (Sección 2.1.2), esa explicación posee un gran valor. Esta exigencia se traduce en un catálogo de **360 escenarios repartidos en cinco bloques de 72**, donde cada uno de los bloques varía un único aspecto del problema mientras todo lo demás permanece fijado en una configuración de referencia. Las diferencias observadas dentro de un bloque son atribuibles al factor que ese bloque aísla. La Sección 5.1 describe la configuración común a todos ellos y la Sección 5.2, los cinco bloques y el factor que cada uno aísla.

5.1. Configuración común de los escenarios

Todos los escenarios del catálogo comparten una estructura común que se resume en la Tabla 5.1 y que se detalla a continuación. Cada uno de ellos constituye una instancia del problema en el sentido de la Definición 2.1, expresada en el formato YAML que utiliza el simulador (Sección 4.2).

Geometría. El grafo de ubicaciones es completo y las distancias son euclídeas. El nodo 1, situado en el origen, alberga la única estación de recarga y cada uno de los m nodos restantes alberga una tarea. Las tareas se reparten en **cuatro racimos** equidistantes del origen, cuyos centros se sitúan en $(0, 4)$, $(4, 0)$, $(0, -4)$ y $(-4, 0)$. Dentro de cada racimo, las tareas se reparten de forma equiespaciada sobre la circunferencia unidad centrada en él. De este modo, la distancia entre la estación y cualquier tarea está comprendida entre 3 y 5 unidades, mientras

Cuadro 5.1: Configuración común a los 360 escenarios del catálogo.

Elemento	**Valor**
Grafo	Completo, $m+1$ nodos: m nodos de tarea y 1 nodo de estación
Geometría	Tareas repartidas en 4 racimos en torno al centro
Robots	Homogéneos, $\mathit{Cap}(i,j)=1$ para todo par, todos parten de la estación
Cinemática	$v_i=1$ (distancia = tiempo) y *rate*-$R_i=1$
Batería	$B_i=40$, con nivel inicial completo
Estaciones de recarga	Una sola, situada en el origen
Duración de tarea	$\mathcal{N}(10,\sigma_j^s)$ en caso de éxito, el fracaso es inmediato
Consumo por tarea	*rate*-$T_j=1$ por unidad de tiempo de ejecución
Reintentos	Uno: un fracaso es definitivo
Horizonte	$T=40$
Carga por robot	$L=6$
Ventana de ejecución	$[t_j^s,\ t_j^l]=[t_0,\ t_0+10+\varepsilon]$
Coalición de referencia	$q_j=1$

que dos tareas del mismo racimo distan a lo sumo 2. Esta disposición introduce deliberadamente una tensión entre dos comportamientos razonables: agrupar el trabajo dentro de un racimo, que es barato en desplazamiento, o repartir el equipo entre racimos, que cubre más ventanas simultáneamente. Es una decisión que el algoritmo debe tomar y condicionará su rendimiento.

Robots y cinemática. Los robots son idénticos: comparten velocidad, capacidad de batería y tasas de consumo, parten todos de la estación de recarga y son compatibles con todas las tareas, esto es, $\mathit{Cap}(i,j)=1$ para todo par (i,j). No se hace uso de la heterogeneidad que admite el modelo formal. Esta es una simplificación deliberada, coherente con el objeto de estudio del trabajo, que es la coordinación entre robots y no la asignación por competencias. La velocidad se fija en $v_i=1$, de manera que las distancias del grafo se leen directamente como tiempos de desplazamiento, y la tasa de consumo al navegar en *rate*-$R_i=1$ unidad de batería por unidad de tiempo. Como la demanda media de una tarea (10) coincide con su duración media (10), ejecutar y desplazarse cuestan lo mismo por unidad de tiempo: ninguna de las dos actividades resulta más barata que la otra.

Tareas. Toda tarea tiene una duración media de 10 unidades de tiempo en caso de éxito, muestreada de $\mathcal{N}(10,\sigma_j^s)$ y truncada en 0 (solo se consideran valores positivos). Como la tasa de consumo durante la ejecución es unitaria, una ejecución con éxito gasta tantas unidades de batería como unidades de tiempo ha durado. El fracaso, en cambio, se modela como **instantáneo**: una ejecución fallida no consume tiempo, pero sí una cantidad fija de 10 unidades

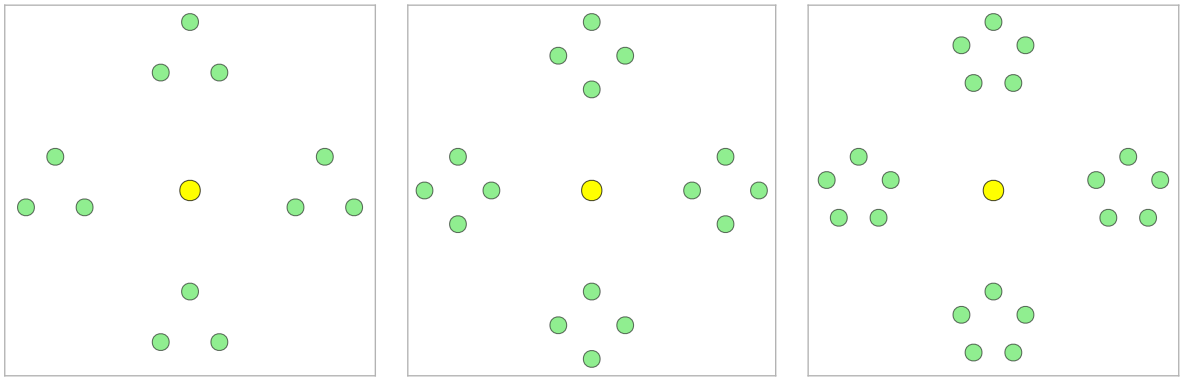


Figura 5.1: Geometría común a todos los escenarios del catálogo, ilustrada sobre tres instancias en su estado inicial con tres, cuatro y cinco tareas por racimo. Las tareas, en verde, se reparten de forma equiespaciada sobre la circunferencia unidad de cada uno de los cuatro racimos, equidistantes de la estación de recarga, en amarillo, situada en el origen. Como el número de racimos es siempre cuatro, un escenario mayor no añade racimos nuevos, sino que puebla más densamente los ya existentes.

de batería. Es decir, el coste de fallar no es la espera, sino la energía gastada y, sobre todo, la oportunidad perdida, pues no hay reintentos: una tarea que fracasa queda definitivamente sin completar. Esta elección concentra el efecto de la incertidumbre en la calidad de la decisión y evita que el fracaso se convierta simplemente en un retraso. A este fracaso por azar se añade un segundo modo, de naturaleza distinta: si la batería del robot no cubre la duración muestreada, la ejecución se interrumpe en el instante en que se agota. La tarea fracasa igualmente y de forma definitiva, pero el coste ya no es el fijo de 10 unidades, sino la energía que le restaba al robot.

La definición de dificultad. La dificultad de un escenario se controla mediante la **carga por robot**

$$L = \frac{1}{n} \sum_{T_j \in \mathcal{T}} q_j, \tag{5.1}$$

esto es, la suma de trabajadores requeridos por tarea entre el número total de robots. Esta cantidad representa el número de tareas que corresponden a cada robot. Todo el catálogo se fija en $L = 6$, fijando además el horizonte en $T = 40$, salvo el bloque que estudia expresamente este factor. El motivo de fijar L y no el número de tareas es que estas dos magnitudes no son intercambiables: si el número de tareas se mantuviera constante al aumentar el equipo, o si el horizonte creciera con él, «más robots» significaría en realidad «menos trabajo por

robot» y cualquier tendencia medida frente al tamaño del equipo estaría midiendo holgura en lugar de capacidad de coordinación. Fijando L y T, el número de tareas de un escenario queda determinado por el tamaño del equipo y por el tamaño medio de coalición $\bar{q}$ mediante $m = Ln/\bar{q}$, y el tamaño del equipo se convierte en un factor aislable.

La restricción de batería. La capacidad de batería se fija en $B_i = 40$ para todos los robots, que además comienzan el episodio con la batería llena. La cifra no es arbitraria. Un ciclo de trabajo típico consiste en salir de la estación hacia un racimo (4 unidades), ejecutar tres tareas (30) desplazándose entre ellas (≈ 2) y regresar a recargar (4), lo que suma unas 40 unidades. La carga inicial alcanza por tanto para la ejecución de aproximadamente tres tareas. Debido a la carga de trabajo por robot descrita anteriormente, cada robot necesita al menos dos ciclos completos para completar su trabajo, de modo que la recarga pasa a ser una parte obligatoria del problema. Esto responde a una exigencia del propio modelo: la Definición 2.2 incluye la factibilidad energética entre las condiciones que toda solución debe satisfacer, y una capacidad holgada dejaría esa condición sin activarse nunca. La única estación de recarga se sitúa además en el origen, coincidiendo con la posición inicial de los robots.

Ventana y coalición de referencia. La configuración de referencia emplea la ventana de ejecución más restrictiva de las consideradas, que denominaremos *estrecha*. Para cada tarea se sortean un instante de apertura $t_0 \sim \mathcal{U}\{0, \ldots, T\}$ y una perturbación $\varepsilon \sim \mathcal{N}(0, 3)$, y la ventana se define como

$$[\, t_j^s,\, t_j^l \,] \;=\; [\, t_0,\;\; t_0 + 10 + \varepsilon \,], \tag{5.2}$$

de anchura media 10, comparable a la duración de la propia tarea. Una ventana así exige que la coalición esté presente en un instante concreto: llegar antes obliga a esperar y llegar tarde inutiliza la tarea. Obsérvese que el horizonte T acota el instante en que las ventanas se *abren*, no la duración del episodio, que termina cuando todos los robots han concluido su misión o agotado su batería. La coalición de referencia es $q_j = 1$, es decir, tareas ejecutables por un solo robot; las variantes con coaliciones mayores se estudian en su bloque correspondiente.

Las fuentes de incertidumbre. Las tres fuentes de incertidumbre introducidas en la Sección 2.1.2 se controlan de forma conjunta mediante el régimen del escenario, recogido en la Tabla 5.2. El régimen fija la probabilidad de éxito ρ_j y la desviación típica σ_j^s de la duración en caso de éxito. El resto de parámetros ($\mu_j^s = 10$, $\mu_j^f = 0$, $\sigma_j^f = 0$) se mantienen siempre

Cuadro 5.2: Regímenes de incertidumbre. El régimen determina la probabilidad de éxito y la dispersión de la duración.

Régimen		ρ_j	σ_j^s
det	Determinista	1,00	0
lev	Incertidumbre leve	0,90	1
est	Incertidumbre media	0,75	3
fue	Incertidumbre fuerte	0,50	5

Cuadro 5.3: Estructura del catálogo. Cada bloque aísla un único factor y aporta 72 escenarios. La columna de estructura indica el diseño factorial que los genera: niveles del factor × tamaños de equipo × regímenes × instancias; en el bloque E el factor es el propio régimen, de ahí que no aparezca como un término independiente.

Bloque	**Factor aislado**	**Niveles**	**Estructura**	**det**	**est**	**Total**
A (control)	Tamaño del equipo n	$2, \ldots, 10$	$9 \times 2 \times 4$	36	36	72
B	Forma de $[t_j^s, t_j^l]$	3 ventanas	$3 \times 3 \times 2 \times 4$	36	36	72
C	Carga por robot L	3, 4, 8	$3 \times 3 \times 2 \times 4$	36	36	72
D	Coalición q_j	$q2$, qm	$2 \times 3 \times 2 \times 6$	36	36	72
E	Régimen ρ_j	1, 0,9, 0,75, 0,5	$4 \times 3 \times 6$	18	54	72
Total				**162**	**198**	**360**

fijos. La tercera fuente de incertidumbre, el consumo energético, no se parametriza de forma independiente: como el gasto durante la ejecución es proporcional a su duración, la variabilidad de esta se traslada directamente al consumo. Los dos regímenes de referencia del catálogo son el determinista (*det*) y el estocástico medio (*est*); los dos restantes, incertidumbre leve (*lev*) y fuerte (*fue*), solo aparecen en el bloque que los estudia.

5.2. Los cinco bloques

Sobre la base anterior, el catálogo se organiza en **cinco bloques de 72 escenarios** cada uno. El bloque A es el **control**: lleva íntegra la configuración de referencia y recorre los nueve tamaños de equipo, de 2 a 10 robots. Los bloques B a E modifican *un único factor* sobre esa referencia y lo hacen para tres tamaños de equipo, $n \in \{2, 5, 10\}$, de modo que cada uno de ellos se compara contra las celdas correspondientes del bloque de control. La Tabla 5.3 resume la estructura resultante.

Los bloques A, B, C y D se reparten **a partes iguales entre el régimen determinista y el estocástico**: cada escenario descrito se instancia en ambos regímenes, lo que permite leer cualquier comparación por separado en uno y otro. Esta simetría es deliberada, pues el efecto

de la incertidumbre sobre los distintos algoritmos es uno de los objetos centrales del estudio y agregar ambos regímenes en una sola cifra podría ocultarlo. El bloque E constituye la **única excepción**: el bloque se centra en el estudio de la incertidumbre y el régimen determinista es uno de sus cuatro niveles, el correspondiente a $\rho_j = 1$. En este bloque, los escenarios se dividen a partes iguales entre los 4 niveles de incertidumbre. La regla se cumple, por tanto, en 288 de los 360 escenarios, y el catálogo completo queda repartido en 162 escenarios deterministas y 198 estocásticos.

Cada combinación de parámetros específica del escenario aparece además en varias **instancias** independientes; cuatro en los bloques A, B y C, seis en los bloques D y E. La diferencia existente entre las instancias de una misma combinación de parámetros aparece en los sorteos de las ventanas de ejecución descritos en la Ecuación (5.2). Estas instancias no son repeticiones de la misma ejecución, sino problemas distintos con una misma configuración y permitirán distinguir un comportamiento sistemático de uno particular de una instancia concreta.

La semilla con la que se generan los sorteos de un escenario se deriva exclusivamente del **tamaño del problema y del número de instancia**, y no del régimen, la forma de la ventana ni la coalición. Dos escenarios con el mismo número de robots, el mismo número de tareas y el mismo índice de instancia comparten, en consecuencia, la geometría y los mismos valores sorteados de t_0 y ε, que se extraen siempre en idéntico orden. La comparación entre dos niveles de una característica es así **pareada tarea a tarea** y no solo escenario a escenario: cuando se contrastan dos formas de ventana, por ejemplo, los plazos de cada tarea son literalmente los mismos números en ambos ficheros, de modo que la diferencia observada solo puede atribuirse al factor variado.

Cada escenario se identifica mediante un nombre de fichero que codifica su posición en el catálogo, con la forma

```
esc_{bloque}_r{robots}_n{tareas}_{ventana}_
    {régimen}_{coalición}_i{instancia}
```

donde `esc_A_r05_n030_E_est_q1_i3` representa la tercera instancia del bloque A con cinco robots y treinta tareas, ventana estrecha, régimen estocástico medio y tareas de un solo trabajador. El catálogo completo se genera de forma determinista a partir de la especificación anterior, de modo que los 360 ficheros pueden reconstruirse íntegramente sin ambigüedad.

Los apartados que siguen describen cada bloque y el factor que aísla.

5.2.1. Bloque A: El tamaño del equipo

El bloque de control recorre los nueve tamaños de equipo, de 2 a 10 robots, con la configuración de referencia completa. Al mantenerse la carga en $L = 6$, el número de tareas acompaña al del equipo ($m = 6n$, entre 12 y 60 tareas), de modo que todos sus escenarios plantean la misma dificultad nominal y las diferencias entre tamaños son atribuibles a la coordinación y no a la holgura. Es el bloque que evalúa cómo responde el rendimiento de los algoritmos al tener que soportar una mayor coordinación, es decir, más robots.

5.2.2. Bloque B: La ventana de ejecución

Este bloque sustituye la ventana estrecha de referencia por otros tres tipos de ventana, construidos a partir de los mismos sorteos (t_0, ε) y representados en la Figura 5.2:

- **Solo plazo**, $[\,0,\ t_0 + 10 + \varepsilon\,]$: conserva *exactamente* los mismos instantes de cierre que la ventana de referencia, pero abre todas las tareas desde el comienzo del episodio.
- **Escalonada**, $[\,0,\ \frac{g}{4}T + \varepsilon\,]$, donde $g \in \{1, 2, 3, 4\}$ identifica el racimo al que pertenece la tarea: los cuatro racimos vencen sucesivamente a un cuarto, la mitad, tres cuartos y la totalidad del horizonte.
- **Ancha**, $[\,0,\ T + 10 + \varepsilon\,]$: el plazo deja de ser una restricción efectiva y el problema se reduce a un problema de recorrido.

El contraste entre la ventana estrecha de referencia y la de solo plazo merece atención particular, porque es el que aísla un factor que la literatura de asignación con ventanas rara vez separa: ambas comparten los mismos plazos y difieren únicamente en si la tarea está disponible desde el instante inicial o hay que esperar a que se abra. Lo que las distingue es, por tanto, la **espera obligatoria**, que introduce un recurso adicional, el tiempo muerto de un robot comprometido con una tarea que aún no puede comenzar, inexistente cuando la ventana solo impone una fecha límite. La ventana escalonada, por su parte, no exige esperar pero sí ordenar el trabajo por urgencia de racimo, y la ancha elimina toda limitación temporal.

5.2.3. Bloque C: La carga por robot

Este bloque evalúa la flexibilidad de los algoritmos en función de la dificultad, medida como la carga por robot. Los niveles $L \in \{3, 4, 8\}$ se recorren manteniendo el horizonte fijo

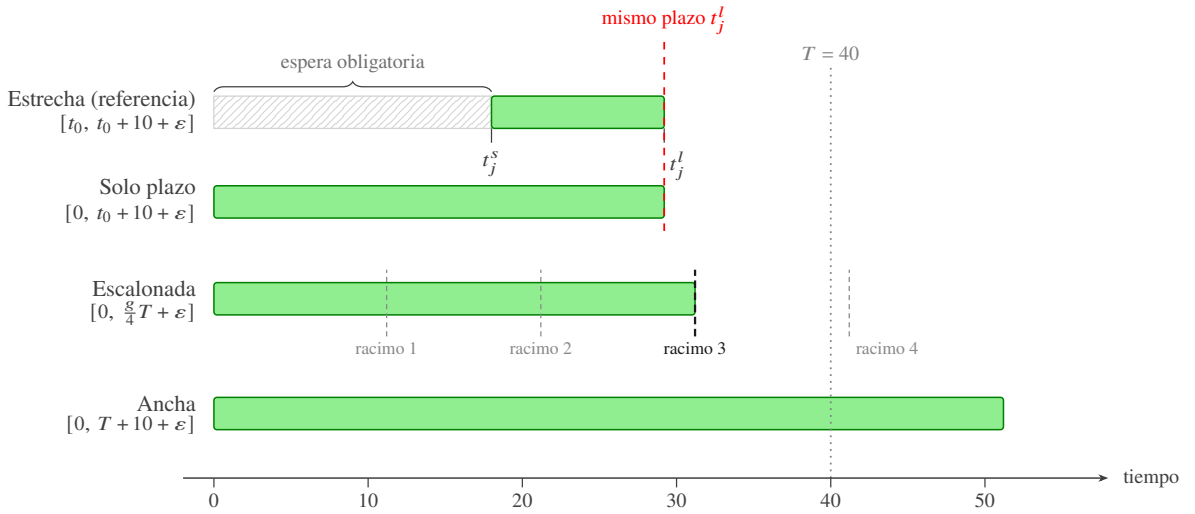


Figura 5.2: Las cuatro tipologías de ventana de ejecución, derivadas de los mismos sorteos t_0 y ε para una misma tarea, perteneciente en el ejemplo al tercer racimo. La ventana estrecha y la de solo plazo cierran *exactamente* en el mismo instante y difieren únicamente en la espera obligatoria que impone la primera, lo que aísla la espera como factor. En la escalonada el plazo depende del racimo: las marcas discontinuas señalan los cuatro cierres posibles y la destacada es la que corresponde a la tarea representada. La ventana ancha no impone acoplamiento temporal alguno.

($T = 40$), lo que sitúa a los escenarios por debajo ($L = 3$, $L = 4$) y por encima ($L = 8$) del punto de operación de referencia. Es el bloque de escenarios más dispares en tamaño, desde 6 hasta 80 tareas, y el que permite comprobar si el orden entre algoritmos se mantiene cuando el problema pasa de holgado a saturado.

5.2.4. Bloque D: Las coaliciones

Las tareas de un solo trabajador se sustituyen aquí por tareas *multi-robot*, en dos variantes: $q2$, en la que todas las tareas requieren exactamente dos robots, y qm, en la que el número de trabajadores sigue un patrón cíclico 1-2-3 con el mismo tamaño medio de coalición, $\bar{q} = 2$. La pareja está construida así a propósito: ambas variantes imponen idéntica carga agregada y difieren solo en su homogeneidad, lo que separa el efecto de *cooperar* del de cooperar en grupos *de tamaño variable*. Para conservar la carga por robot, el número de tareas se reajusta según $m = Ln/\bar{q}$, de modo que estos escenarios tienen la mitad de tareas que los de referencia y cada una de ellas es el doble de pesada; conviene tenerlo presente al interpretar la métrica, pues cada tarea completada vale aquí el doble en la fracción final. Los tamaños de equipo empleados son $n \in \{3, 6, 10\}$, elegidos para que el número de tareas resultante se reparta de

forma equilibrada entre los cuatro racimos.

5.2.5. Bloque E: El nivel de incertidumbre

El nivel completo de incertidumbre se recorre sobre los cuatro regímenes descritos en la Tabla 5.2, desde el determinista hasta la incertidumbre fuerte con $\rho_j = 0{,}5$, para tres tamaños de equipo. Mientras que el resto de bloques contrasta dos regímenes extremos, este permite observar la *transición*: cómo se degrada el rendimiento de cada algoritmo conforme la incertidumbre crece de forma gradual y si esa degradación depende del tamaño del equipo.

Capítulo 6

Experimentación y pruebas

El catálogo descrito en el capítulo anterior reúne **360 escenarios** que suman $10\,896$ tareas y $12\,264$ plazas de trabajador, con equipos de 2 a 10 robots y escenarios de entre 6 y 80 tareas. Cada escenario se resuelve con los **cinco algoritmos** presentados en la Sección 2.4 y, dado que tanto el entorno como los propios algoritmos son estocásticos, cada combinación de escenario y algoritmo se repite en **tres réplicas** independientes cuyo resultado se promedia. La evaluación completa asciende por tanto a

$$360 \text{ escenarios} \times 5 \text{ algoritmos} \times 3 \text{ réplicas} = 5\,400 \text{ ejecuciones}, \tag{6.1}$$

cada una de las cuales produce un registro completo de su traza de eventos (Sección 4.2.2), todos ellos se encuentran en el repositorio online [22]. Las $5\,400$ ejecuciones se lanzaron repartidas en seis procesos independientes sobre el equipo de la Tabla 4.1, con un tiempo total aproximado de 45 minutos.

Este capítulo analiza sus resultados con un doble objetivo: **cuantificar el rendimiento relativo** de los cinco algoritmos y **caracterizar las condiciones bajo las cuales ese rendimiento cambia**. Se presentan todos los cortes que el diseño del catálogo define, con independencia de a qué algoritmo favorezcan, y los criterios de comparación se fijan de antemano en la Sección 6.1 para aplicarse sin excepción. La Sección 6.2 examina el catálogo como un único conjunto; la Sección 6.3 lo desglosa en sus cinco bloques, donde reside el grueso del análisis; y la Sección 6.4 estudia magnitudes distintas de la métrica objetivo.

6.1. Protocolo experimental y criterios de análisis

La unidad es el **escenario**, entendido como la media de sus tres réplicas, de modo que cada escenario aporta una única observación a cada estadístico. La métrica objetivo es la definida en la Sección 4.3: la **fracción de tareas completadas**.

Todas las comparaciones entre algoritmos se realizan **escenario a escenario**, restando sus recompensas sobre el mismo problema antes de promediar y el error estándar se calcula entre escenarios. El motivo es que la variabilidad dominante del catálogo no procede de

Cuadro 6.1: Recompensa media (fracción de tareas completadas) por algoritmo. Las columnas *det* y *est* recogen los 162 escenarios de cada régimen de referencia; las cinco últimas, los 72 escenarios de cada bloque.

	Catálogo	**det**	**est**	**A**	**B**	**C**	**D**	**E**
Random	0,268	0,305	0,242	0,263	0,312	0,353	0,172	0,240
Greedy	0,346	0,396	0,313	0,332	0,413	0,407	0,281	0,297
CBAA	0,401	0,483	0,339	0,359	0,435	0,431	**0,418**	0,362
CBBA	0,459	0,532	0,405	0,466	0,482	0,537	0,391	0,416
Dec-MCTS	**0,502**	**0,586**	**0,443**	**0,499**	**0,537**	**0,628**	0,403	**0,441**

los algoritmos sino de los propios escenarios: la desviación típica de la recompensa entre escenarios se sitúa entre $0{,}10$ y $0{,}15$, un orden de magnitud por encima de las diferencias que se pretenden medir. Al tomar muestras pareadas, esa variabilidad se cancela.

Buena parte del capítulo se centra en el estudio de la diferencia pareada

$$\Delta_e \;=\; r_e^{\text{Dec-MCTS}} \;-\; r_e^{\text{CBBA}}, \tag{6.2}$$

donde $r_e^{\cdot}$ es la recompensa media del algoritmo en el escenario e. Se privilegia esta pareja porque son los dos algoritmos de mayor recompensa media y porque contrapone la propuesta de este trabajo al mejor de los algoritmos de referencia, de modo que su diferencia mide directamente lo que aporta sustituir la puja por una distribución sobre planes. Junto a la media de Δ se estudian su error estándar (EE), el estadístico $t = \Delta/\mathrm{EE}$ y el **recuento de escenarios** con diferencia positiva, nula y negativa, notado $G/E/P$ (Gana/Empata/Pierde).

6.2. Resultados sobre el conjunto del catálogo

6.2.1. Recompensa media

La Tabla 6.1 recoge la recompensa media de los cinco algoritmos sobre el conjunto del catálogo, separando los dos regímenes de referencia y los cinco bloques; la Figura 6.1 la representa gráficamente.

La ordenación sobre el agregado es Random $0{,}268$ < Greedy $0{,}346$ < CBAA $0{,}401$ < CBBA $0{,}459$ < Dec-MCTS $0{,}502$. Su interpretación es directa: añadir una heurística mínima de eficiencia a la elección aleatoria (Random → Greedy) aporta $0{,}078$; introducir comunicación y resolución de conflictos mediante pujas (Greedy → CBAA) aporta otros $0{,}055$; pujar por paquetes en lugar de por tareas sueltas (CBAA → CBBA) aporta $0{,}058$; y sustituir la puja

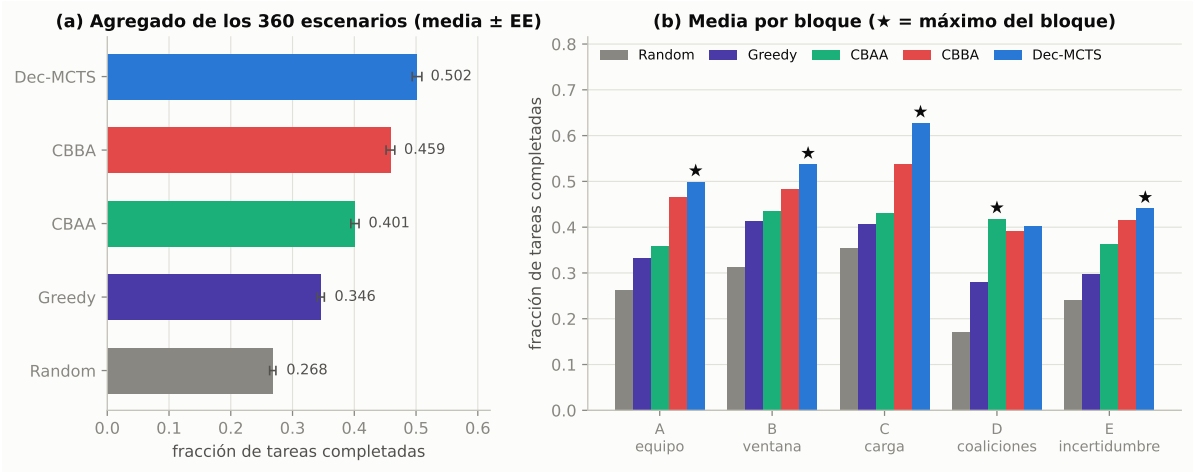


Figura 6.1: Recompensa media por algoritmo. En (a), el agregado de los 360 escenarios con su error estándar entre escenarios. En (b), la misma magnitud calculada dentro de cada bloque; la estrella señala el algoritmo con la media más alta del bloque.

por una distribución sobre planes (CBBA $\rightarrow$ Dec-MCTS) aporta $0{,}043$. Cada escalón de coordinación se traduce en rendimiento medible.

Las dos primeras separaciones son además **estables**: no cambian de signo en ninguno de los cinco bloques. La tercera sí lo hace, y en un único bloque: en el de coaliciones, CBAA supera a CBBA por $0{,}027$ (Sección 6.3.4). La cuarta, que separa a Dec-MCTS de CBBA por $0{,}043$, debe compararse con la desviación típica entre escenarios, de $0{,}10$ a $0{,}15$: es la menor de las cuatro y la que exige un examen más detenido, que ocupa el resto del capítulo. El panel (b) muestra que Dec-MCTS obtiene la media más alta en cuatro de los cinco bloques, y CBAA en el restante.

6.2.2. La diferencia entre CBBA y Dec-MCTS

Pese a que el algoritmo CBAA haya resultado ganador en el bloque D, la diferencia media entre este y los dos primeros es bastante superior a la incertidumbre media por escenario. Es por eso que podemos otorgar de manera robusta la tercera posición a este algoritmo. Es la separación entre los dos primeros (CBBA y Dec-MCTS) la más estrecha de las cuatro y la que más varía de un corte a otro, es por eso que nos centramos en su diferencia. La Figura 6.2 representa la distribución de la diferencia pareada (6.2) y su desglose por régimen, cuyos valores recoge la primera mitad de la Tabla 6.3.

La media sobre el catálogo completo es $\Delta = +0{,}0431 \pm 0{,}0039$, con $t = 10{,}9$. El recuento es de **244 escenarios positivos, 33 nulos y 83 negativos**, la mediana de Δ es $+0{,}037$ y el rango

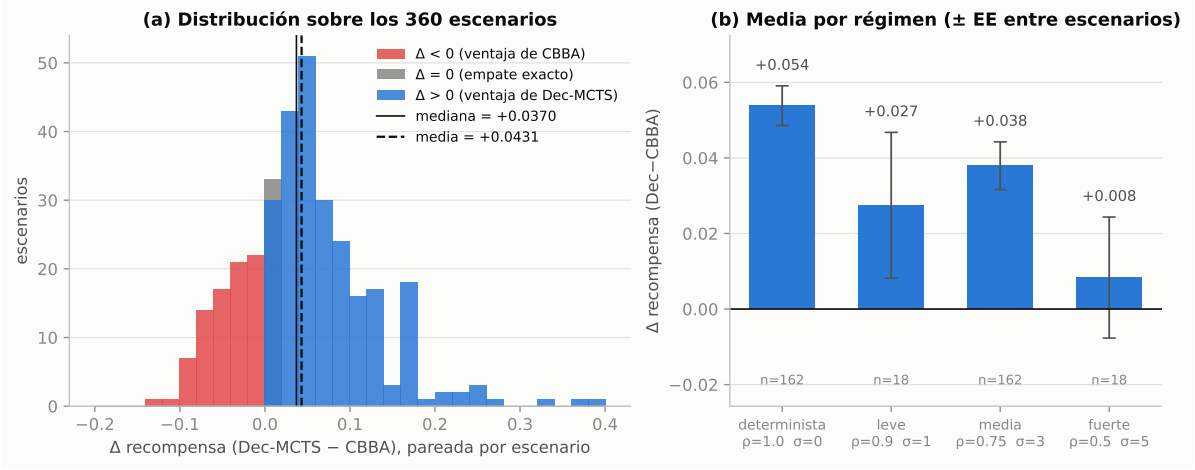


Figura 6.2: Diferencia pareada Δ entre Dec-MCTS y CBBA. En (a), su distribución sobre los 360 escenarios: cada escenario aporta una observación, coloreada según el signo de la diferencia; la línea continua marca la mediana y la discontinua, la media. En (b), la media por régimen de incertidumbre con su error estándar entre escenarios; los regímenes *lev* y *fue* proceden únicamente del bloque E, de ahí su menor número de escenarios y su mayor variabilidad.

intercuartílico es $[0{,}000,\ +0{,}080]$. La distribución está desplazada hacia valores positivos, pero conserva una cola negativa apreciable: en 83 escenarios CBBA obtiene mayor recompensa y en 85 de los 360 la diferencia es inferior a $0{,}02$, esto es, menos de una tarea de cada cincuenta. La ventaja media es, por tanto, clara en el conjunto, pero no se realiza en todos los escenarios.

El desglose por régimen gradúa esa ventaja. La diferencia es positiva en los cuatro regímenes, pero su magnitud se reduce al aumentar la incertidumbre: es máxima y claramente separable de cero en determinista ($+0{,}0538 \pm 0{,}0052$), se mantiene con incertidumbre media ($+0{,}0379 \pm 0{,}0063$) y se anula con incertidumbre fuerte ($+0{,}0083 \pm 0{,}0160$). Los dos regímenes intermedios no son directamente comparables entre sí, porque *lev* procede únicamente del bloque E mientras que *est* recorre los cinco. La dependencia del régimen resulta más informativa que el agregado, como se comprobará a lo largo del capítulo.

La Tabla 6.2 recoge otros dos estadísticos adicionales: el número de escenarios en que cada algoritmo obtiene la **máxima** recompensa de los cinco y el **arrepentimiento medio**, definido como la distancia media al máximo de cada escenario, que cuantifica el coste de fijar un único algoritmo sin conocer de antemano el problema al que se enfrentará.

Dec-MCTS obtiene la máxima recompensa en 190 de los 360 escenarios, más que los otros cuatro algoritmos juntos, y acumula a la vez el menor arrepentimiento ($0{,}018$ frente a $0{,}061$ de CBBA): cuando no es el mejor, queda cerca de serlo. Más llamativo es que CBAA, tercero en la

Cuadro 6.2: Escenarios en los que cada algoritmo obtiene la máxima recompensa de los cinco y arrepentimiento medio, esto es, distancia media al máximo de cada escenario.

	Escenarios con la máxima recompensa	**Arrepentimiento medio**
Random	0	0,252
Greedy	1	0,174
CBAA	89	0,119
CBBA	80	0,061
Dec-MCTS	**190**	**0,018**

Cuadro 6.3: Diferencia pareada Δ entre Dec-MCTS y CBBA, por régimen de incertidumbre y por bloque. $G/E/P$ es el número de escenarios con diferencia positiva, nula y negativa. La última columna indica el valor que tomaría la diferencia sobre el catálogo completo ($+0{,}0431$) si se excluyera ese bloque.

Corte	**#esc.**	$\boldsymbol{\Delta}$	**EE**	$\boldsymbol{t}$	$\boldsymbol{G/E/P}$	**Sin ese bloque**
Catálogo completo	360	$+0{,}0431$	$0{,}0039$	10,9	$244/33/83$	—
Por régimen de incertidumbre						
det ($\rho_j = 1{,}00$)	162	$+0{,}0538$	$0{,}0052$	10,3	$125/16/21$	—
lev ($\rho_j = 0{,}90$)	18	$+0{,}0275$	$0{,}0193$	1,4	$11/2/5$	—
est ($\rho_j = 0{,}75$)	162	$+0{,}0379$	$0{,}0063$	6,0	$101/15/46$	—
fue ($\rho_j = 0{,}50$)	18	$+0{,}0083$	$0{,}0160$	0,5	$7/0/11$	—
Por bloque						
A · tamaño del equipo	72	$+0{,}0330$	$0{,}0061$	5,4	$50/4/18$	$+0{,}0456$
B · ventana	72	$+0{,}0550$	$0{,}0082$	6,7	$56/6/10$	$+0{,}0401$
C · carga por robot	72	$+0{,}0900$	$0{,}0114$	7,9	$58/6/8$	$\mathbf{+0{,}0313}$
D · coaliciones	72	$+0{,}0120$	$0{,}0065$	1,8	$35/12/25$	$+0{,}0508$
E · incertidumbre	72	$+0{,}0250$	$0{,}0077$	3,3	$45/5/22$	$+0{,}0476$

media global, alcance el máximo en más escenarios que CBBA (89 frente a 80), lo que indica que su rendimiento no está uniformemente por debajo, sino concentrado en un subconjunto identificable del catálogo (Sección 6.3.4).

6.2.3. Sensibilidad del agregado a la composición del catálogo

El agregado es una media sobre cinco bloques y, pese a que estos miden factores completamente distintos, sus diferencias apuntan todas en el mismo sentido. La segunda mitad de la Tabla 6.3 recoge Δ dentro de cada bloque y, en su última columna, el valor que tomaría la diferencia global si se excluyera ese bloque del conjunto.

Esta última columna delimita hasta dónde llega el agregado. La diferencia es positiva en los cinco bloques, de modo que **ninguna exclusión cambia su signo**: el valor global oscila entre $+0{,}0313$ (sin el bloque C, el más favorable a Dec-MCTS) y $+0{,}0508$ (sin el bloque D, el único

en que Dec-MCTS no obtiene la media más alta), y en los cinco casos Dec-MCTS conserva la mayor media del catálogo. La ordenación de los dos primeros algoritmos no depende, por tanto, de la composición del conjunto de prueba.

Lo que sí depende de ella es la **magnitud** de la ventaja, que crece un 62 % entre esos dos extremos, y que dentro del catálogo recorre un rango mucho mayor: de $+0{,}0120$ en el bloque de coaliciones a $+0{,}0900$ en el de carga por robot, es decir, de una diferencia inapreciable a una de casi una tarea de cada once. Obliga por ello a una declaración que acompaña a todos los resultados de este capítulo: **el tamaño de la ventaja es una propiedad del conjunto de prueba tanto como de los algoritmos**, y citar únicamente la cifra agregada oculta que en algunas regiones del problema esa ventaja desaparece. Los apartados siguientes localizan esas regiones.

6.3. Resultados por bloque

Cada bloque aísla un único factor sobre la configuración de referencia y, salvo el bloque C, se compara contra las celdas del bloque de control generadas con la misma semilla (Sección 5.2). El pareado se realiza a nivel de escenario de manera que la diferencia observada solo puede atribuirse al factor variado.

6.3.1. Bloque A: El tamaño del equipo

El bloque de control mantiene la carga por robot constante mientras el equipo crece de 2 a 10 robots, de modo que el estadístico a estudiar es la **pendiente de la recompensa frente al tamaño del equipo**, estimada mediante la recta de mínimos cuadrados sobre los 36 escenarios de cada régimen (Figura 6.3).

En régimen **determinista**, solo dos algoritmos convierten robots adicionales en recompensa: CBBA, con pendiente $+0{,}0135$ por robot ($t = 6{,}9$), y Dec-MCTS, con $+0{,}0121$ ($t = 7{,}4$). Las otras tres pendientes son prácticamente nulas (Random $+0{,}0041$, Greedy $+0{,}0038$ y CBAA $-0{,}0055$, ninguna separable de cero), lo que es coherente con su naturaleza: los *baselines* no comunican, y CBAA, que puja por una tarea cada vez, carece de mecanismo para repartir trabajo futuro. La diferencia entre ambas pendientes es inapreciable: la de la propia Δ vale $-0{,}0014$ por robot ($t = -0{,}7$), de modo que la ventaja de Dec-MCTS se mantiene en todo el rango, de $+0{,}062$ con dos robots a $+0{,}035$ con diez.

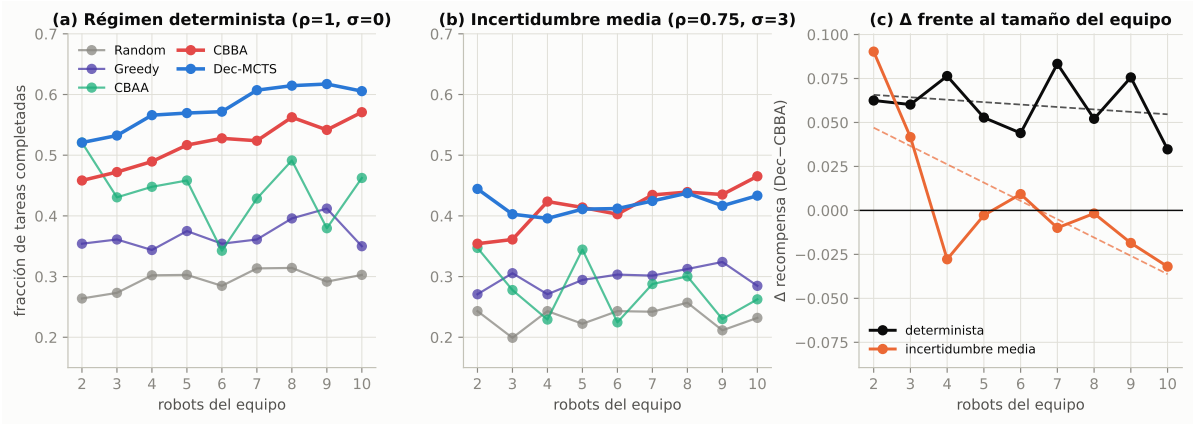


Figura 6.3: Bloque A. Recompensa media frente al tamaño del equipo con carga por robot constante $L = 6$, en régimen determinista (a) y con incertidumbre media (b). El panel (c) representa la diferencia pareada Δ en ambos regímenes, junto con su recta de mínimos cuadrados.

Bajo **incertidumbre media** las dos pendientes se separan. La de CBBA se mantiene en $+0{,}0120$ por robot ($t = 4{,}5$); la de Dec-MCTS cae hasta $+0{,}0016$ ($t = 0{,}7$), indistinguible de cero. La formulación más precisa del resultado es la pendiente de la propia diferencia, en el panel (c): Δ **decrece** $-0{,}0104$ **por robot adicional** ($t = -3{,}5$), pasando de $+0{,}090$ con dos robots a $-0{,}032$ con diez, con cambio de signo en torno a los cuatro. Es decir: la ventaja de Dec-MCTS bajo incertidumbre **no es un nivel constante sino una cantidad que decrece de manera aproximadamente lineal con el tamaño del equipo**, hasta invertirse en los equipos grandes.

Una explicación compatible con esta observación es un exceso de deconflicción: Dec-MCTS descuenta las tareas que estima cubiertas por sus vecinos a partir de las distribuciones sobre planes que estos publican (Sección 2.4.3); si la incertidumbre invalida esos planes la estimación falla, y el número de vecinos sobre los que se condiciona crece con el equipo.

6.3.2. Bloque B: La forma de la ventana de ejecución

Este bloque produce el contraste más nítido del catálogo, porque las tres ventanas alternativas modifican dónde reside la dificultad del problema sin alterar su tamaño. La Figura 6.4 recoge en su panel (a) la recompensa media de los cinco algoritmos en cada una de las cuatro formas de ventana y en su panel (b) la diferencia pareada correspondiente. El eje horizontal de ambos paneles está ordenado **de la ventana menos restrictiva a la más restrictiva**: la ancha, que apenas limita el instante de ejecución; la de solo plazo, que impone un límite superior

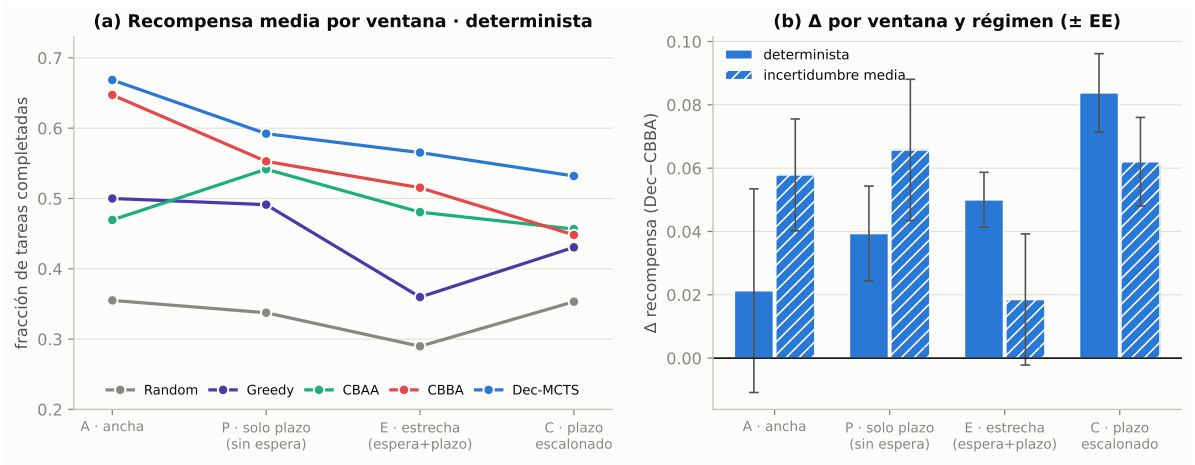


Figura 6.4: Bloque B. En (a), recompensa media de los cinco algoritmos en régimen determinista para cada forma de ventana. En (b), diferencia pareada Δ en esas mismas ventanas, en los dos regímenes y con su error estándar entre escenarios.

pero no obliga a esperar; la estrecha, que añade esa espera; y la escalonada, cuyos plazos son además heterogéneos entre tareas.

El panel (a) muestra que la restricción temporal no afecta por igual a los cinco. **Solo Dec-MCTS y CBBA decrecen de forma monótona** al recorrer el eje, lo que es coherente con que sean los dos únicos que planifican sobre una secuencia de tareas: Dec-MCTS pasa de $0{,}669$ a $0{,}532$ y CBBA de $0{,}647$ a $0{,}448$. Los otros tres describen trayectorias irregulares y mucho más planas. La pérdida al endurecer la ventana es, por tanto, el precio de planificar, y **es mayor para CBBA** ($0{,}199$) **que para Dec-MCTS** ($0{,}137$).

La consecuencia se lee en la separación entre las dos curvas superiores, que el panel (b) cuantifica: en régimen determinista Δ **crece de forma monótona con la restricción**, de $+0{,}021 \pm 0{,}032$ con la ventana ancha a $+0{,}084 \pm 0{,}012$ con el plazo escalonado. En el extremo opuesto las dos curvas casi se juntan ($0{,}669$ frente a $0{,}647$) y la diferencia deja de ser separable de cero. Bajo incertidumbre media la ordenación no se conserva ($+0{,}058$, $+0{,}066$, $+0{,}019$ y $+0{,}062$ recorriendo el mismo eje), de modo que esta monotonía es un resultado del régimen determinista y no del bloque.

Los tres resultados se ordenan según la complejidad existente en los plazos de resolución de las tareas: cuando los plazos son heterogéneos y el orden de ejecución condiciona el resultado, un método que simula secuencias completas dispone de información que una puja no utiliza; cuando la ventana deja de ser una restricción efectiva, esa ventaja se reduce. Dec-MCTS demuestra aquí su mayor capacidad de adaptación a entornos más complejos, donde el orden

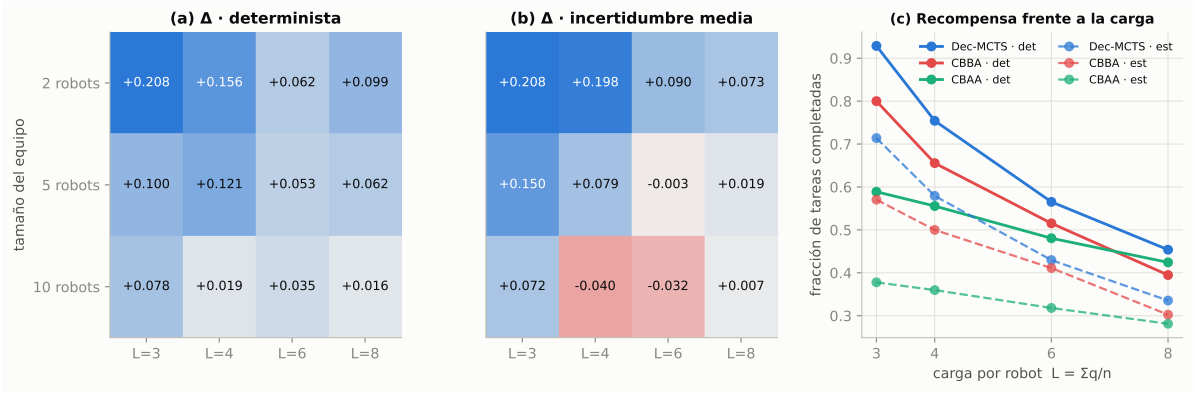


Figura 6.5: Bloque C. Diferencia pareada Δ en la rejilla de tamaño de equipo $\times$ carga por robot, en régimen determinista (a) y con incertidumbre media (b): el azul señala $\Delta > 0$ y el rojo $\Delta < 0$, con intensidad proporcional a la magnitud. La columna $L = 6$ corresponde al control del bloque A. El panel (c) recoge la recompensa media de los tres algoritmos coordinados frente a la carga.

en que se seleccionan incluso las primeras tareas puede arrastrar una penalización hasta la resolución de las últimas.

6.3.3. Bloque C: La carga por robot

La rejilla completa de tamaños de equipo y cargas por robot, representada en los dos primeros paneles de la Figura 6.5, muestra un comportamiento claro. La diferencia **decrece de forma monótona al aumentar el número de robots** en las ocho columnas de la rejilla, sin excepción, mientras que a lo largo del eje de la carga la tendencia es decreciente pero no monótona. Los extremos son $+0{,}208$ (dos robots, $L = 3$, en los dos regímenes) y $-0{,}040$ (diez robots, $L = 4$, incertidumbre media); solo tres de las veinticuatro celdas son negativas, todas con incertidumbre y con cinco o diez robots.

En niveles absolutos, recogidos en el panel (c), con $L = 3$ y régimen determinista Dec-MCTS alcanza $0{,}929$ frente a $0{,}800$ de CBBA; con $L = 8$ ambas medias se aproximan ($0{,}454$ y $0{,}395$) y bajo incertidumbre la separación se reduce igualmente, de $+0{,}144$ con $L = 3$ a $+0{,}033$ con $L = 8$. Una posible explicación es que la saturación reduce el margen en que cualquier política puede diferenciarse: cuando la carga supera lo que el horizonte permite completar, una fracción creciente de la recompensa se pierde con independencia de la asignación.

Cuadro 6.4: Bloque D. Recompensa media de los tres algoritmos coordinados y diferencia pareada Δ por tipo de coalición. La fila $q_j = 1$ es el control del bloque A. Las coaliciones de tamaño variable (qm) mantienen $\bar{q} = 2$, idéntico al de la coalición uniforme.

Régimen	Coalición	CBAA	CBBA	Dec-MCTS	$\Delta \pm$ EE
det	$q_j = 1$ (control)	0,412	0,524	**0,570**	$\mathbf{+0{,}046 \pm 0{,}009}$
	$q_j = 2$ uniforme	**0,443**	0,407	0,400	$-0{,}007 \pm 0{,}015$
	q_j variable (qm)	0,509	0,496	**0,524**	$\mathbf{+0{,}028 \pm 0{,}009}$
est	$q_j = 1$ (control)	0,255	0,410	**0,416**	$+0{,}006 \pm 0{,}013$
	$q_j = 2$ uniforme	**0,334**	0,288	0,303	$+0{,}015 \pm 0{,}014$
	q_j variable (qm)	**0,387**	0,373	0,385	$+0{,}013 \pm 0{,}014$

6.3.4. Bloque D: Las coaliciones

El bloque de coaliciones produce los dos resultados menos previsibles del catálogo, recogidos en la Tabla 6.4.

El primero es que **CBAA obtiene la media más alta del bloque** ($0{,}418$ frente a $0{,}403$ de Dec-MCTS y $0{,}391$ de CBBA), alcanzando el máximo en 46 de sus 72 escenarios. Es el único bloque del catálogo en que la subasta de tarea única supera a la de paquete y el único en que Dec-MCTS no obtiene la media más alta, invirtiendo un orden que se mantiene en todos los demás. Una explicación coherente con lo que el bloque introduce es que el algoritmo CBAA es capaz de explotar la mejor estrategia en estos casos, asignar grupos de dos robots y mantenerlos a lo largo de toda la ejecución, mientras que la incertidumbre introducida con las distribuciones de Dec-MCTS no permite aprovechar tan bien esta estrategia.

El segundo es que **la coalición uniforme anula la ventaja de Dec-MCTS y la variable no**, pese a compartir tamaño medio de coalición y carga por robot: la uniforme da $-0{,}007 \pm 0{,}015$ en régimen determinista, frente al $+0{,}046 \pm 0{,}009$ de su control, mientras que la variable conserva $+0{,}028 \pm 0{,}009$. El efecto de la uniforme se acentúa con el equipo, desde $+0{,}019$ con tres robots hasta $-0{,}048$ con diez. Conviene señalar que este efecto **aparece ya en el régimen determinista**, donde los demás contrastes no muestran desventaja alguna para Dec-MCTS.

6.3.5. Bloque E: El nivel de incertidumbre

El último bloque recorre los cuatro regímenes de la Tabla 5.2 para tres tamaños de equipo, lo que permite observar la transición completa en lugar de únicamente dos extremos (Figura 6.6).

Los cinco algoritmos decrecen de forma monótona a lo largo del gradiente, pero no al mismo ritmo. Entre $\rho_j = 1$ y $\rho_j = 0{,}5$, **CBAA pierde el** $57{,}9$ **% de su recompensa, la mayor**

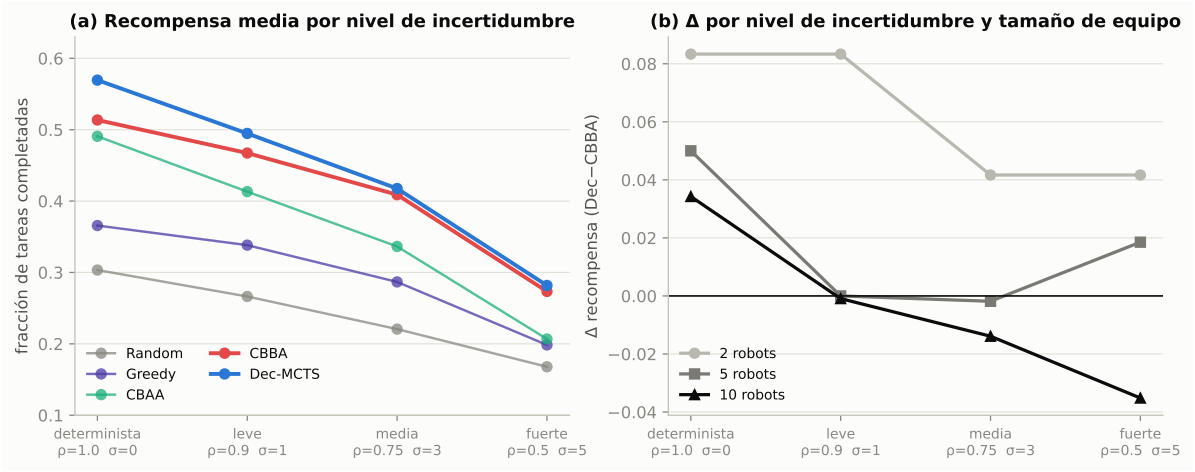


Figura 6.6: Bloque E. En (a), recompensa media de los cinco algoritmos en cada uno de los cuatro niveles del gradiente de incertidumbre. En (b), diferencia pareada Δ en esos mismos niveles, desglosada por tamaño de equipo. Cada nivel del gradiente reúne 18 escenarios.

caída de los cinco, y Random el 44,7 **%, la menor**; Dec-MCTS pierde el 50,5 %, CBBA el 46,8 % y Greedy el 45,7 %. Dec-MCTS conserva la media más alta en los cuatro niveles, pero su margen se estrecha de forma continua: de $+0{,}056$ en el extremo determinista (0,569 frente a 0,514 de CBBA) a $+0{,}008$ en el de incertidumbre fuerte (0,282 frente a 0,273), donde deja de ser separable de cero. La erosión de la ventaja tiene una explicación estructural que conviene recalcar, porque es la contrapartida del resultado del bloque B: **una puja compromete un precio, no una trayectoria**; cuando el plan de un vecino se desvía, la puja sigue siendo válida y se recalcula en la siguiente ronda de consenso, mientras que condicionar simulaciones sobre trayectorias muestreadas deja de serlo.

El desglose por tamaño de equipo del panel (b) aporta el resultado más informativo del bloque: **la incertidumbre y el tamaño del equipo interactúan**. Con dos robots, Δ es positiva en los cuatro niveles de ρ_j y no baja de $+0{,}042$; con diez, recorre el camino contrario, de $+0{,}034$ en determinista a $-0{,}035$ con incertidumbre fuerte, y las tres curvas se separan de forma creciente con la incertidumbre. Ninguno de los dos factores por separado explica el patrón observado, lo que concuerda con la pendiente medida en el bloque A.

6.4. Análisis de magnitudes secundarias

La métrica objetivo resume el resultado pero no describe cómo se alcanza. Esta sección examina magnitudes adicionales obtenidas de la traza de eventos (Sección 4.2.2) que no inter-

vienen en la recompensa y que, por tanto, caracterizan el comportamiento de cada algoritmo sin alterar su rendimiento.

6.4.1. Robots que agotan la batería

Un robot que agota la batería deja de ser útil para el resto del episodio. Es un **modo de fallo distinto de la mala asignación** y por eso se contabiliza por separado.

Los cinco algoritmos sufren pérdidas, y su orden no es el del rendimiento. Random pierde $2{,}79$ robots por escenario, Greedy $2{,}54$, **CBBA** $2{,}51$, CBAA $1{,}83$ y **Dec-MCTS** $1{,}48$, la cifra más baja de los cinco; el 80 % de sus escenarios registra alguna baja, frente al 94 % de Random y al 91 % de CBBA. Sobre el bloque A, donde la dificultad se mantiene constante al crecer el equipo, la fracción de flota afectada ordena igual: **CBBA el** 55 **%**, Random el 45 %, CBAA el 37 %, Greedy el 35 % y **Dec-MCTS el** 30 **%**. CBBA es, con diferencia, el que más apura de los tres algoritmos coordinados, lo que resulta coherente con su mecanismo: sus paquetes encadenan tareas y comprometen al robot con una secuencia cuyo tramo final de vuelta a la estación no entra en la comprobación de factibilidad.

El desglose por régimen separa a Dec-MCTS del resto de manera más nítida todavía. Sus bajas **decrecen** al aumentar la incertidumbre —de $2{,}04$ por escenario en determinista a $0{,}83$ con incertidumbre fuerte—, mientras que las de Random y Greedy crecen ($2{,}53 \rightarrow 3{,}61$ y $2{,}31 \rightarrow 2{,}94$) y las de CBBA oscilan entre $2{,}37$ y $3{,}11$ sin tendencia clara. Una posible explicación es que la reserva de energía de las subastas es un umbral fijo calculado sobre la demanda esperada, insensible por construcción a la dispersión de las duraciones, mientras que Dec-MCTS estima el consumo simulando trayectorias completas y por tanto lo sobreestima justo cuando la dispersión crece.

6.4.2. Descomposición de la recompensa

La fracción de tareas completadas se factoriza de forma exacta en dos magnitudes observables,

$$\underbrace{\frac{|\mathcal{T}_{\mathrm{comp}}|}{m}}_{\text{recompensa}} = \underbrace{\frac{|\mathcal{T}_{\mathrm{int}}|}{m}}_{\text{tasa de intento}} \times \underbrace{\frac{|\mathcal{T}_{\mathrm{comp}}|}{|\mathcal{T}_{\mathrm{int}}|}}_{\text{tasa de éxito}}, \tag{6.3}$$

donde $\mathcal{T}_{\mathrm{int}}$ es el conjunto de tareas que el equipo llega a emprender y $\mathcal{T}_{\mathrm{comp}}$ el que llega a completar. La primera mide la **cobertura** y la segunda el **acierto** de lo emprendido; separarlas

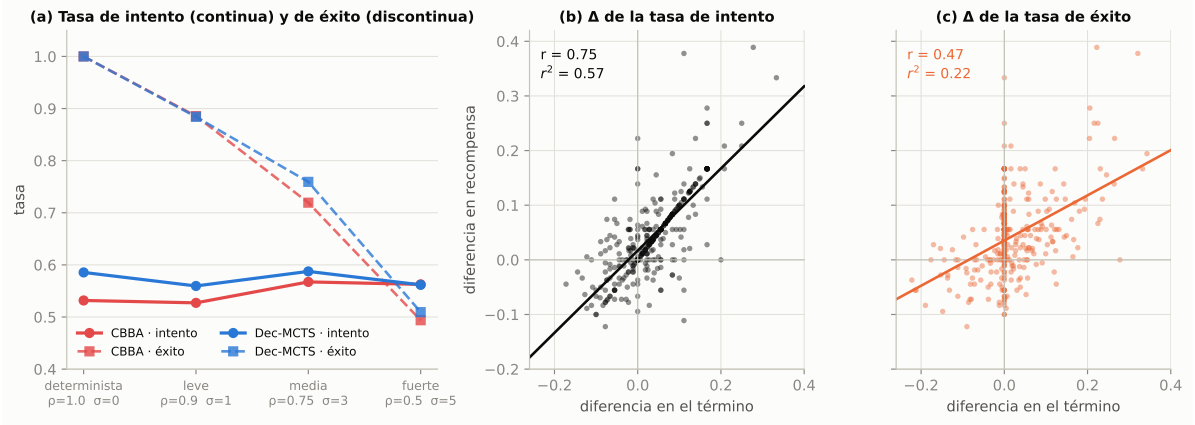


Figura 6.7: Descomposición (6.3) de la recompensa. En (a), ambas tasas para los dos algoritmos de cabeza a lo largo de los cuatro regímenes: línea continua para la tasa de intento y discontinua para la de éxito. En (b) y (c), un punto por escenario que enfrenta la diferencia en cada término con la diferencia en recompensa, con su recta de mínimos cuadrados y su coeficiente de correlación; los dos paneles comparten escalas.

indica si un algoritmo pierde recompensa por una mala elección o por no llegar a intentarlo. El panel (a) de la Figura 6.7 recoge ambas a lo largo del gradiente de incertidumbre.

Los paneles (b) y (c) responden a una pregunta distinta: cuál de los dos términos acompaña a la diferencia de recompensa. Su construcción es la siguiente. En cada uno de los 360 escenarios se calculan tres diferencias pareadas entre Dec-MCTS y CBBA, una por magnitud: ΔR en la recompensa, ΔI en la tasa de intento y ΔE en la tasa de éxito. Cada escenario aporta así un punto $(\Delta I, \Delta R)$ al panel (b) y un punto $(\Delta E, \Delta R)$ al panel (c); ambos paneles comparten escalas para que las dos nubes puedan compararse directamente. Sobre cada una se ajusta una recta por mínimos cuadrados y se anota su coeficiente de correlación de Pearson r, junto con r^2, que es la fracción de la varianza de ΔR que la nube explica.

El coeficiente de correlación mide en qué grado **la variación de la brecha de recompensa entre escenarios** acompaña a la variación de cada término: un r alto significa que los escenarios en los que un algoritmo aventaja al otro en ese término son, en buena medida, los mismos en los que le aventaja en recompensa. Debe tenerse en cuenta además que la relación (6.3) es una identidad y que la recompensa es el producto de ambos términos, de modo que los dos correlacionan con ella por construcción: lo informativo no es el valor absoluto de cada coeficiente, sino la comparación entre ambos.

La descomposición muestra que los dos términos se relevan según el régimen. En **determinista**, la tasa de éxito vale exactamente la unidad para los dos algoritmos y toda la diferencia vive en la **cobertura**: Dec-MCTS emprende el $58{,}6$ % de las tareas frente al $53{,}2$ % de CBBA.

Cuadro 6.5: Distancia recorrida, ocupación del horizonte y coste de cómputo por algoritmo, promediados sobre los 360 escenarios. El instante medio de retirada promedia el momento en que cada robot da por concluida su misión y regresa a la estación; los robots que agotan la batería no intervienen en él, de modo que los escenarios en que ningún robot llega a retirarse quedan excluidos del promedio. El tiempo de cómputo procede de una tanda con seis procesos en paralelo: es comparable entre algoritmos, pero no representa el coste de una ejecución aislada.

	Distancia por robot	**Distancia por tarea completada**	**Instante medio de retirada**	**Tiempo (s/ejec.)**
Random	15,04	15,27	51,31	0,0008
Greedy	9,21	6,72	50,72	0,0008
CBAA	10,91	6,14	35,02	0,0011
CBBA	11,85	6,33	44,91	0,0075
Dec-MCTS	18,23	8,34	**54,22**	13,96

Al introducir incertidumbre la ventaja de cobertura se desvanece (con incertidumbre media, 0,588 frente a 0,567, esto es, 17,48 tareas distintas por escenario frente a 17,51) y es el **acierto** el que pasa a sostener la diferencia: de las tareas que emprende, Dec-MCTS completa el 75,9 % frente al 71,9 % de CBBA. Con incertidumbre fuerte la cobertura llega a invertirse (0,562 frente a 0,563) y el acierto (+0,015) es lo único que queda, lo que explica que la diferencia de recompensa se anule en ese régimen.

En cuanto a qué término acompaña a la variación de la diferencia entre escenarios, los paneles (b) y (c) responden con claridad: la nube de la tasa de intento se agrupa en torno a su recta con $r = 0{,}75$ ($r^2 = 0{,}57$), mientras que la de la tasa de éxito está mucho más dispersa, con $r = 0{,}47$ ($r^2 = 0{,}22$). La cobertura explica algo más de la mitad de la variación de la brecha entre escenarios y el acierto algo más de una quinta parte.

Las dos lecturas no se contradicen, porque responden a preguntas distintas: la cobertura es lo que más varía de un escenario a otro, pero *en media* deja de aportar ventaja en cuanto hay incertidumbre. Una interpretación compatible con ambas, que este capítulo no contrasta, es que la anticipación de Dec-MCTS le permite encajar más tareas en las ventanas mientras los planes propios y ajenos se cumplen, y que la incertidumbre anula esa capacidad sin afectar al criterio con que elige cada tarea.

6.4.3. Distancia, uso del horizonte y coste de cómputo

Distancia. Dec-MCTS recorre un 53,8 % más de distancia por robot que CBBA, y la diferencia relativa aumenta con la incertidumbre. Normalizada por unidad de tarea resuelta,

Dec-MCTS paga $8{,}34$ unidades de distancia por tarea completada frente a $6{,}33$ de CBBA y $6{,}14$ de CBAA, el valor más bajo de los cinco. Es el precio de su ventaja, y no es gratuito, porque desplazarse consume batería y agotarla retira al robot del episodio. Parte de ese recorrido adicional es lo que sostiene su mayor cobertura, y explica por qué es también el algoritmo que más prolonga su actividad. Random ilustra el caso opuesto: necesita $15{,}27$ unidades por tarea completada, en torno al doble de los demás, porque buena parte de ese recorrido termina en tareas fallidas o en robots sin batería.

Uso del horizonte. El instante medio en que los robots se retiran (por concluir su plan o por no disponer de tareas alcanzables) resulta revelador. Dec-MCTS es el que más prolonga su actividad ($56{,}19$ en régimen determinista y $52{,}48$ con incertidumbre media) y CBAA el que antes la abandona ($39{,}84$ y $31{,}62$), con una caída que se acentúa a lo largo del gradiente hasta $23{,}41$ con incertidumbre fuerte. Esa retirada temprana surge de robots que dejan de encontrar tareas alcanzables dentro de plazo y quedan inactivos. El *makespan* del último robot activo, en cambio, apenas discrimina entre algoritmos: se mantiene entre $52{,}6$ y $54{,}9$ en cuatro de los cinco, y solo Dec-MCTS lo estira hasta $59{,}2$.

Coste de cómputo. Los tiempos de ejecución separan a los algoritmos en dos grupos: los *baselines* y los algoritmos basados en pujas tardan escasos milisegundos en ejecutarse. Dec-MCTS tarda de media $13{,}96$ segundos, unas $1\,900$ veces el coste de CBBA (el segundo más costoso). Esta gran diferencia supone un comportamiento esperable: el cálculo de rollouts, el número de iteraciones por llamada y la naturaleza *anytime* del algoritmo elevan considerablemente el coste de ejecución. Este tiempo de ejecución evoluciona con el tamaño del equipo: sobre el bloque A, el tiempo de Dec-MCTS crece de $0{,}59$ segundos con dos robots a $40{,}26$ con diez. Debe precisarse que ese coste es lineal en el presupuesto de iteraciones por ronda, hiperparámetro fijado durante el desarrollo (Tabla 4.4) y no una propiedad del método.

6.5. Síntesis de los resultados

1. **El aumento de coordinación se traduce en rendimiento.** La ordenación Random $<$ Greedy $<$ CBAA $<$ CBBA $<$ Dec-MCTS es amplia y estricta escenario a escenario: Dec-MCTS supera a Random en 359 de 360 escenarios, a Greedy en 351, a CBAA en 265 y a CBBA en 244; CBBA supera a CBAA en 215. La única inversión del catálogo

es la de CBAA sobre CBBA en el bloque de coaliciones.

2. **Dec-MCTS obtiene la mayor recompensa media del catálogo** ($+0{,}0431 \pm 0{,}0039$ sobre CBBA, $244/33/83$), y esa ordenación no depende de la composición del conjunto de prueba: la diferencia es positiva en los cinco bloques y en los cuatro regímenes, y excluir cualquier bloque la deja entre $+0{,}0313$ y $+0{,}0508$. Lo que sí depende de la composición es su tamaño, que va de $+0{,}0120$ en el bloque de coaliciones a $+0{,}0900$ en el de carga.

3. **La ventaja se erosiona con la incertidumbre y con el tamaño del equipo.** Pasa de $+0{,}0538 \pm 0{,}0052$ en determinista a $+0{,}0083 \pm 0{,}0160$ con incertidumbre fuerte. A dificultad constante y bajo incertidumbre, CBBA convierte cada robot adicional en $+0{,}0120$ de recompensa ($t = 4{,}5$) mientras que la pendiente de Dec-MCTS, $+0{,}0016$ ($t = 0{,}7$), no se distingue de cero; la diferencia entre ambos decrece $-0{,}0104$ por robot ($t = -3{,}5$) y cambia de signo en torno a los cuatro robots. Los dos factores interactúan: con dos robots la ventaja se sostiene en todo el gradiente de incertidumbre y con diez se invierte.

4. **Los dos términos de la recompensa se relevan según el régimen.** En determinista la ventaja es íntegramente de cobertura (tasa de intento $0{,}586$ frente a $0{,}532$, con tasa de éxito unitaria en ambos); bajo incertidumbre la cobertura se iguala y es el acierto el que la sostiene ($0{,}759$ frente a $0{,}719$ con incertidumbre media). Entre escenarios, sin embargo, es la cobertura la que más acompaña a la variación de la diferencia ($r = 0{,}75$ frente a $r = 0{,}47$).

5. **Hay regiones del problema bien delimitadas en las que la ventaja es máxima**: plazos escalonados ($+0{,}084$, positiva en 12 de 12 escenarios deterministas), carga holgada (hasta $+0{,}208$) y equipos pequeños; y otras en las que se anula o se invierte: coaliciones uniformes ($-0{,}007$ en determinista), equipos de diez robots bajo incertidumbre y ventanas anchas con equipos grandes.

6. **Dec-MCTS es también el que menos robots pierde**: $1{,}48$ bajas por batería y escenario, frente a las $2{,}51$ de CBBA y las $1{,}83$ de CBAA, y es el único cuyas bajas *decrecen* al aumentar la incertidumbre. La contrapartida está en el recorrido: un $53{,}8$ % más de distancia por robot que CBBA y un $31{,}9$ % más por tarea completada.

7. **El coste de ejecución es asumible**: pese a costar unas $1\,900$ veces más tiempo de cómputo que CBBA, el algoritmo Dec-MCTS implementado (con la selección de hiperparámetros descrita) ha demostrado tener un coste de ejecución asumible. Un escenario con diez robots tiene un tiempo de ejecución medio de 40 segundos en un equipo modesto (realizando en paralelo el cómputo de los diez robots además del cómputo de simulación). Esto responde a una necesidad de diseño ya que los algoritmos evaluados están pensados para ejecutarse en tiempo real a bordo de robots con recursos limitados.

Conviene, por último, distinguir lo medido de lo interpretado. Los resultados anteriores son medidas; las explicaciones apuntadas a lo largo del capítulo (el descuento excesivo de tareas que se creen cubiertas, la diferencia entre comprometer un precio y comprometer una trayectoria, o el carácter binario de la decisión de bloqueo) son **hipótesis compatibles con los datos** que este capítulo no contrasta. El Capítulo 7 retoma esa distinción para valorar qué aportan estos resultados al objetivo del trabajo y qué líneas abren.

Capítulo 7

Conclusiones y trabajo futuro

El propósito de este trabajo era llevar el problema de asignación de tareas multi-robot bajo incertidumbre desde el planteamiento centralizado del que parte [25], en el que un único planificador decide con acceso al estado global del sistema, hasta un esquema **puramente distribuido** en el que no existe coordinador ni estado compartido y cada robot decide de forma autónoma a partir de su información local y de los mensajes que recibe de sus compañeros. Junto a esta transición se formuló la hipótesis de partida: que el algoritmo de mayor grado de coordinación de los considerados, Dec-MCTS, debía rendir por encima de los cuatro algoritmos de referencia.

El desarrollo del trabajo ha recorrido las cinco etapas necesarias para contrastar esa hipótesis. El Capítulo 2 formalizó el problema como una progresión de modelos de decisión que relaja sucesivamente el tiempo discreto, el determinismo y la observabilidad global, hasta alcanzar el DEC-POGSMDP (Sección 2.3.3), y definió sobre ese modelo los cinco algoritmos comparados. El Capítulo 3 situó esa formalización y esos algoritmos frente a la literatura existente. El Capítulo 4 describió la realización del sistema: el simulador distribuido de eventos discretos (Sección 4.4), la función de recompensa (Sección 4.3) y las decisiones de implementación de Dec-MCTS (Sección 4.5). El Capítulo 5 construyó el conjunto de prueba, un catálogo de 360 escenarios repartidos en cinco bloques, cada uno de los cuales aísla un factor del problema manteniendo el resto fijado. Y el Capítulo 6 analizó el resultado de las $5\,400$ ejecuciones que componen la evaluación.

La respuesta a la hipótesis de partida es afirmativa: **Dec-MCTS obtiene la mayor recompensa media del catálogo**, por delante de los cuatro algoritmos de referencia, y lo hace de forma consistente a lo largo de los cuatro regímenes de incertidumbre y de cuatro de los cinco bloques. Este capítulo valora qué aportan estos resultados al objetivo del trabajo. La Sección 7.1 recoge los logros y aportaciones del trabajo; la Sección 7.2, sus limitaciones y el alcance exacto de lo que puede afirmarse; y la Sección 7.3, las posibles líneas de continuación que los resultados abren.

7.1. Logros y aportaciones principales

La primera aportación del trabajo consiste en la definición formal del problema. La Definición 2.8 incorpora a un DEC-POSMDP el mecanismo de eventos y relojes ocultos del GSMP, obteniendo un modelo que describe simultáneamente el tiempo continuo, la duración estocástica de las acciones y la información parcial de cada agente. Esa combinación resuelve el problema de la fuga temporal de información, esto es, que la política de un robot pueda condicionarse a instantes de finalización que todavía no se han producido. Su materialización en el simulador, que muestrea el desenlace y la duración de una tarea en el instante en que el evento se dispara y los mantiene ocultos hasta su consumo, es lo que garantiza que la evaluación respete el modelo implementado.

La segunda aportación es conceptual. Los cinco algoritmos de la Sección 2.4 no se han presentado como métodos inconexos reunidos por conveniencia, sino como una secuencia de grados crecientes de coordinación que se distinguen por su nivel de comunicación: nada en Random y Greedy, una única tarea en CBAA, un paquete ordenado de tareas en CBBA y una distribución de probabilidad sobre planes completos en Dec-MCTS. Esta lectura permite atribuir las diferencias de rendimiento observadas a la riqueza de la información intercambiada. Los resultados la respaldan, ya que la ordenación medida reproduce con exactitud el escalonamiento previsto y cada escalón de coordinación aporta una mejora de magnitud comparable.

Junto a lo anterior, el trabajo entrega el sistema que produce los resultados, y no solo los resultados: un simulador de eventos discretos en el que cada robot ejecuta su propio solver sin acceso al estado global, los cinco algoritmos implementados sobre una interfaz común y un catálogo de escenarios diseñado para atribuir diferencias a factores concretos del problema. Este último merece mención propia porque su construcción condiciona todo lo que de él se deduce.

El principal logro de este trabajo es la construcción del algoritmo Monte Carlo Tree Search descentralizado. El punto de partida es el Dec-MCTS de Best et al. [5], formulado para un problema de percepción activa multi-robot que comparte con el MRTAU una estructura esencial pero con rasgos concretos completamente distintos. La construcción de este algoritmo es por tanto un logro en sí mismo. Pero además este algoritmo ha demostrado ser también el que mejor resuelve el problema. Sobre el catálogo completo, la diferencia pareada frente a CBBA (el segundo con mejores resultados) es de $+0{,}0431 \pm 0{,}0039$ ($t = 10{,}9$), con un recuento de

244 escenarios ganados, 33 empatados y 83 perdidos de los 360 evaluados (Sección 6.2.2). Esa ventaja se mantiene positiva en los cuatro regímenes de incertidumbre y en cuatro de los cinco bloques, por lo que define un comportamiento sistemático a lo largo del conjunto.

El estudio no se limita a evaluar cuál es el mejor algoritmo, sino que también se pregunta cuánto de la conclusión procede del algoritmo y cuánto del conjunto elegido. La conclusión que este trabajo sostiene es que la ordenación es robusta al diseño del conjunto de prueba, mientras que su magnitud no es una constante del método sino una función de las condiciones del problema.

Tomadas en conjunto, estas evidencias permiten afirmar que un método de búsqueda en árbol de Monte Carlo puede llevarse a un esquema puramente distribuido, con decisión local y coordinación emergente de la comunicación, resultando ganador frente a los algoritmos distribuidos de referencia.

7.2. Limitaciones

La principal limitación encontrada es que la ventaja de Dec-MCTS no es uniforme, sino que se reduce con la incertidumbre y con el tamaño del equipo. A lo largo del gradiente de incertidumbre pasa de $+0{,}0538 \pm 0{,}0052$ en régimen determinista a $+0{,}0083 \pm 0{,}0160$ con incertidumbre fuerte, valor que ya no se distingue de cero. A dificultad constante y bajo incertidumbre, CBBA convierte cada robot adicional en $+0{,}0120$ de recompensa, mientras que la pendiente de Dec-MCTS es $+0{,}0016$. Ambos factores interactúan, de manera que la ventaja se sostiene en todo el gradiente con equipos de dos robots y se invierte con equipos de diez. Existe, pues, una región del espacio de problemas, la de los equipos grandes bajo incertidumbre elevada, en la que el método no es la mejor opción disponible. La explicación apuntada en el capítulo anterior, según la cual el descuento por tareas que se suponen cubiertas introduce un sesgo que se acumula con el número de vecinos, es una hipótesis compatible con los datos que este trabajo no ha contrastado.

A ello se añade que existe un bloque en el que el algoritmo no lidera. El de coaliciones es la única inversión de la ordenación de toda la evaluación, con CBAA alcanzando $0{,}418$ frente a los $0{,}403$ de Dec-MCTS y los $0{,}391$ de CBBA (Sección 6.3.4). Este comportamiento parece surgir de aquellos escenarios en los que todas las tareas exigen dos trabajadores. Es aquí donde Dec-MCTS tiene peor rendimiento. La posible explicación descrita en el capítulo anterior es que CBAA es capaz de asignar grupos de dos robots y mantenerlos a lo largo de

toda la ejecución mientras que Dec-MCTS no suele aprovechar tan bien esta estrategia.

El rendimiento obtenido se paga, además, en dos recursos. El primero es el desplazamiento, pues Dec-MCTS recorre un $53{,}8\ \%$ más de distancia por robot que CBBA y un $31{,}9\ \%$ más por tarea completada; ese recorrido es el precio de su mayor cobertura y resulta rentable mientras el desplazamiento no sea el recurso que limita el sistema. El segundo es el cómputo, del orden de $1\,900$ veces el de CBBA, si bien en términos absolutos resulta asumible (unos 40 segundos en un escenario de diez robots, con el cómputo de los diez en paralelo).

Conviene, por último, delimitar el alcance de las conclusiones anteriores.

- Los resultados proceden de *un* catálogo, *una* familia de problemas y *una* implementación concreta de cada algoritmo, por lo que su extrapolación a otras variantes de MRTA no está garantizada.
- Las explicaciones ofrecidas en el capítulo anterior y en este son interpretaciones consistentes con lo observado, no hipótesis contrastadas experimentalmente.
- La comparación con el modelo centralizado de partida no se ha medido sobre este catálogo, de modo que la pérdida asociada a distribuir la decisión permanece sin cuantificar.

7.3. Trabajo futuro

Las limitaciones anteriores señalan con bastante precisión cuáles son los siguientes pasos. Las posibles ramas de trabajo futuro se describen a continuación.

- **Refinamiento de la política de comunicación y del criterio de decisión de Dec-MCTS.** La versión evaluada deja identificadas varias direcciones de refinamiento, ninguna de ellas contrastada. Una primera mejora, sugerida por el bloque de coaliciones, es enriquecer la comunicación con una noción de coalición, de modo que un robot pueda comunicar que acudirá a una tarea si otros se suman, en lugar de la decisión binaria que hoy toma la probabilidad de bloqueo p_j^{blq}. Una segunda afecta al criterio con que se puntúan las simulaciones, que podría atender a la contribución *marginal* del robot al objetivo global en lugar de al objetivo completo, o ponderar la información recibida de un vecino por la incertidumbre que este le atribuya. Son hipótesis de diseño plausibles y coherentes con lo observado, pero la evaluación realizada no permite anticipar si alguna de ellas se traduciría en una mejora de rendimiento, ni descartar que introduzcan efectos adversos.

- **Cuantificación de la pérdida de rendimiento frente al planificador centralizado de referencia.** Este trabajo evalúa el desempeño del algoritmo de búsqueda en árbol descentralizado propuesto frente a los algoritmos distribuidos de referencia, pero no cuantifica cuánto se paga por prescindir del estado global, porque el modelo centralizado de partida [25] no se ha evaluado sobre este catálogo. Ejecutarlo sobre los mismos escenarios, con la misma función de recompensa y un presupuesto de cómputo comparable, convertiría esa afirmación cualitativa en una cifra y proporcionaría, además, una referencia superior frente a la que situar a los cinco algoritmos distribuidos.
- **Validación sobre conjuntos de prueba externos.** El catálogo empleado se diseñó para aislar factores del problema, no para compararse con la literatura, y ha sido construido por el propio autor, lo que acota el alcance de sus conclusiones. Evaluar los cinco algoritmos sobre un conjunto de instancias estándar, como las de Solomon para el VRPTW [27], permitiría comprobar si los resultados sobreviven a un conjunto ajeno y situarlos frente a los publicados por terceros. La adaptación no es inmediata, ya que esas instancias carecen de coaliciones, de restricciones de batería y de incertidumbre en el desenlace, de modo que habría que extenderlas y declarar con precisión qué se añade, al precio de debilitar la comparabilidad con los resultados publicados sobre ellas.

Bibliografía

[1] Christopher Amato, George Konidaris, Leslie P. Kaelbling, and Jonathan P. How. Modeling and planning with macro-actions in decentralized pomdps. *Journal of Artificial Intelligence Research*, 64:817–859, 2019.

[2] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine learning*, 47(2-3):235–256, 2002.

[3] Richard Bellman. *Dynamic Programming*. Princeton University Press, Princeton, NJ, 1957.

[4] Daniel S. Bernstein, Robert Givan, Neil Immerman, and Shlomo Zilberstein. The complexity of decentralized control of markov decision processes. *Mathematics of Operations Research*, 27(4):819–840, 2002.

[5] Graeme Best, Oliver M. Cliff, Timothy Patten, Ramgopal R. Mettu, and Robert Fitch. Dec-mcts: Decentralized planning for multi-robot active perception. *The International Journal of Robotics Research*, 38(2–3):316–337, 2019.

[6] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.

[7] Han-Lim Choi, Luc Brunet, and Jonathan P. How. Consensus-based decentralized auctions for robust task allocation. *IEEE Transactions on Robotics*, 25(4):912–926, 2009.

[8] Shushman Choudhury, Jayesh K. Gupta, Mykel J. Kochenderfer, Dorsa Sadigh, and Jeannette Bohg. Dynamic multi-robot task allocation under uncertainty and temporal constraints. In *Proceedings of Robotics: Science and Systems (RSS)*, 2020.

[9] M. Bernardine Dias, Robert Zlot, Nidhi Kalra, and Anthony Stentz. Market-based multirobot coordination: A survey and analysis. *Proceedings of the IEEE*, 94(7):1257–1270, 2006.

[10] Aurélien Garivier and Eric Moulines. On upper-confidence bound policies for non-stationary bandit problems. In *International Conference on Algorithmic Learning Theory (ALT)*, pages 174–188. Springer Berlin Heidelberg, 2011.

[11] Brian P. Gerkey and Maja J. Matarić. Sold!: Auction methods for multirobot coordination. *IEEE Transactions on Robotics and Automation*, 18(5):758–768, 2002.

[12] Brian P. Gerkey and Maja J. Matarić. A formal analysis and taxonomy of task allocation in multi-robot systems. *The International Journal of Robotics Research*, 23(9):939–954, 2004.

[13] Peter W. Glynn. A GSMP formalism for discrete event systems. *Proceedings of the IEEE*, 77(1):14–23, 1989.

[14] Ronald A. Howard. *Dynamic Probabilistic Systems, Volume II: Semi-Markov and Decision Processes*. John Wiley & Sons, New York, 1971.

[15] simon hunt, qinggang meng, chris hinde, and tingwen huang. a consensus-based grouping algorithm for multi-agent cooperative task allocation with complex requirements. *cognitive computation*, 6(3):338–350, 2014.

[16] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.

[17] Alaa Khamis, Ahmed Hussein, and Ahmed Elmogy. Multi-robot task allocation: A review of the state-of-the-art. In *Cooperative Robots and Sensor Networks 2015*, volume 604 of *Studies in Computational Intelligence*, pages 31–51. Springer, Cham, 2015.

[18] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *Proceedings of the 17th European Conference on Machine Learning (ECML)*, pages 282–293, 2006.

[19] G. Ayorkor Korsah, Anthony Stentz, and M. Bernardine Dias. A comprehensive taxonomy for multi-robot task allocation. *The International Journal of Robotics Research*, 32(12):1495–1512, 2013.

[20] Harold W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.

[21] Ernesto Nunes, Marie Manner, Hakim Mitiche, and Maria Gini. A taxonomy for task allocation problems with temporal and ordering constraints. *Robotics and Autonomous Systems*, 90:55–70, 2017.

[22] Manuel Olías López. DEC-MRTAU: implementación, escenarios y material experimental. `https://github.com/manuolias/DEC-MRTAU`, 2026. Licencia MIT.

[23] Frans A. Oliehoek and Christopher Amato. *A Concise Introduction to Decentralized POMDPs*. SpringerBriefs in Intelligent Systems. Springer, 2016.

[24] Shayegan Omidshafiei, Ali-akbar Agha-mohammadi, Christopher Amato, Shih-Yuan Liu, Jonathan P. How, and John Vian. Decentralized control of multi-robot partially observable markov decision processes using belief space macro-actions. *The International Journal of Robotics Research*, 36(2):231–258, 2017.

[25] Ignacio Pérez-Hurtado, Jesús Capitán, Luis Merino, and Pedro U. Lima. Online Monte-Carlo Tree Search for Multi-Robot Task Allocation in Uncertain Domains requiring Recharging Operations. In preparation, 2026.

[26] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, New York, 1994.

[27] Marius M. Solomon. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2):254–265, 1987.

[28] Paolo Toth and Daniele Vigo, editors. *Vehicle Routing: Problems, Methods, and Applications*. MOS-SIAM Series on Optimization. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 2 edition, 2014.

[29] Håkan L. S. Younes and Reid G. Simmons. Solving generalized semi-Markov decision processes using continuous phase-type distributions. In *Proceedings of the 19th National Conference on Artificial Intelligence (AAAI)*, pages 742–747, 2004.